Centralized Log Aggregation and Monitoring using ELK Stack

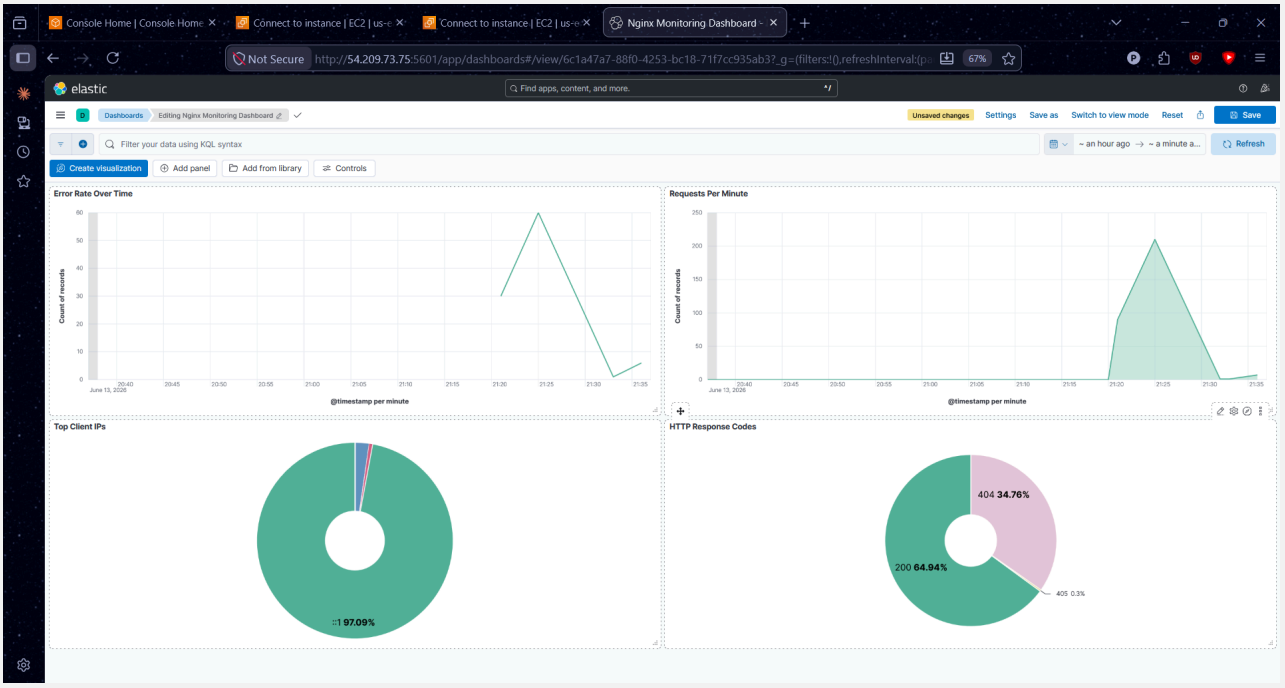
> A logging setup on AWS where Filebeat collects Nginx logs from multiple EC2 instances, Logstash processes them, and the logs are stored in Elasticsearch. Everything can then be viewed through a single Kibana dashboard, with alerts configured to detect spikes in errors.

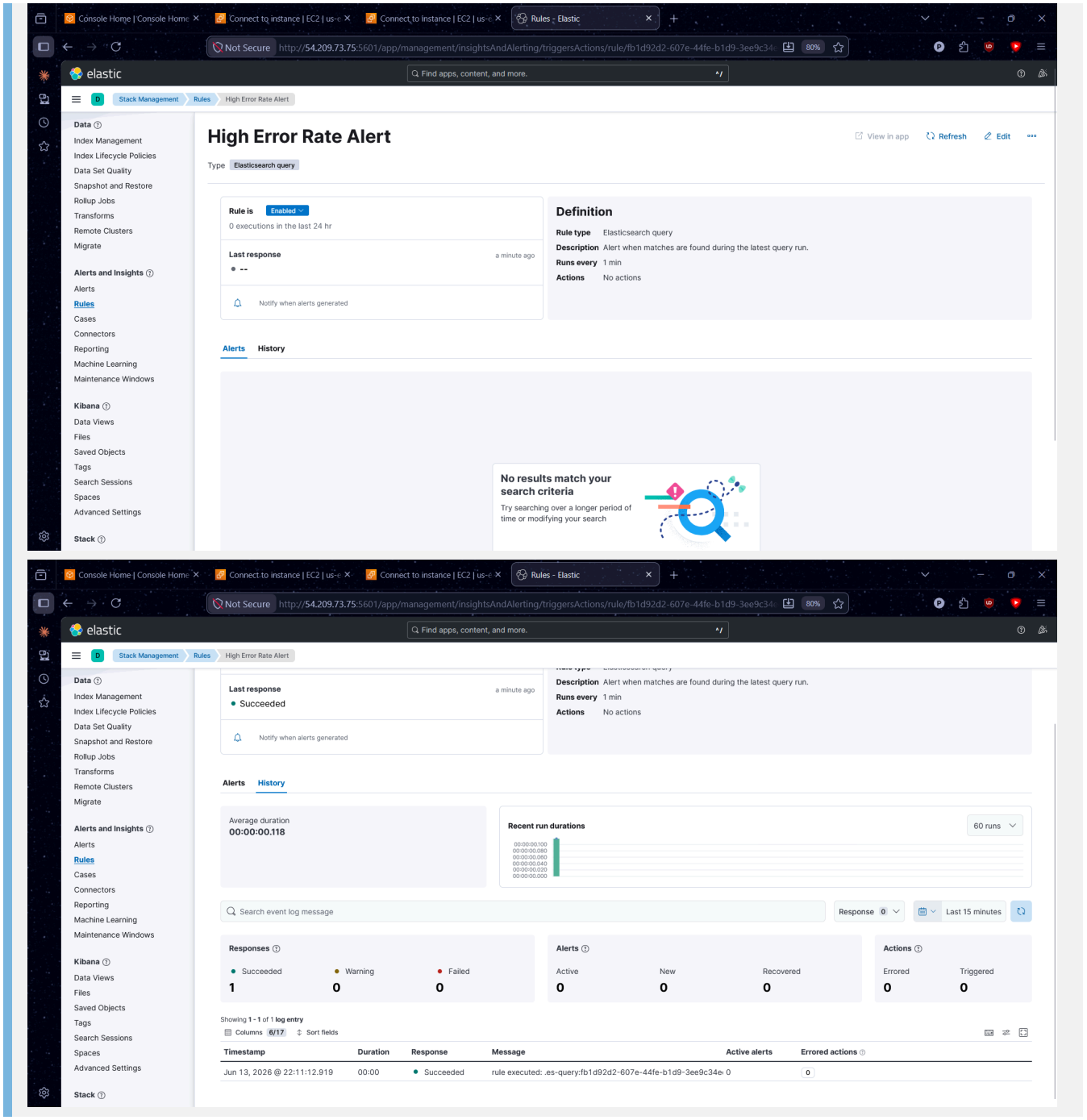
> It's not truly real-time, but the logs are processed quickly enough to give near-real-time visibility and help catch issues before they become bigger problems.



Project Preview

> **Kibana Nginx Monitoring Dashboard** showing real-time error rate, requests per minute, top client IPs, and HTTP response code breakdown from both application servers.





📖 Introduction

When something breaks in production, the logs you need are often spread across multiple servers. Figuring out what happened can mean SSHing into each server, searching through log files, and trying to piece everything together. It's slow, and a simple investigation can take much longer than it should.

That's the problem ELK is designed to solve - bringing logs from different servers into one place so you don't have to search through each machine whenever something goes wrong.

I built this project mainly to understand how that pipeline works end to end, rather than just reading about it. Filebeat collects the Nginx logs from each server, Logstash processes and tags them, Elasticsearch stores and indexes the data, and Kibana provides a central place to search and visualize everything. I also added alerts for spikes in error rates so the system can flag potential issues without someone having to constantly watch the dashboard.

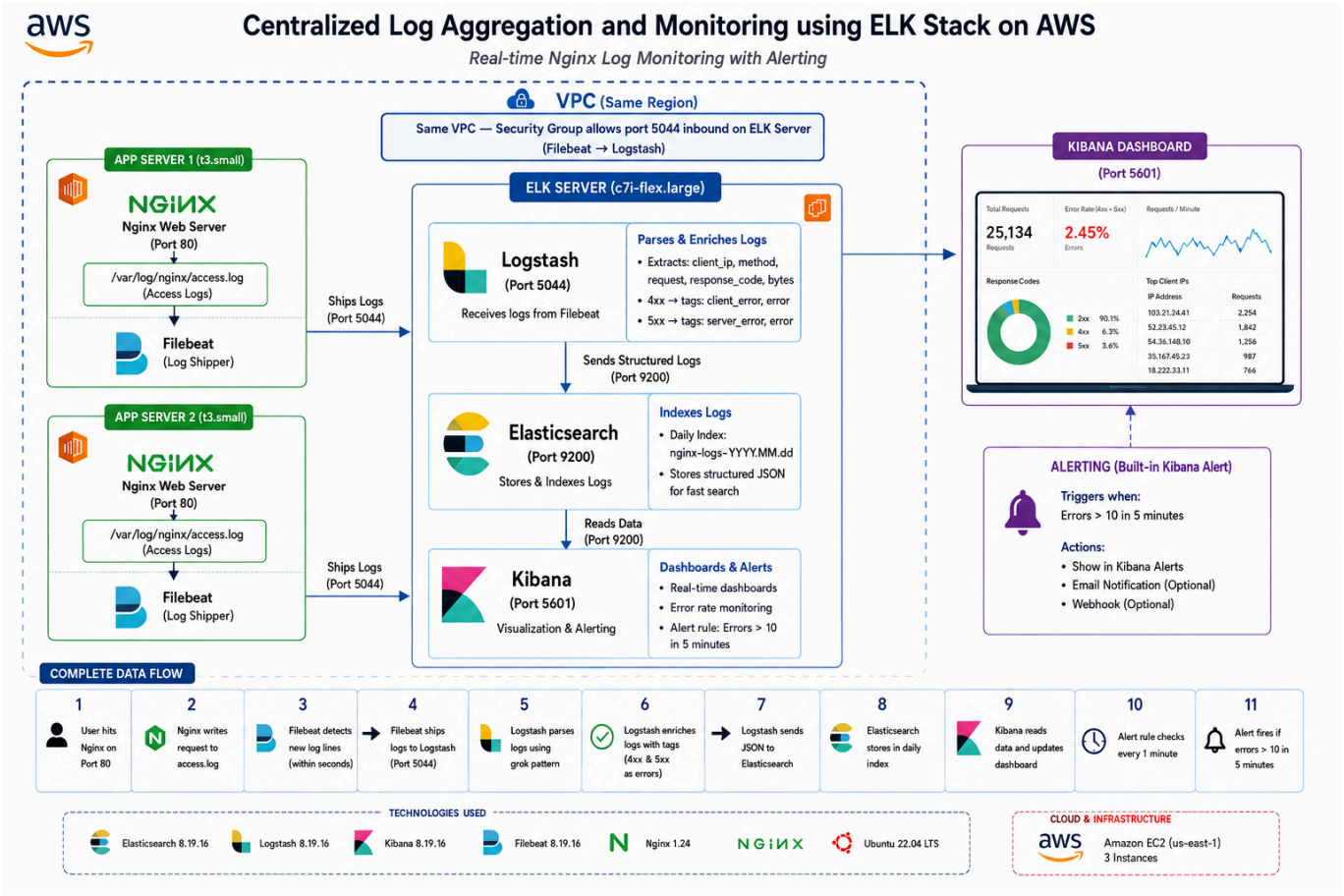
Project Overview

This project builds a complete centralized log aggregation system with 3 EC2 instances — one ELK server running all three Elastic components, and two application servers running Nginx and Filebeat — with Kibana dashboards and alerting for error detection.

What	Details
Type	Centralized Log Aggregation and Monitoring
Role	Site Reliability Engineer
Servers	3 EC2 instances (1 ELK + 2 App servers)
Stack	Elasticsearch + Logstash + Kibana + Filebeat + Nginx
OS	Ubuntu Server 26.04 LTS
Region	us-east-1 (N. Virginia)
Status	Completed

Architecture

```
App Server 1 (t3.small)
  Nginx → /var/log/nginx/access.log
  Filebeat → ships logs
                    ↓
                    ↓ port 5044
                    ↓
            ELK Server (c7i-flex.large)
              Logstash  → parses + tags errors
              Elasticsearch → stores + indexes
              Kibana → dashboard + alerts
                    ↑
                    ↑ port 5044
                    ↑
App Server 2 (t3.small)
  Nginx → /var/log/nginx/access.log
  Filebeat → ships logs
```



Project Structure

```
elk-stack-monitoring/
├── README.md
├── image/
│   ├── 01-ec2-instances-running.png
│   ├── 02-elk-security-group.png
│   ├── 03-app-security-group.png
│   ├── ... (all 21 screenshots)
│   └── 21-all-services-running.png
├── config/
│   ├── elasticsearch.yml
│   ├── logstash/
│   │   └── nginx-pipeline.conf
│   └── filebeat/
│       └── filebeat.yml
└── dashboard/
    └── nginx-monitoring-dashboard.ndjson
```

Complete Data Flow

```
1. User hits Nginx on port 80
2. Nginx writes request to /var/log/nginx/access.log
```

```
3. Filebeat watches log file — detects new lines within seconds
4. Filebeat ships raw log lines to Logstash on port 5044
5. Logstash grok pattern parses each line:
   Extracts: client_ip, method, request, response_code, bytes
6. Logstash tags 4xx responses as "client_error" and "error"
7. Logstash tags 5xx responses as "server_error" and "error"
8. Logstash sends structured JSON to Elasticsearch on port 9200
9. Elasticsearch stores in daily index: nginx-logs-YYYY.MM.dd
10. Kibana reads from Elasticsearch — dashboard updates in real time
11. Alert rule checks every 1 minute — fires if errors > 10 in 5 min
```

Infrastructure

Server	Instance Type	Services	Purpose
elk-server	c7i-flex.large	Elasticsearch, Logstash, Kibana	Central log processing and visualization
app-server-1	t3.small	Nginx, Filebeat	Application server — generates logs
app-server-2	t3.small	Nginx, Filebeat	Application server — generates logs

---

Security Groups

ELK Server (elk-sg):

Port	Protocol	Source	Purpose
22	SSH	My IP	SSH access
9200	TCP	0.0.0.0/0	Elasticsearch API
5044	TCP	0.0.0.0/0	Logstash Beats input
5601	TCP	My IP	Kibana UI
80	HTTP	0.0.0.0/0	Nginx web traffic

Part 1 — EC2 Instances

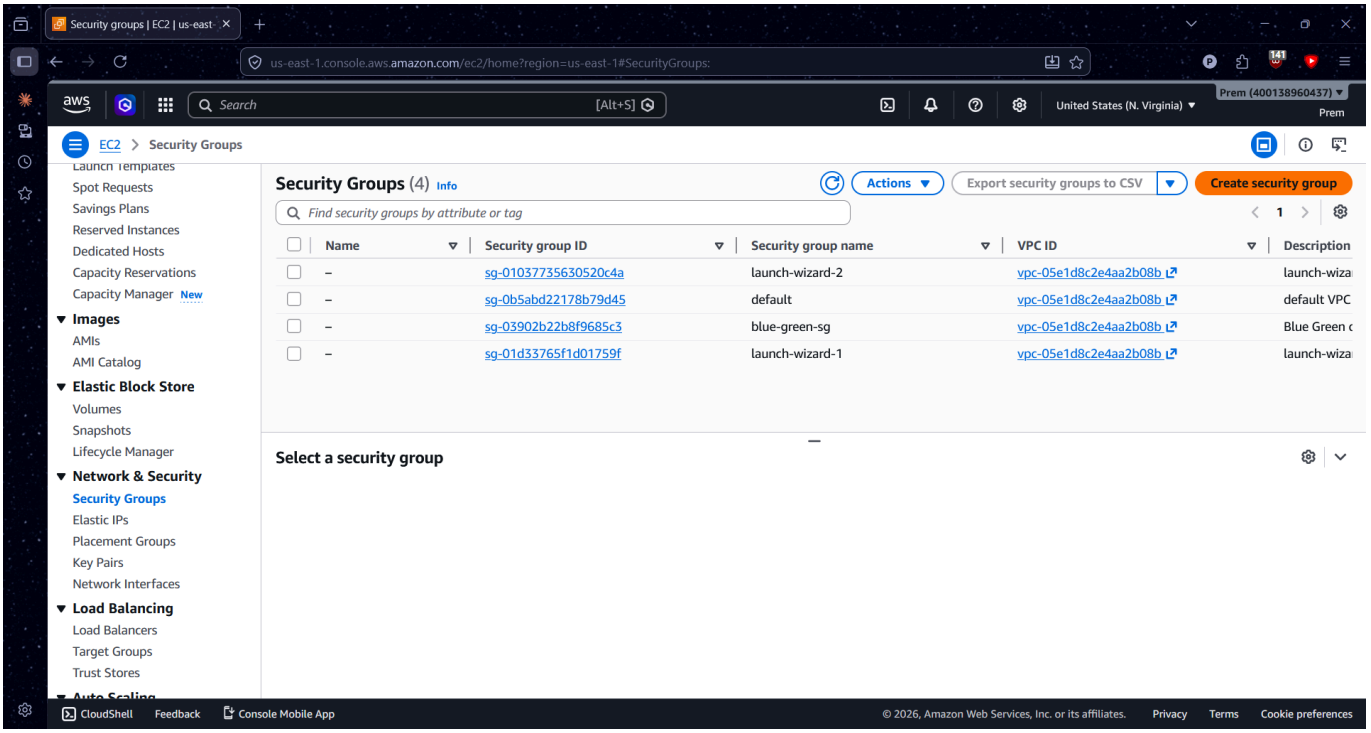
Launch all 3 instances in us-east-1 with correct security groups.

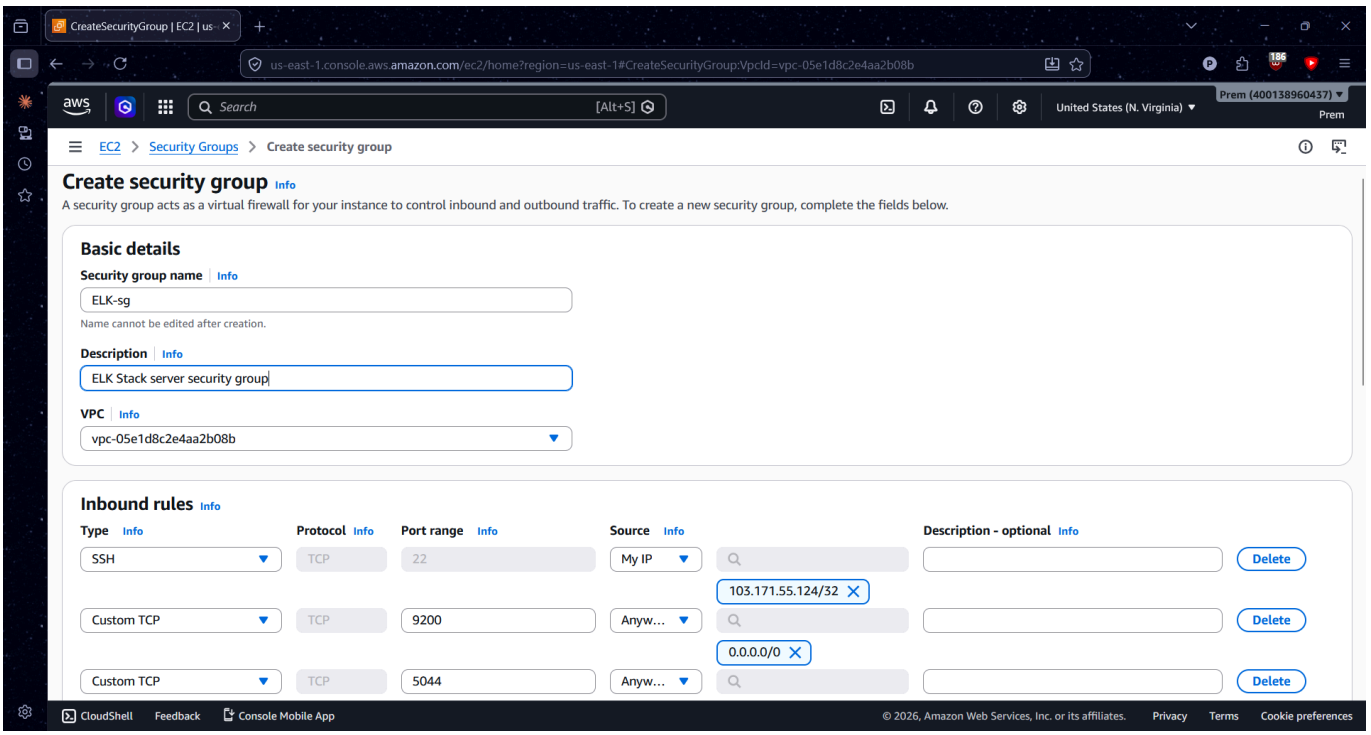
Instance 1 — ELK Server: → AMI: Ubuntu 26.04 LTS → Instance type: c7i-flex.large (since t2.medium was not available on free tier account) → Storage: 20 GB ← increase during launch → Security group: ELK-sg

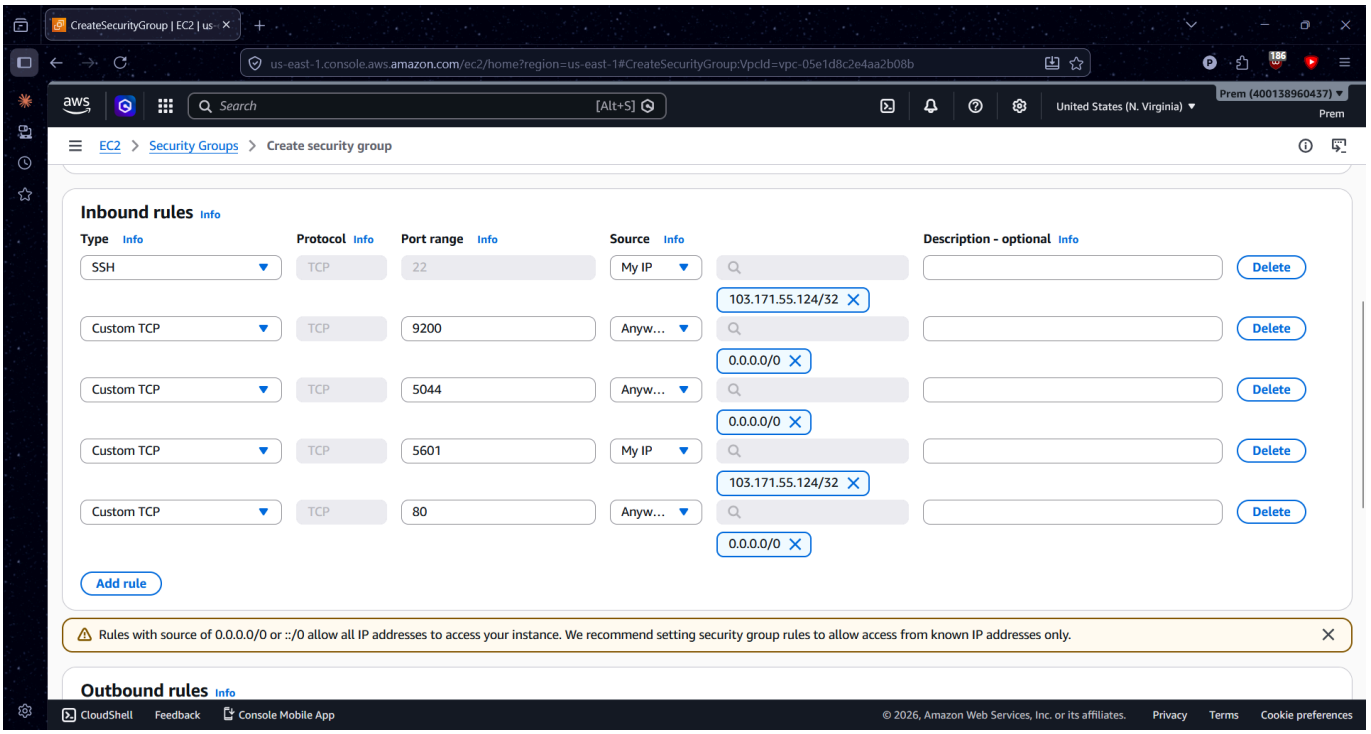
Instance 2 — App Server 1: → AMI: Ubuntu 26.04 LTS → Instance type: t3.small → Storage: 8 GB default is fine → Security group: ELK-sg

Instance 3 — App Server 2: → AMI: Ubuntu 26.04 LTS → Instance type: t3.small → Storage: 8 GB default is fine → Security group: ELK-sg

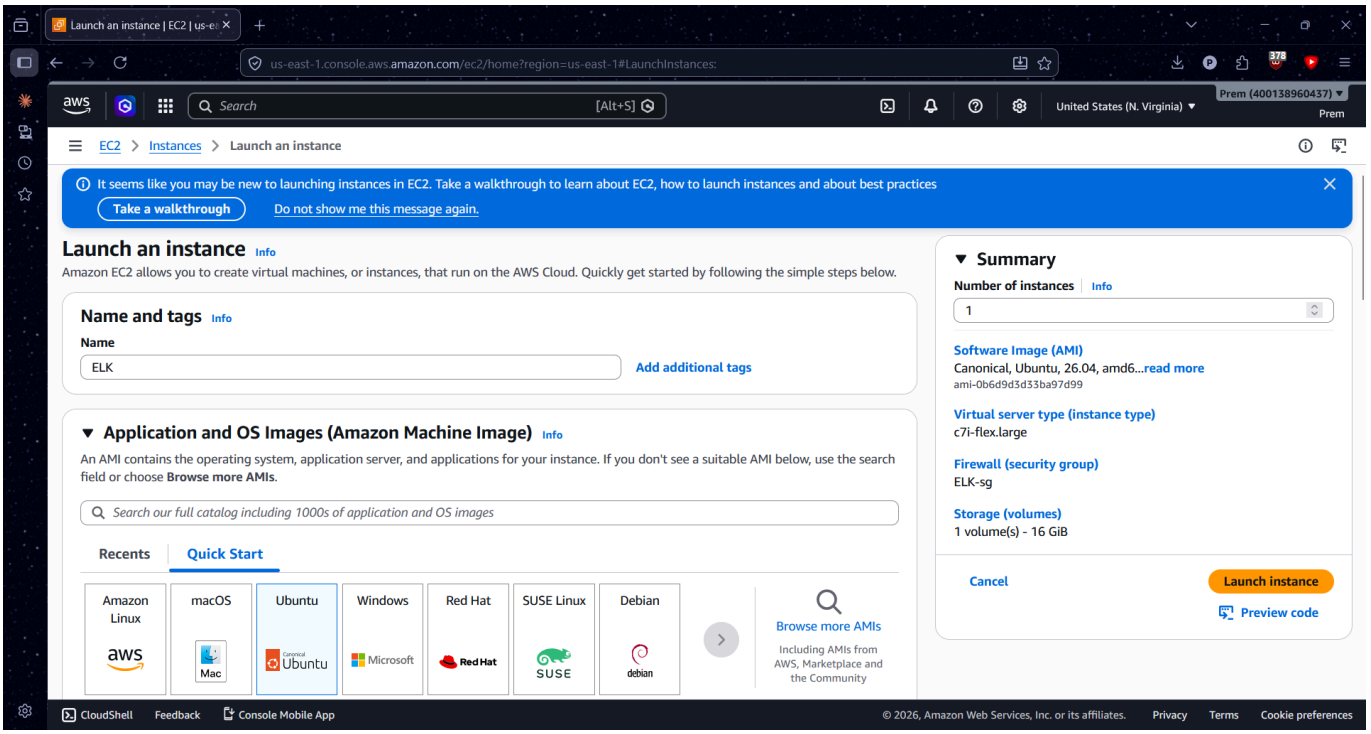
> **ELK server security group** showing ports 22, 9200, 5044, 5601 inbound rules.

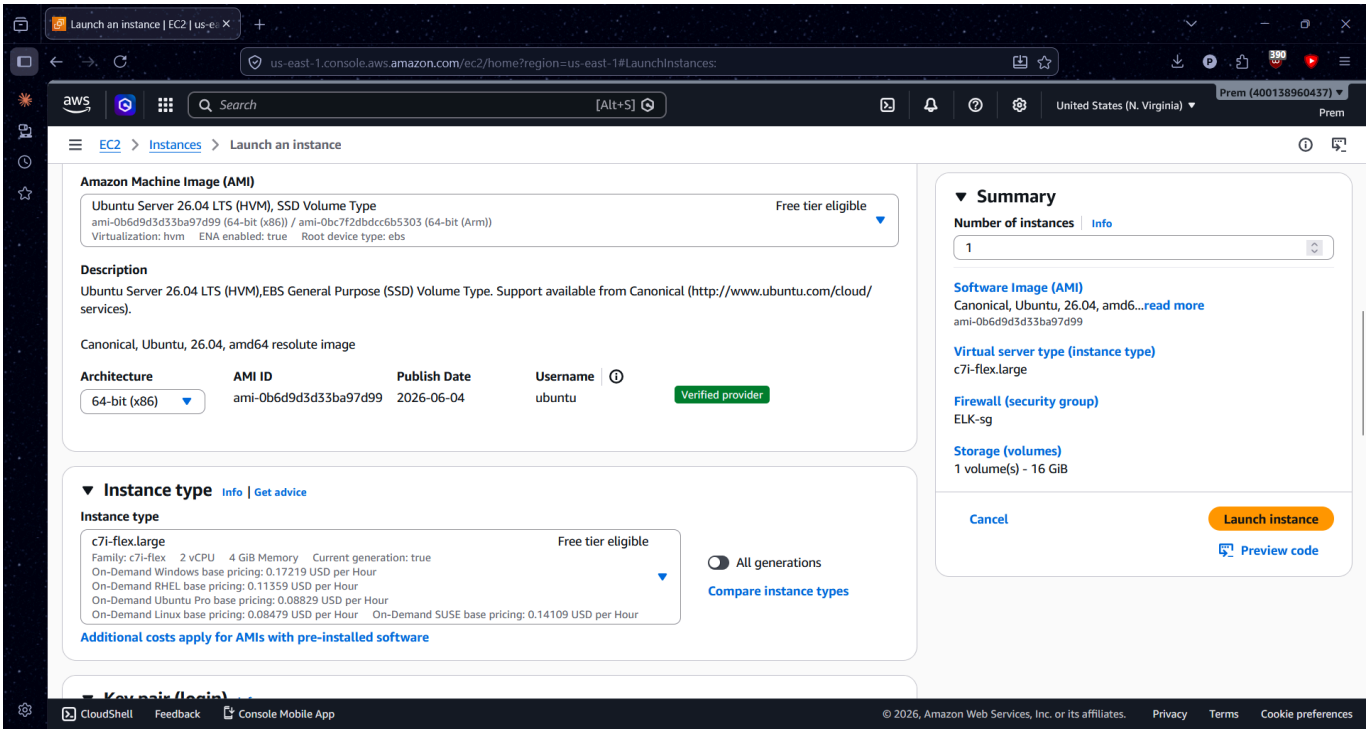


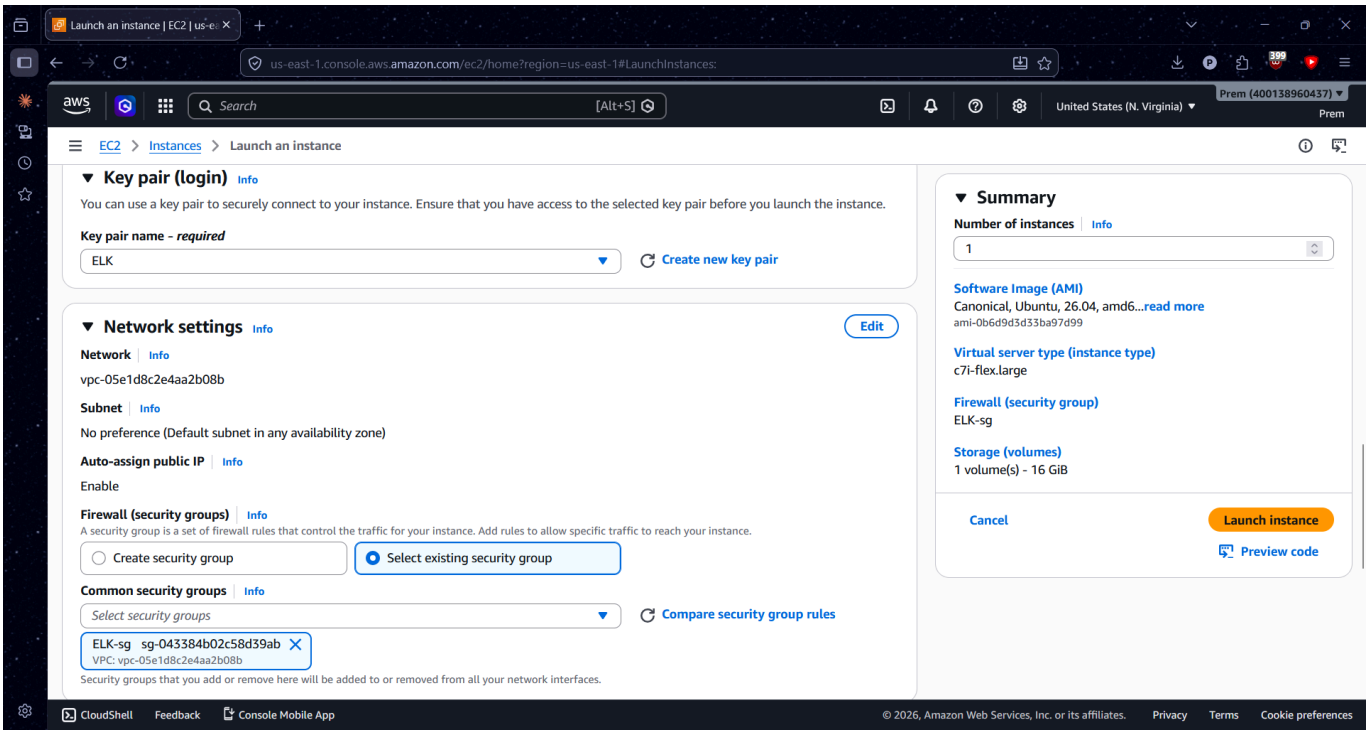


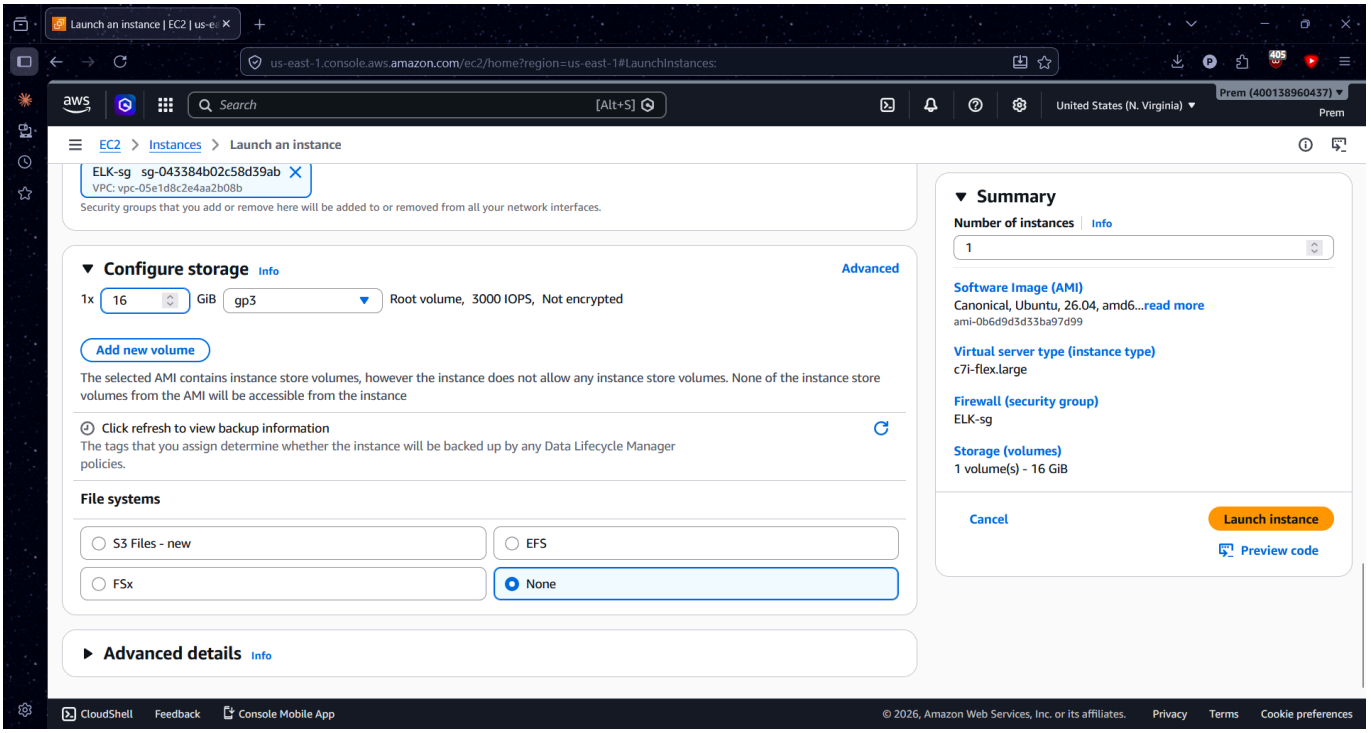


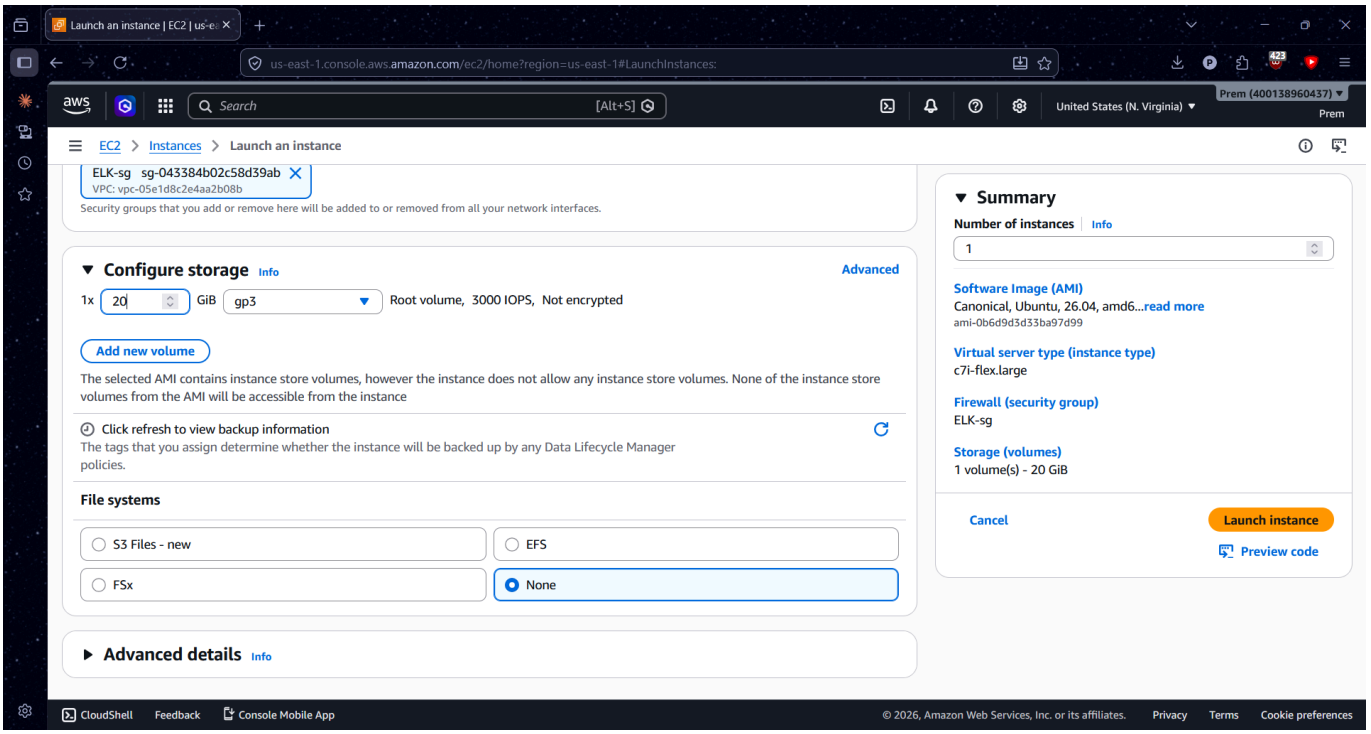
> **All 3 EC2 instances running** — elk-server (c7i-flex.large), app-server-1 (t3.small), app-server-2 (t3.small).

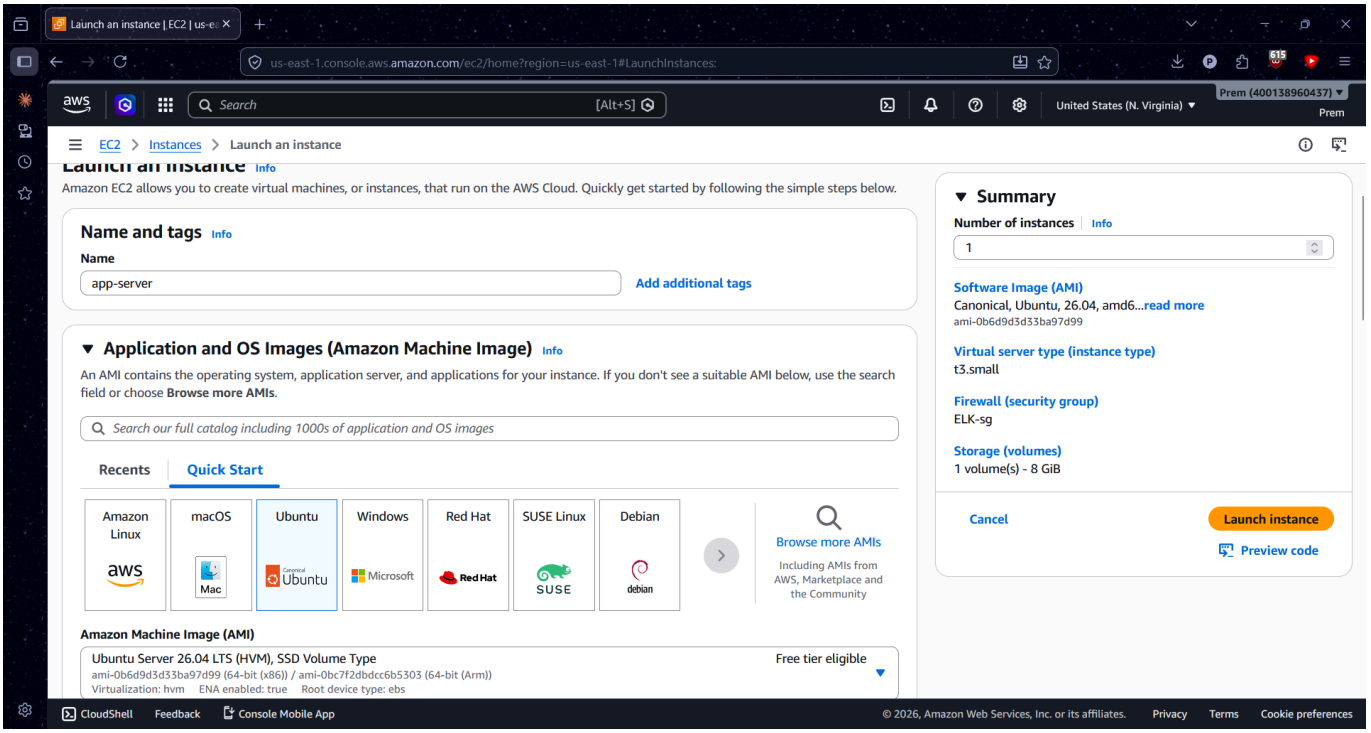

Launch an instance | EC2 | us-east-1
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
EC2 > Instances > Launch an instance
It seems like you may be new to launching instances in EC2. Take a walkthrough to learn about EC2, how to launch instances and about best practices
Take a walkthrough
Do not show me this message again.
Launch an instance Info
Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.
Name and tags Info
Name
ELK
Add additional tags
Application and OS Images (Amazon Machine Image) Info
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.
Search our full catalog including 1000s of application and OS images
Recents
Quick Start
Amazon Linux
macOS
Ubuntu
Windows
Red Hat
SUSE Linux
Debian
Browse more AMIs
Including AMIs from AWS, Marketplace and the Community
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
c7i-flex.large
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 16 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

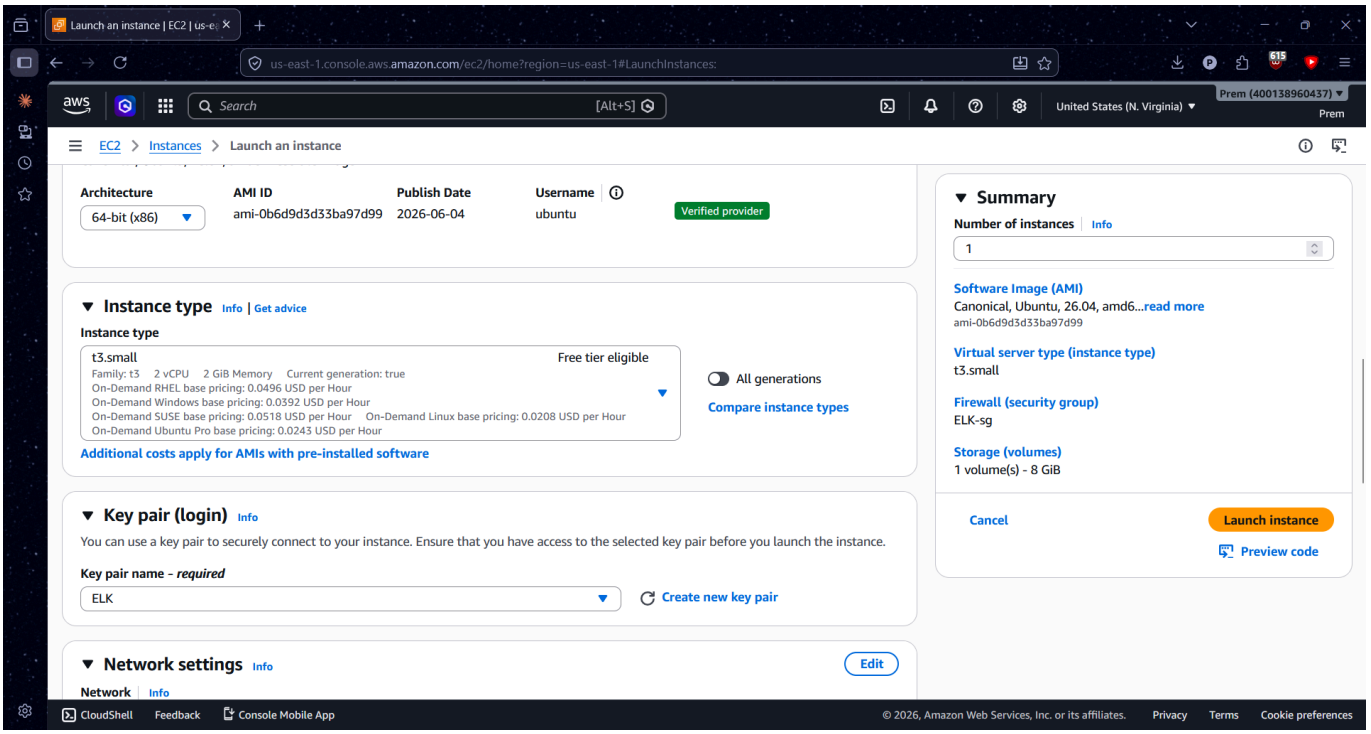

Launch an instance | EC2 | us-east-1
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
EC2 > Instances > Launch an instance
Amazon Machine Image (AMI)
Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
Free tier eligible
ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdcc6b5303 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs
Description
Ubuntu Server 26.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (http://www.ubuntu.com/cloud/services).
Canonical, Ubuntu, 26.04, amd64 resolute image
Architecture
AMI ID
Publish Date
Username
64-bit (x86)
ami-0b6d9d3d33ba97d99
2026-06-04
ubuntu
Verified provider
Instance type Info | Get advice
Instance type
c7i-flex.large
Free tier eligible
Family: c7i-flex 2 vCPU 4 GiB Memory Current generation: true
On-Demand Windows base pricing: 0.17219 USD per Hour
On-Demand RHEL base pricing: 0.11359 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.08829 USD per Hour
On-Demand Linux base pricing: 0.08479 USD per Hour On-Demand SUSE base pricing: 0.14109 USD per Hour
Additional costs apply for AMIs with pre-installed software
All generations
Compare instance types
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
c7i-flex.large
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 16 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

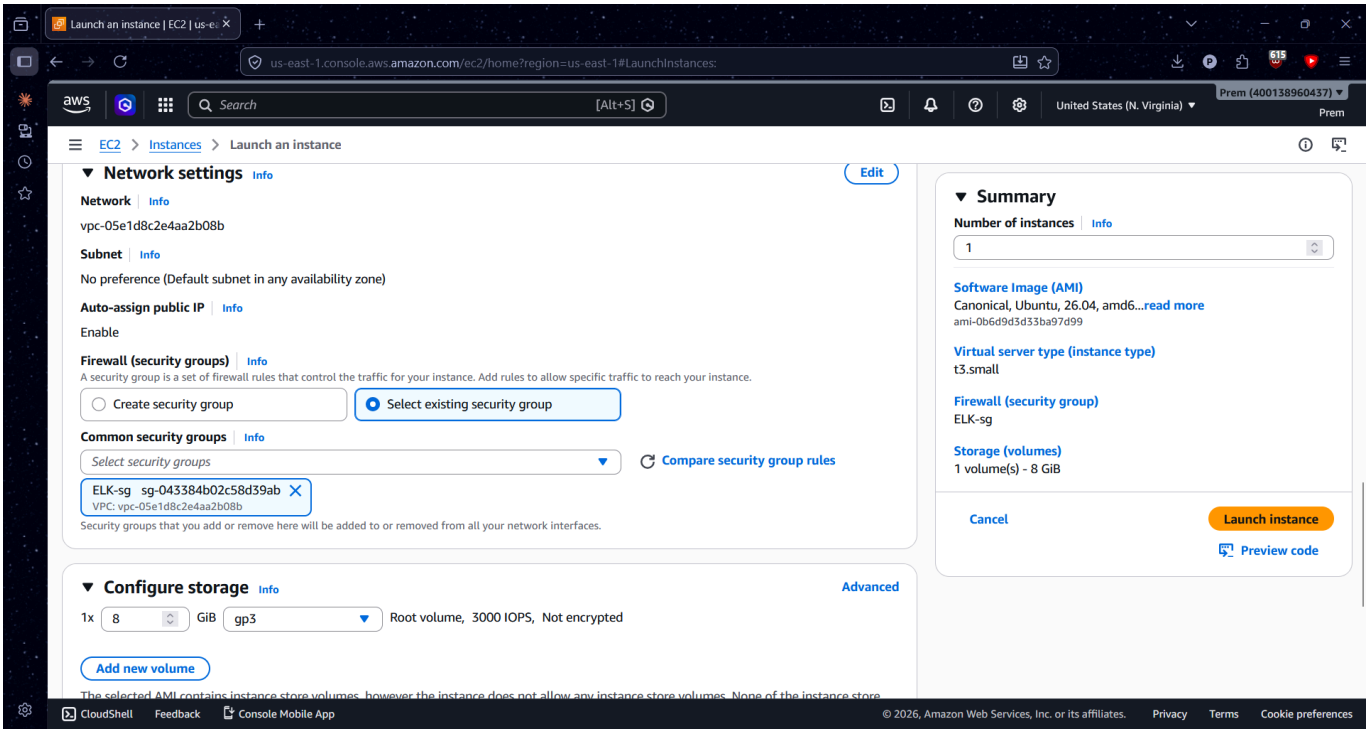

Launch an instance | EC2 | us-e
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
Key pair (login) Info
You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.
Key pair name - required
ELK
Create new key pair
Network settings Info
Edit
Network Info
vpc-05e1d8c2e4aa2b08b
Subnet Info
No preference (Default subnet in any availability zone)
Auto-assign public IP Info
Enable
Firewall (security groups) Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.
Create security group
Select existing security group
Common security groups Info
Select security groups
Compare security group rules
ELK-sg sg-043384b02c58d39ab
VPC: vpc-05e1d8c2e4aa2b08b
Security groups that you add or remove here will be added to or removed from all your network interfaces.
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
c7i-flex.large
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 16 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

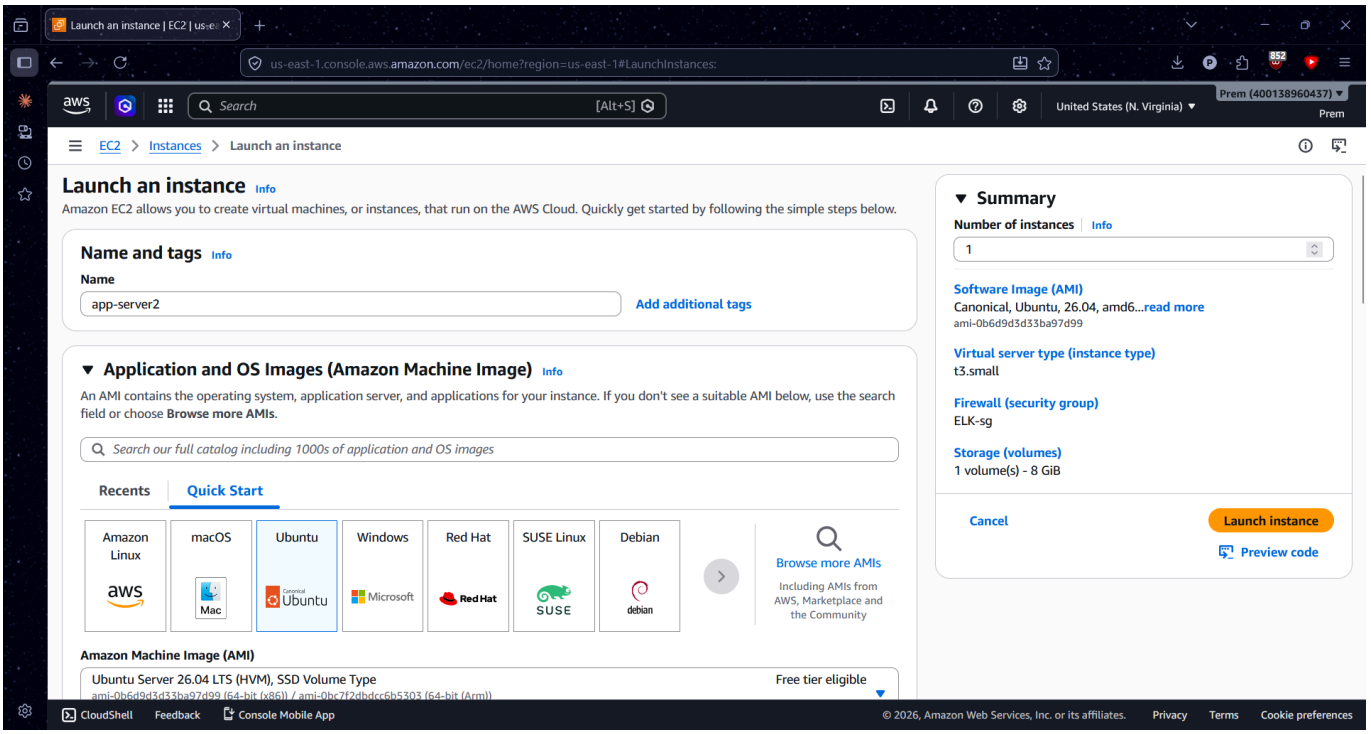

Launch an instance | EC2 | us-e
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
ELK-sg sg-043384b02c58d39ab
VPC: vpc-05e1d8c2e4aa2b08b
Security groups that you add or remove here will be added to or removed from all your network interfaces.
Configure storage Info
Advanced
1x 16 GiB gp3 Root volume, 3000 IOPS, Not encrypted
Add new volume
The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance
Click refresh to view backup information
The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.
File systems
S3 Files - new
EFS
FSx
None
Advanced details Info
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
c7i-flex.large
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 16 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

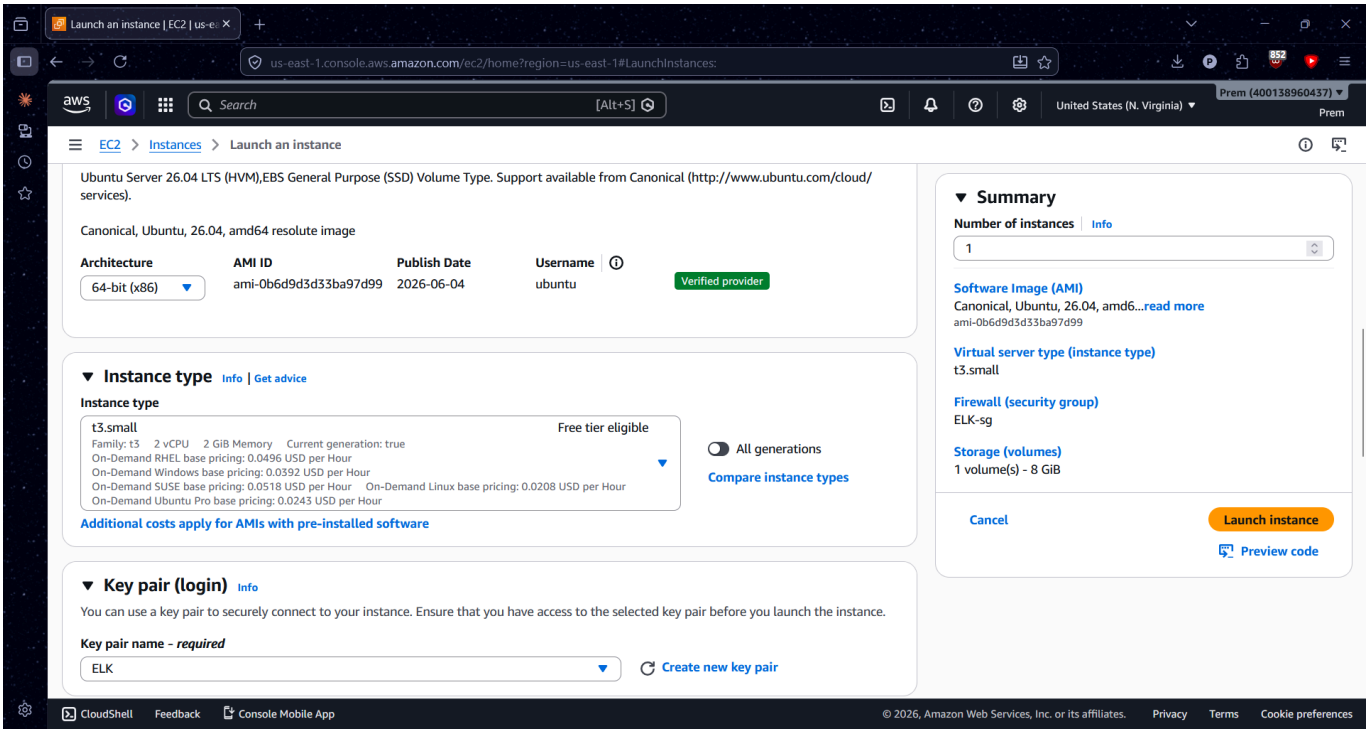

Launch an instance | EC2 | us-ea
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
ELK-sg sg-043384b02c58d39ab
VPC: vpc-05e1d8c2e4aa2b08b
Security groups that you add or remove here will be added to or removed from all your network interfaces.
Configure storage Info
Advanced
1x 20 GiB gp3 Root volume, 3000 IOPS, Not encrypted
Add new volume
The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance
Click refresh to view backup information
The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.
File systems
S3 Files - new
EFS
FSx
None
Advanced details Info
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
c7i-flex.large
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 20 GiB
Cancel
Launch instance
Preview code
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

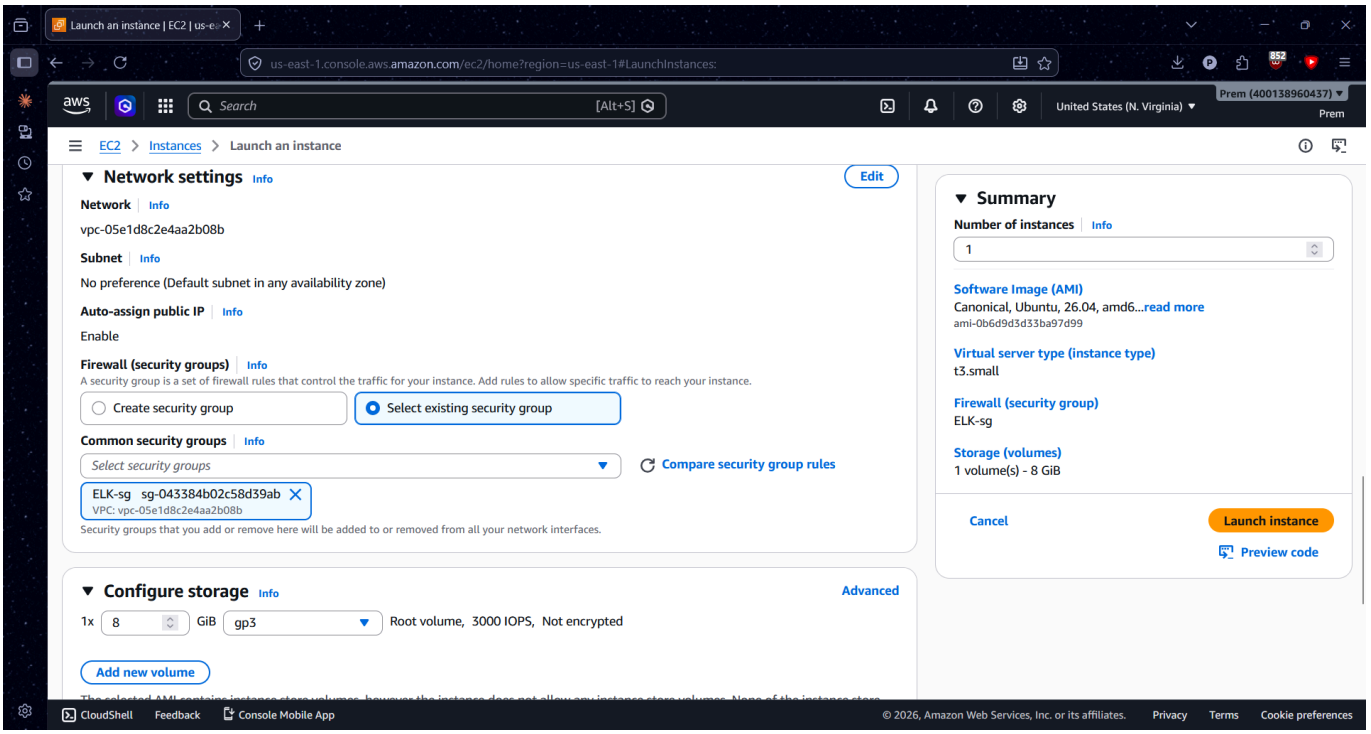

Launch an instance | EC2 | us-ea
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.
Name and tags Info
Name
app-server
Add additional tags
Application and OS Images (Amazon Machine Image) Info
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.
Search our full catalog including 1000s of application and OS images
Recents
Quick Start
Amazon Linux
macOS
Ubuntu
Windows
Red Hat
SUSE Linux
Debian
Browse more AMIs
Including AMIs from AWS, Marketplace and the Community
Amazon Machine Image (AMI)
Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
Free tier eligible
ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdcc6b5303 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
t3.small
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Launch instance
Preview code
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

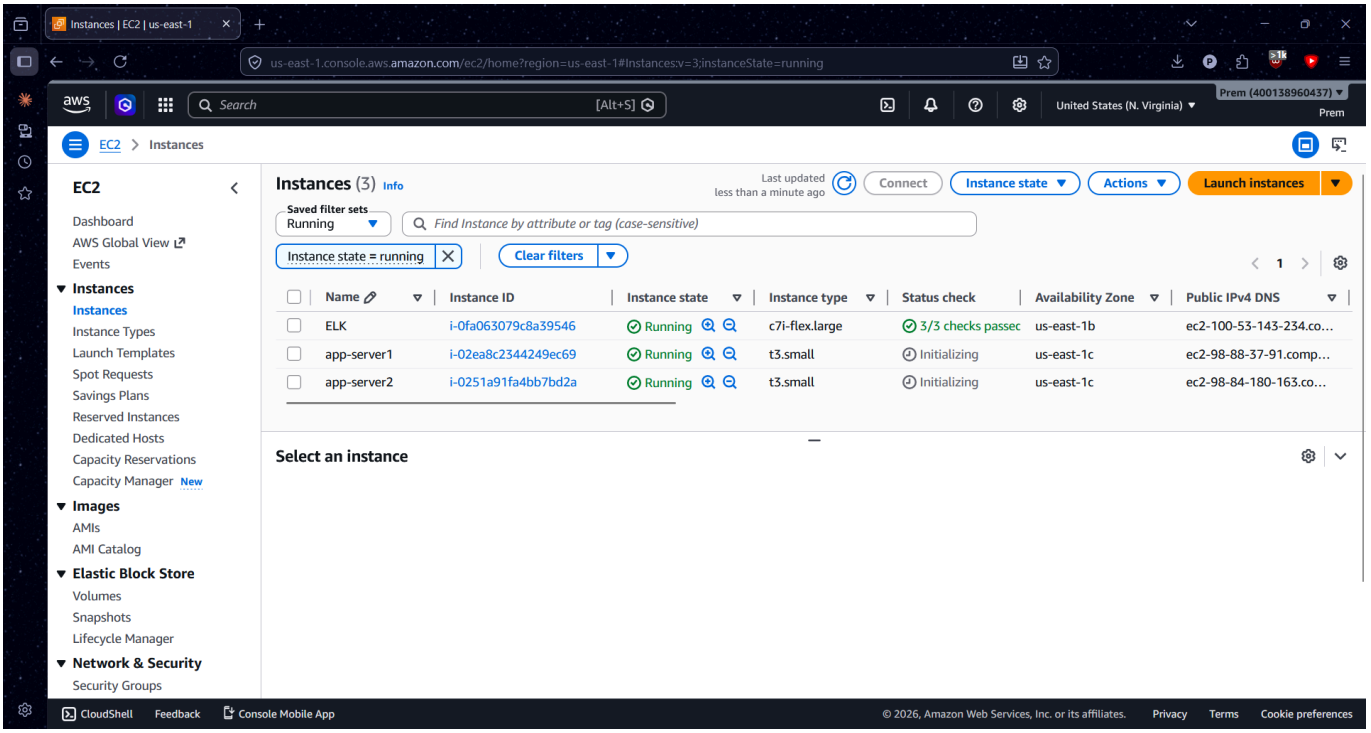

Launch an instance | EC2 | us-ea
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
Architecture
64-bit (x86)
AMI ID
ami-0b6d9d3d33ba97d99
Publish Date
2026-06-04
Username
ubuntu
Verified provider
Instance type Info | Get advice
Instance type
t3.small
Free tier eligible
Family: t3 2 vCPU 2 GiB Memory Current generation: true
On-Demand RHEL base pricing: 0.0496 USD per Hour
On-Demand Windows base pricing: 0.0392 USD per Hour
On-Demand SUSE base pricing: 0.0518 USD per Hour On-Demand Linux base pricing: 0.0208 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.0243 USD per Hour
All generations
Compare instance types
Additional costs apply for AMIs with pre-installed software
Key pair (login) Info
You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.
Key pair name - required
ELK
Create new key pair
Network settings Info
Edit
Network Info
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
t3.small
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Launch instance
Preview code
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


Launch an instance | EC2 | us-ea
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
Network settings Info
Edit
Network Info
vpc-05e1d8c2e4aa2b08b
Subnet Info
No preference (Default subnet in any availability zone)
Auto-assign public IP Info
Enable
Firewall (security groups) Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.
Create security group
Select existing security group
Common security groups Info
Select security groups
Compare security group rules
ELK-sg sg-043384b02c58d39ab
VPC: vpc-05e1d8c2e4aa2b08b
Security groups that you add or remove here will be added to or removed from all your network interfaces.
Configure storage Info
Advanced
1x 8 GiB gp3 Root volume, 3000 IOPS, Not encrypted
Add new volume
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
t3.small
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Launch instance
Preview code
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


Launch an instance | EC2 | us-ea
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
Launch an instance Info
Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.
Name and tags Info
Name
app-server2
Add additional tags
Application and OS Images (Amazon Machine Image) Info
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.
Search our full catalog including 1000s of application and OS images
Recents
Quick Start
Amazon Linux
macOS
Ubuntu
Windows
Red Hat
SUSE Linux
Debian
Browse more AMIs
Including AMIs from AWS, Marketplace and the Community
Amazon Machine Image (AMI)
Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
Free tier eligible
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
t3.small
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Launch an instance | EC2 | us-ea
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > Launch an instance
Ubuntu Server 26.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (http://www.ubuntu.com/cloud/services).
Canonical, Ubuntu, 26.04, amd64 resolute image
Architecture
64-bit (x86)
AMI ID
ami-0b6d9d3d33ba97d99
Publish Date
2026-06-04
Username
ubuntu
Verified provider
Instance type Info | Get advice
Instance type
t3.small
Free tier eligible
Family: t3 2 vCPU 2 GiB Memory Current generation: true
On-Demand RHEL base pricing: 0.0496 USD per Hour
On-Demand Windows base pricing: 0.0392 USD per Hour
On-Demand SUSE base pricing: 0.0518 USD per Hour On-Demand Linux base pricing: 0.0208 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.0243 USD per Hour
All generations
Compare instance types
Additional costs apply for AMIs with pre-installed software
Key pair (login) Info
You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.
Key pair name - required
ELK
Create new key pair
Summary
Number of instances Info
1
Software Image (AMI)
Canonical, Ubuntu, 26.04, amd6...read more
ami-0b6d9d3d33ba97d99
Virtual server type (instance type)
t3.small
Firewall (security group)
ELK-sg
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences





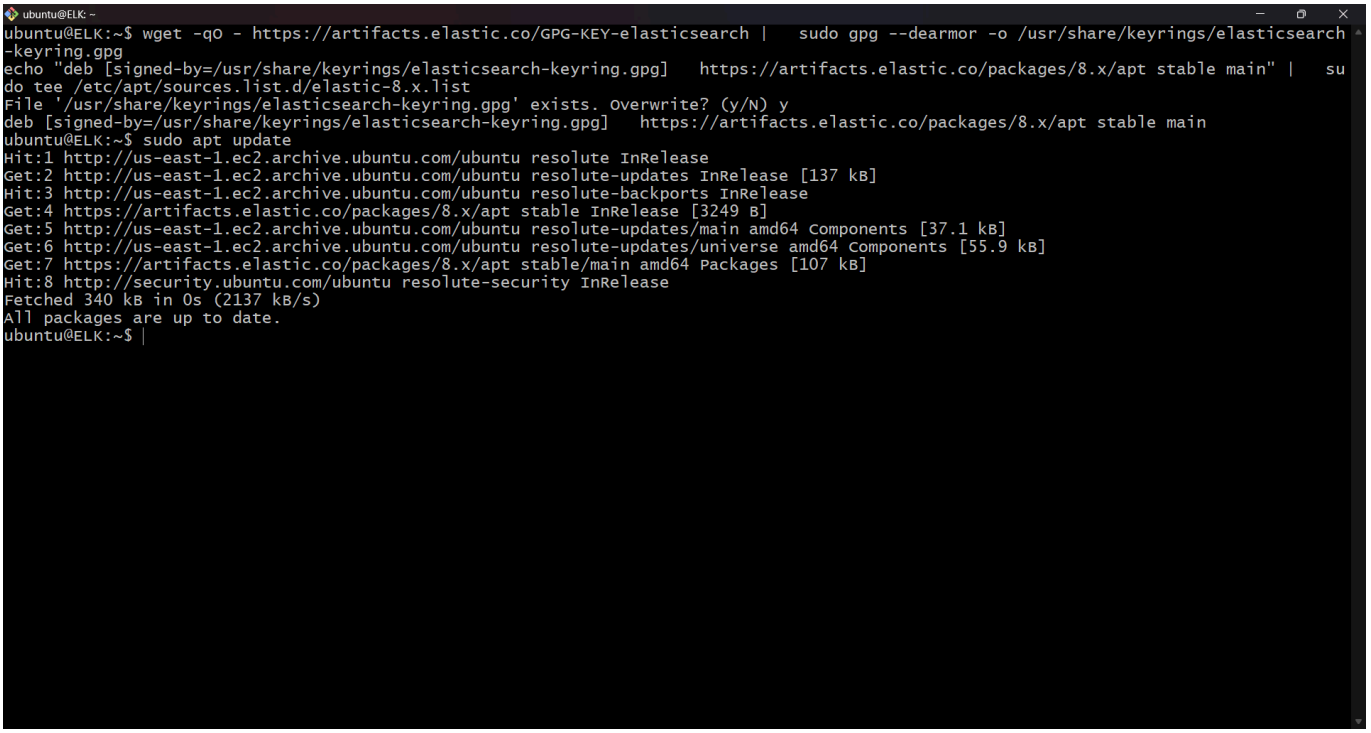
Part 2 — Elasticsearch Setup

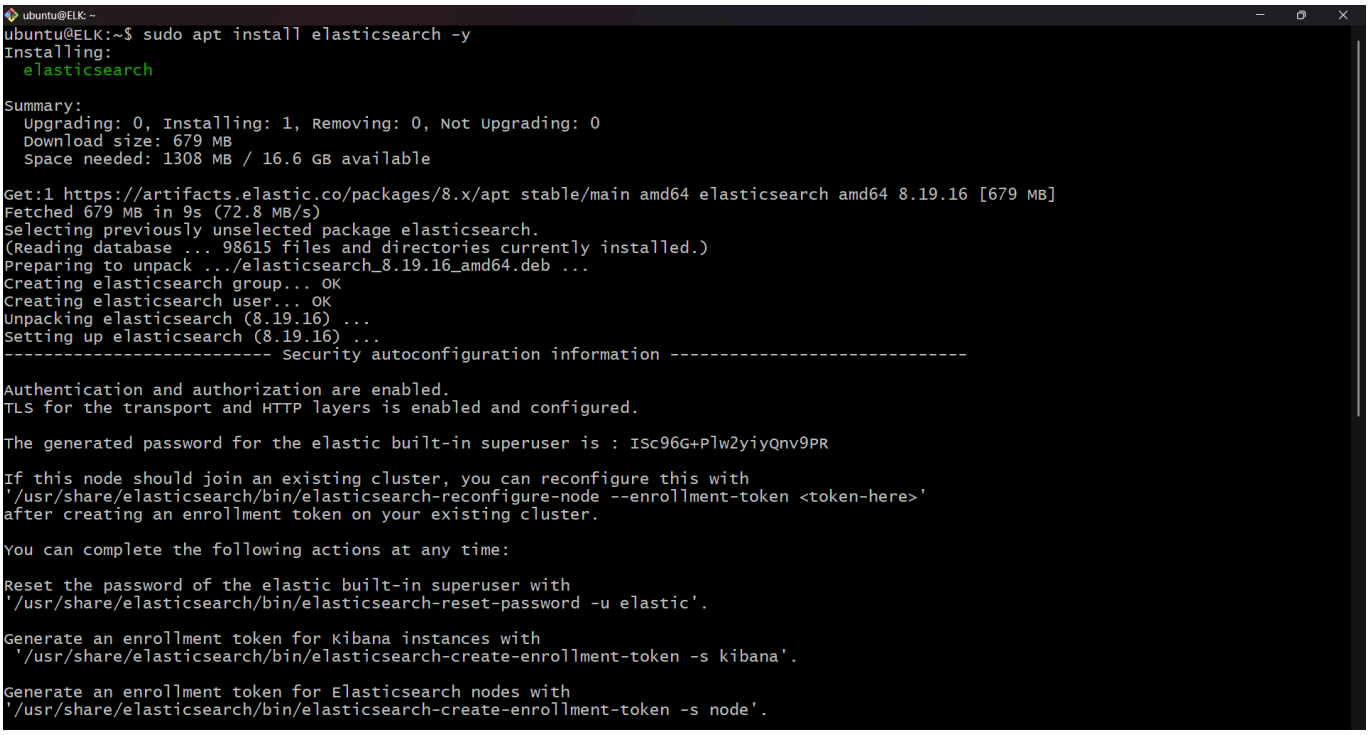
SSH into the ELK server and install Elasticsearch 8.x. Disable security for this lab and configure to listen on all network interfaces so Logstash and Kibana can connect.

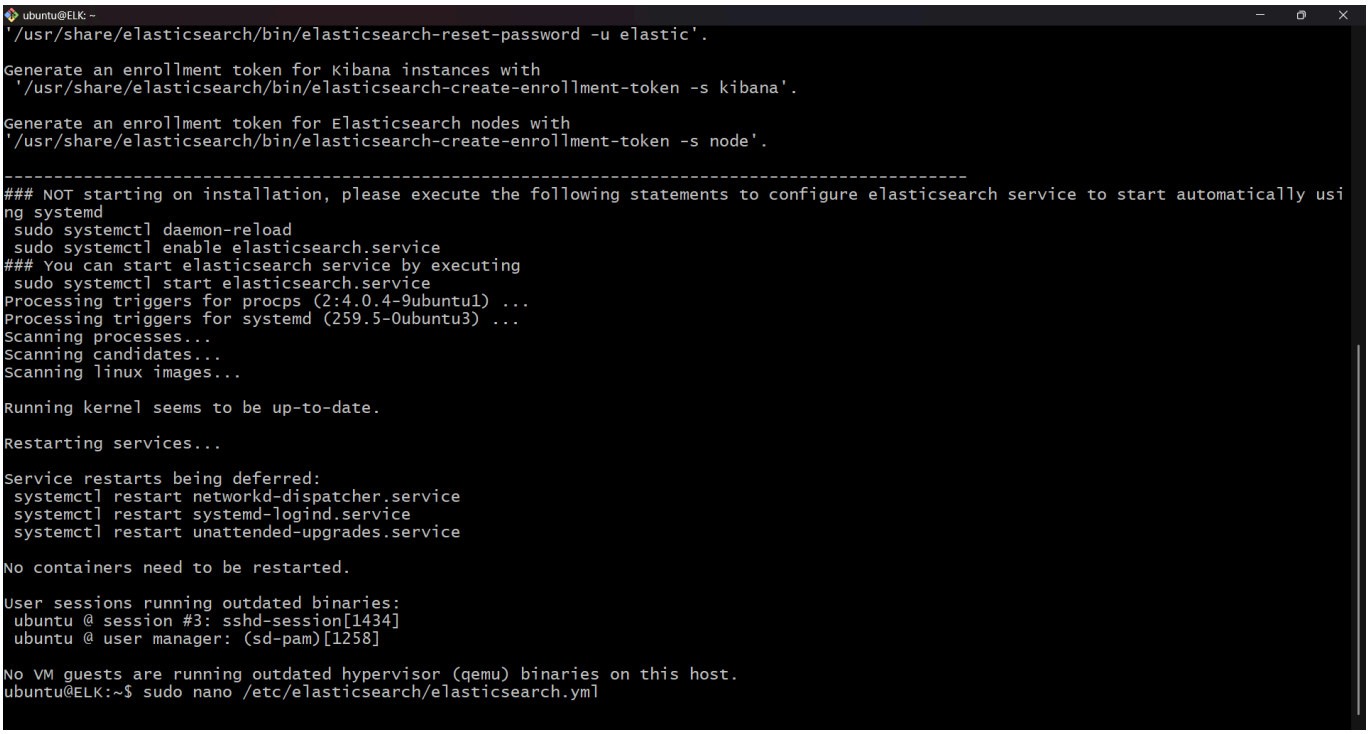
```
# Add Elastic repository
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
  sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] \
  https://artifacts.elastic.co/packages/8.x/apt stable main" | \
  sudo tee /etc/apt/sources.list.d/elastic-8.x.list
```

```
sudo apt update
sudo apt install elasticsearch -y
```

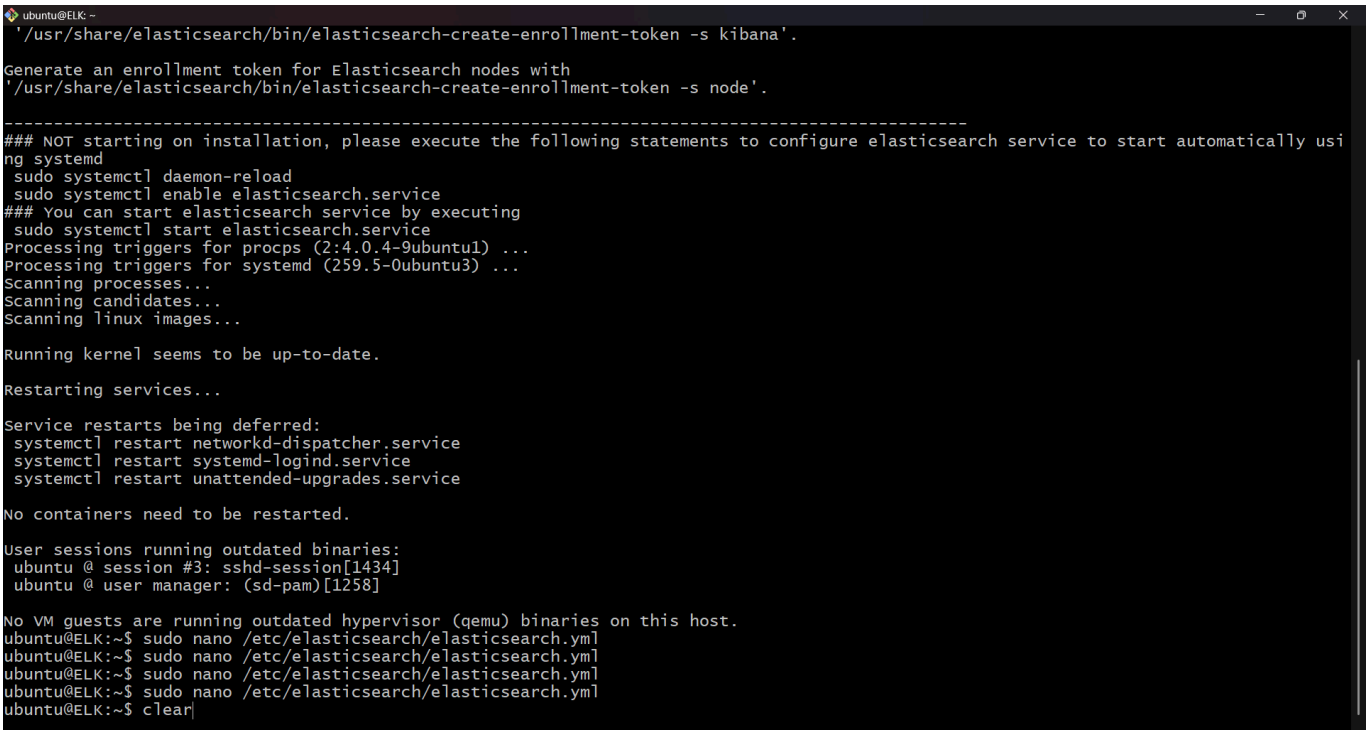


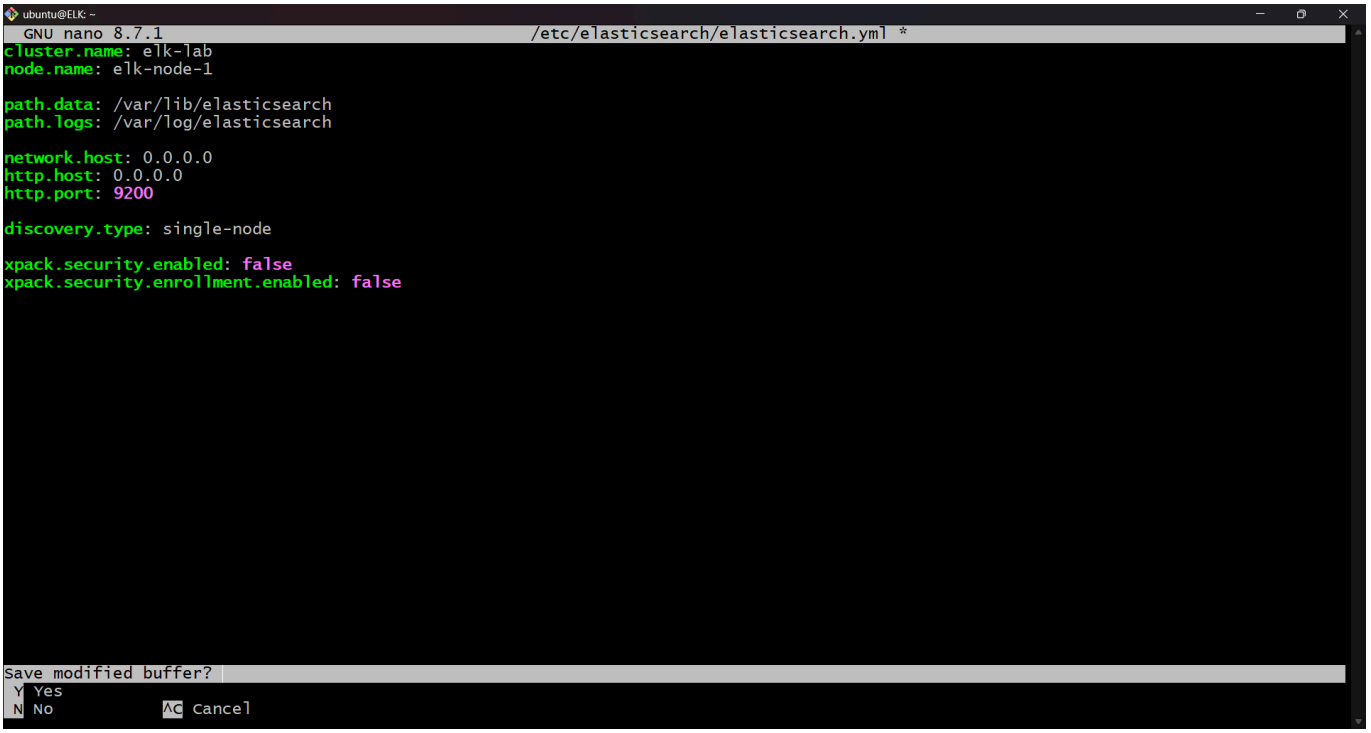




Elasticsearch configuration (`/etc/elasticsearch/elasticsearch.yml`):

```
cluster.name: elk-lab
node.name: elk-node-1
path.data: /var/lib/elasticsearch
path.logs: /var/log/elasticsearch
network.host: 0.0.0.0
http.port: 9200
discovery.type: single-node
xpack.security.enabled: false
xpack.security.enrollment.enabled: false
```





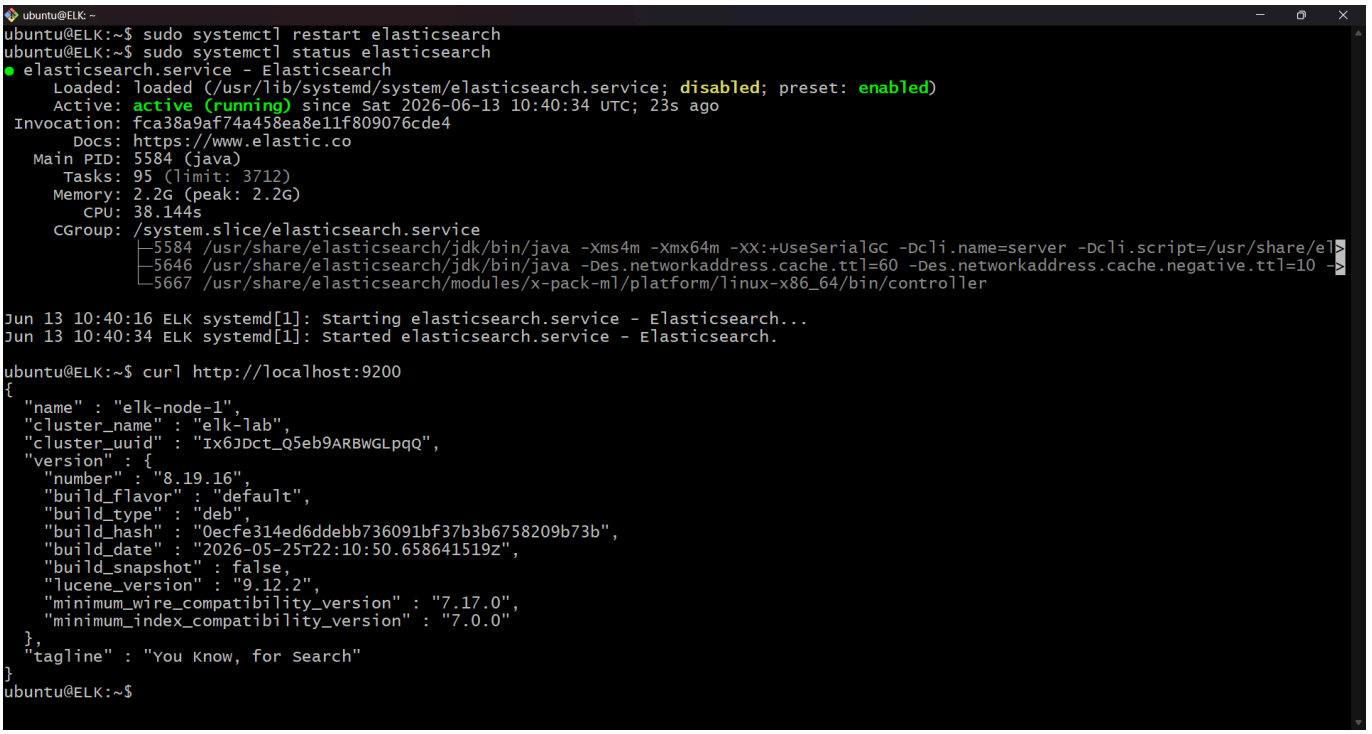
JVM heap size (`/etc/elasticsearch/jvm.options.d/heap.options`):

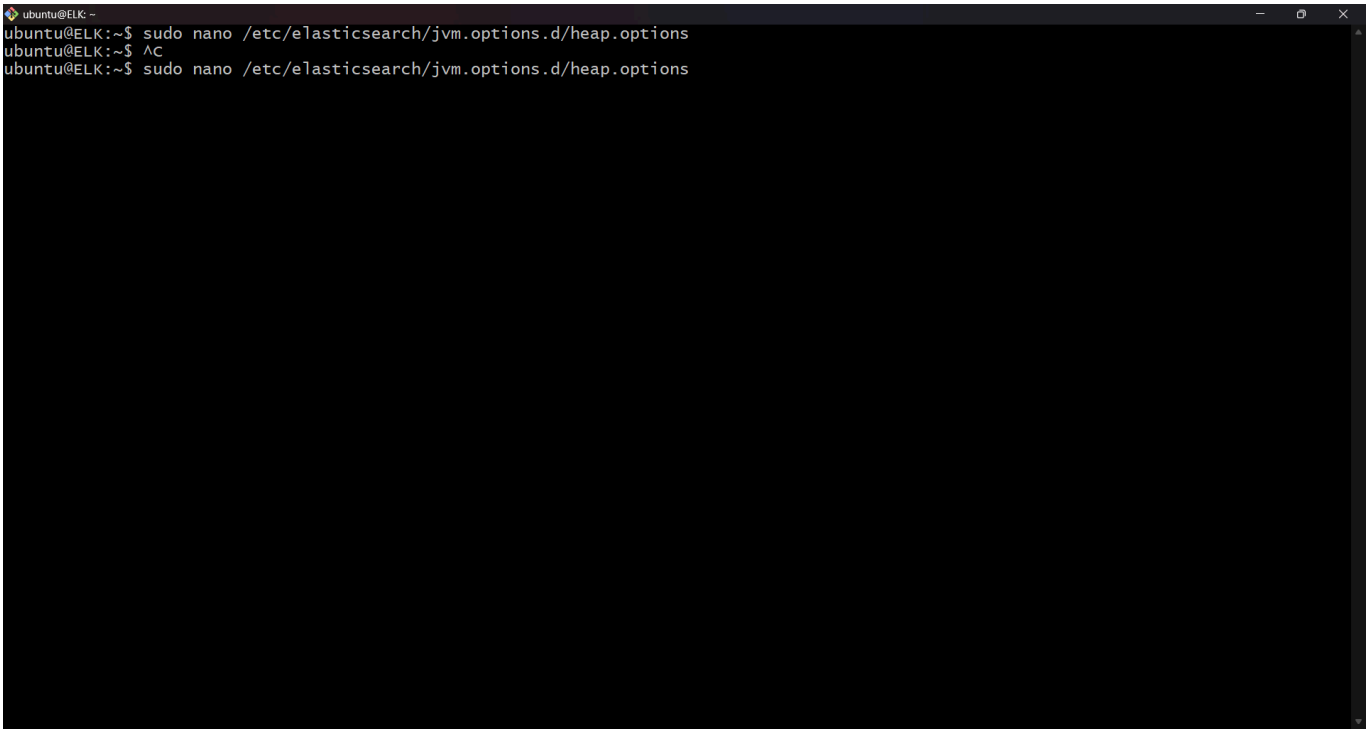
```
-Xms512m
-Xmx512m
```

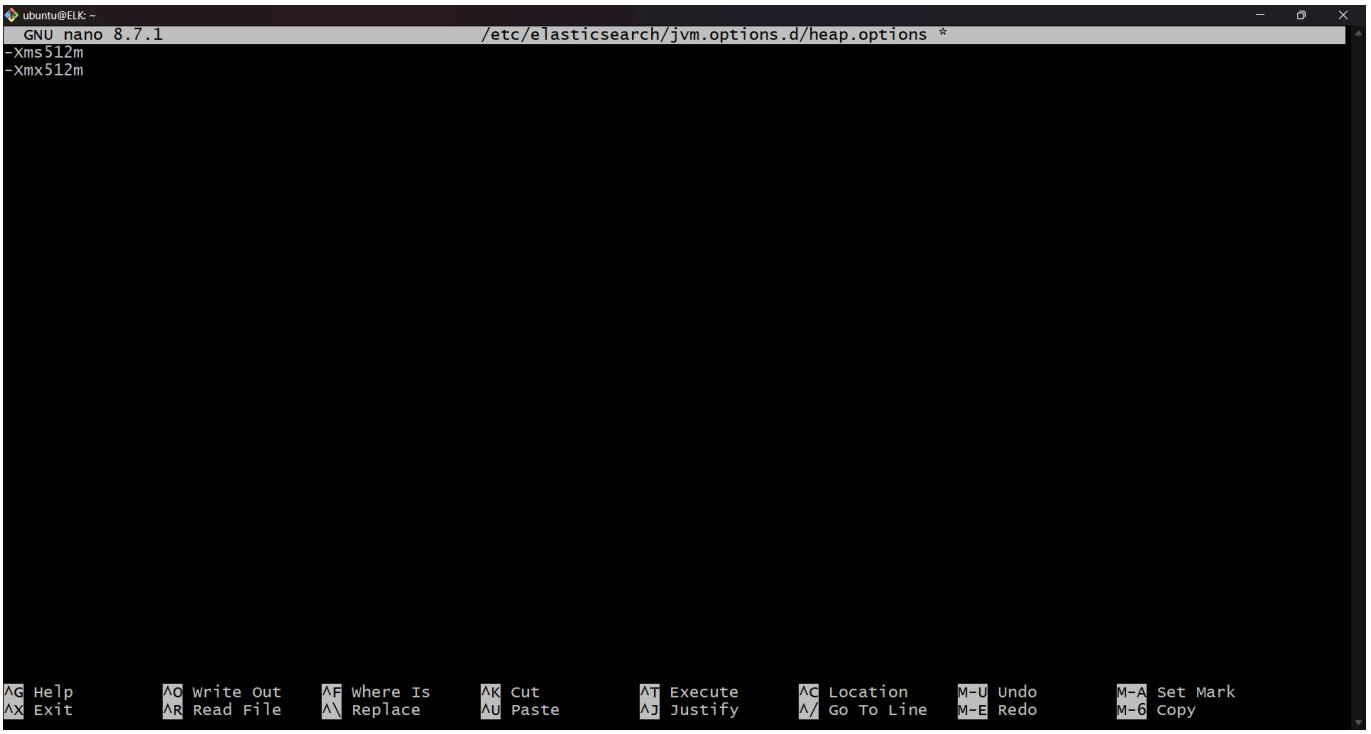
```
sudo systemctl start elasticsearch
sudo systemctl enable elasticsearch

# Test
curl http://localhost:9200
```

> **Elasticsearch responding** — curl returning cluster JSON with cluster_name: elk-lab.







Part 3 — Logstash Pipeline

Install Logstash and create the Nginx log parsing pipeline. The grok pattern extracts structured fields from raw Nginx log lines and tags 4xx and 5xx errors automatically.

```
sudo apt install logstash -y
```

Pipeline configuration (`/etc/logstash/conf.d/nginx-pipeline.conf`):

```
input {
  beats {
    port => 5044
  }
}

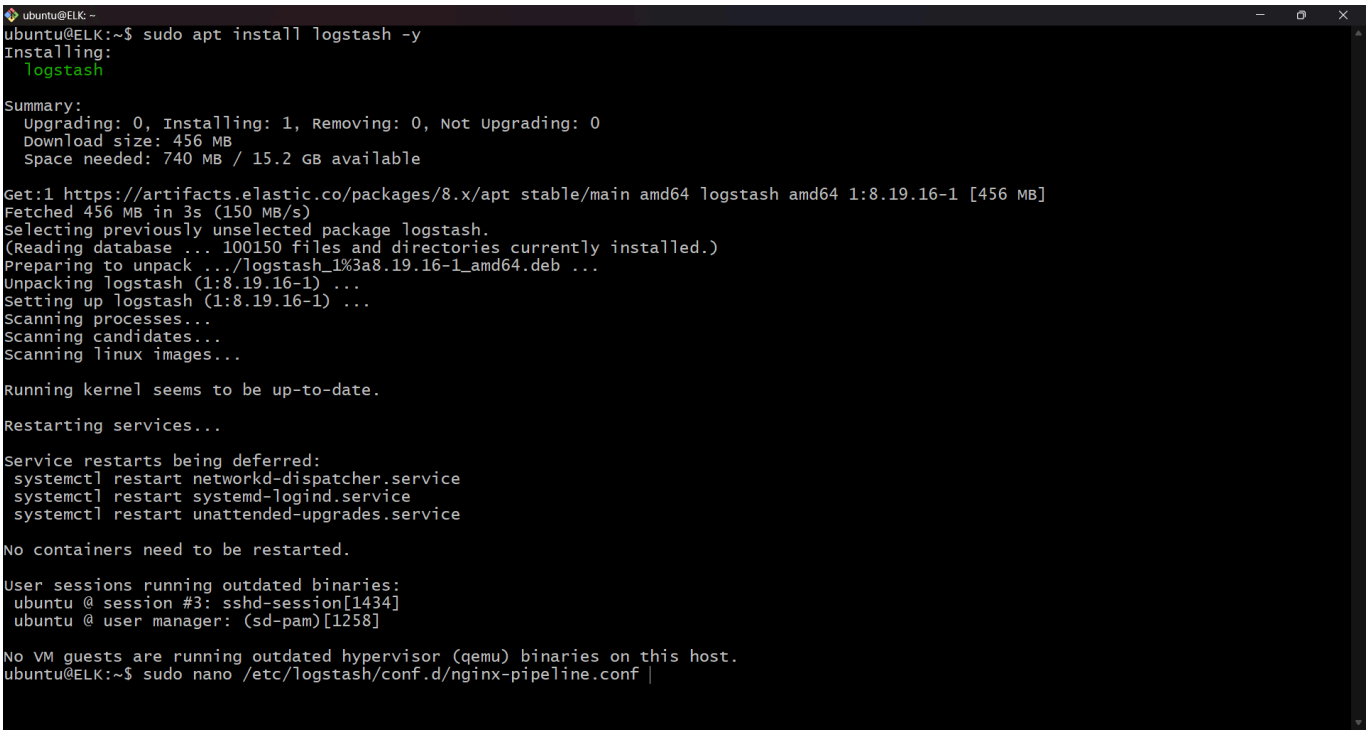
filter {
  if [log_type] == "nginx_access" {
    grok {
      match => {
        "message" => [
          '%{IPORHOST:client_ip} - %{DATA:user} \[%{HTTPDATE:timestamp}\] \"%
{WORD:method} %{DATA:request} HTTP/%{NUMBER:http_version}\" %
{NUMBER:response_code:int} %{NUMBER:bytes:int} \"%{DATA:referrer}\" \"%
{DATA:user_agent}\"',
          '%{IPORHOST:client_ip} - %{DATA:user} \[%{HTTPDATE:timestamp}\] \"%
{WORD:method} %{DATA:request} HTTP/%{NUMBER:http_version}\" %
{NUMBER:response_code:int} %{NUMBER:bytes:int}'
        ]
      }
```

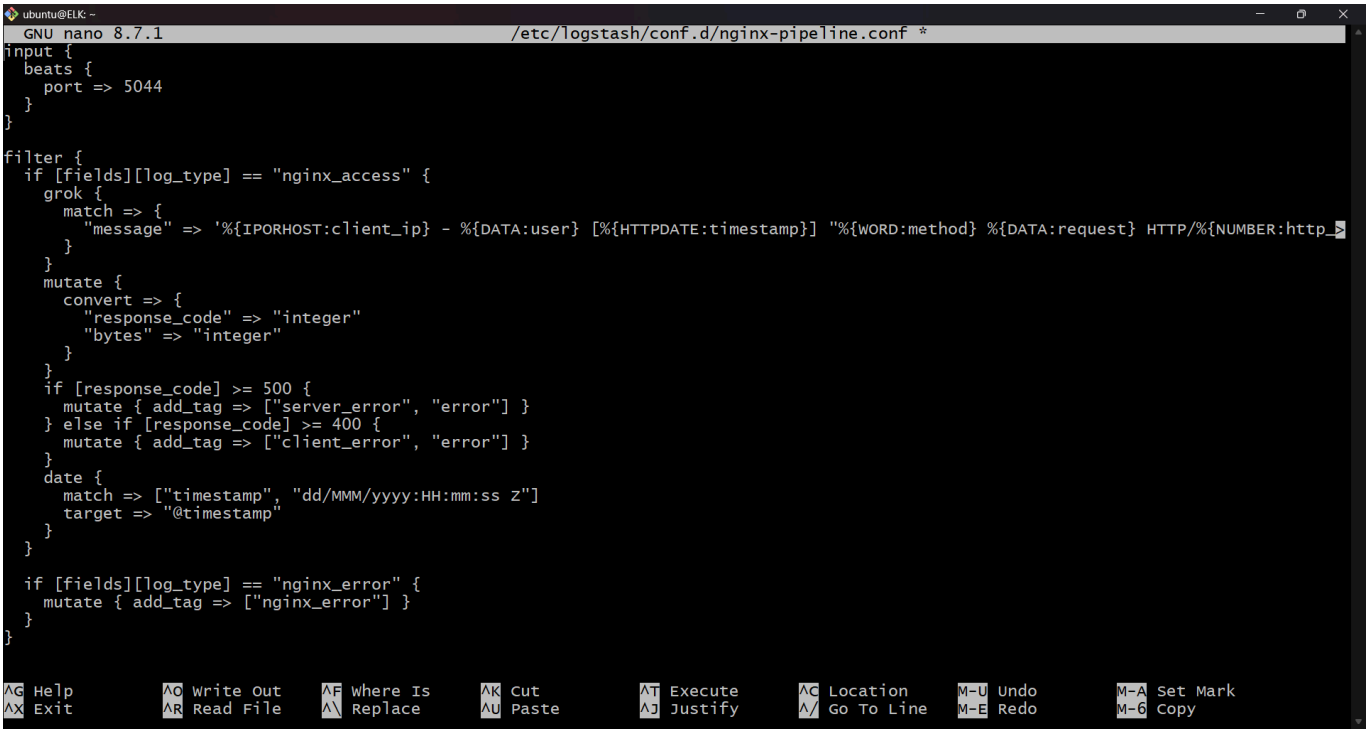
```
    }
    if "_grokparsefailure" not in [tags] {
      if [response_code] >= 500 {
        mutate { add_tag => ["server_error", "error"] }
      } else if [response_code] >= 400 {
        mutate { add_tag => ["client_error", "error"] }
      }
    }
  }
}

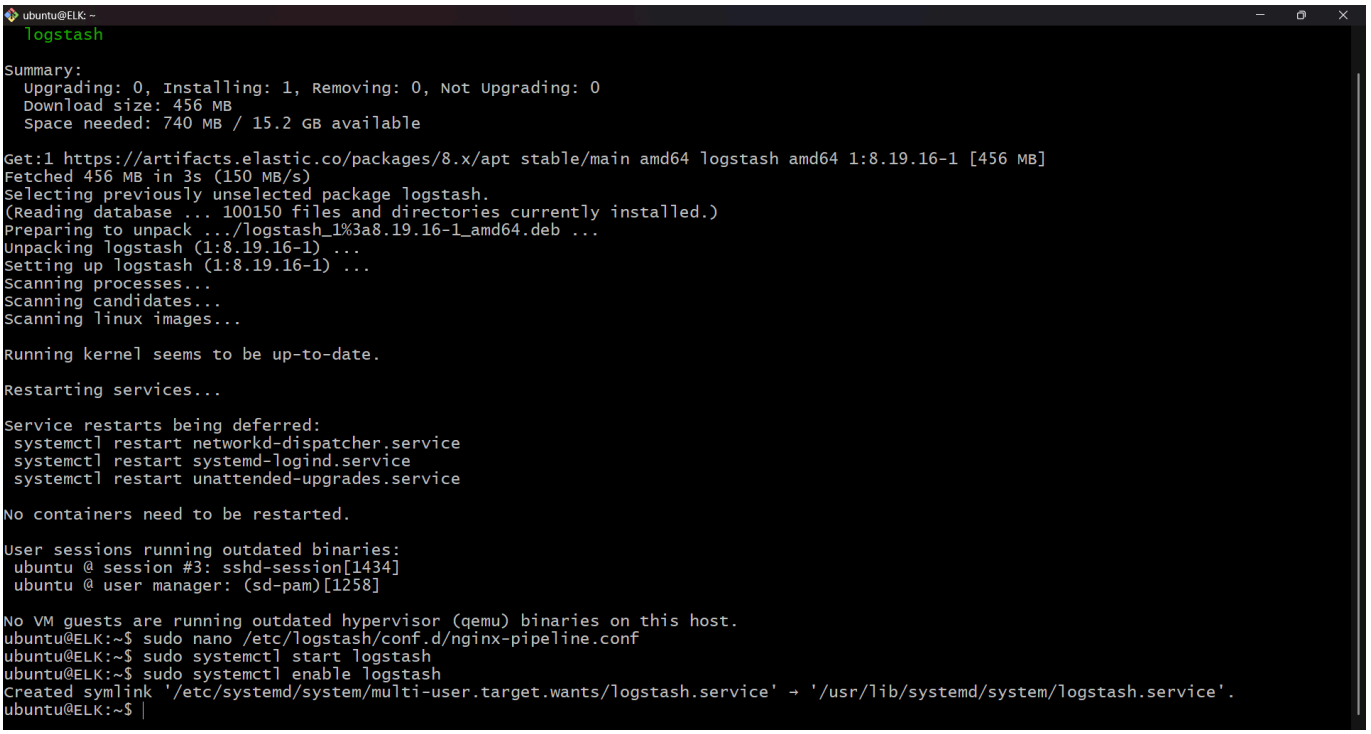
output {
  elasticsearch {
    hosts => ["http://localhost:9200"]
    index => "nginx-logs-%{+YYYY.MM.dd}"
  }
}
```

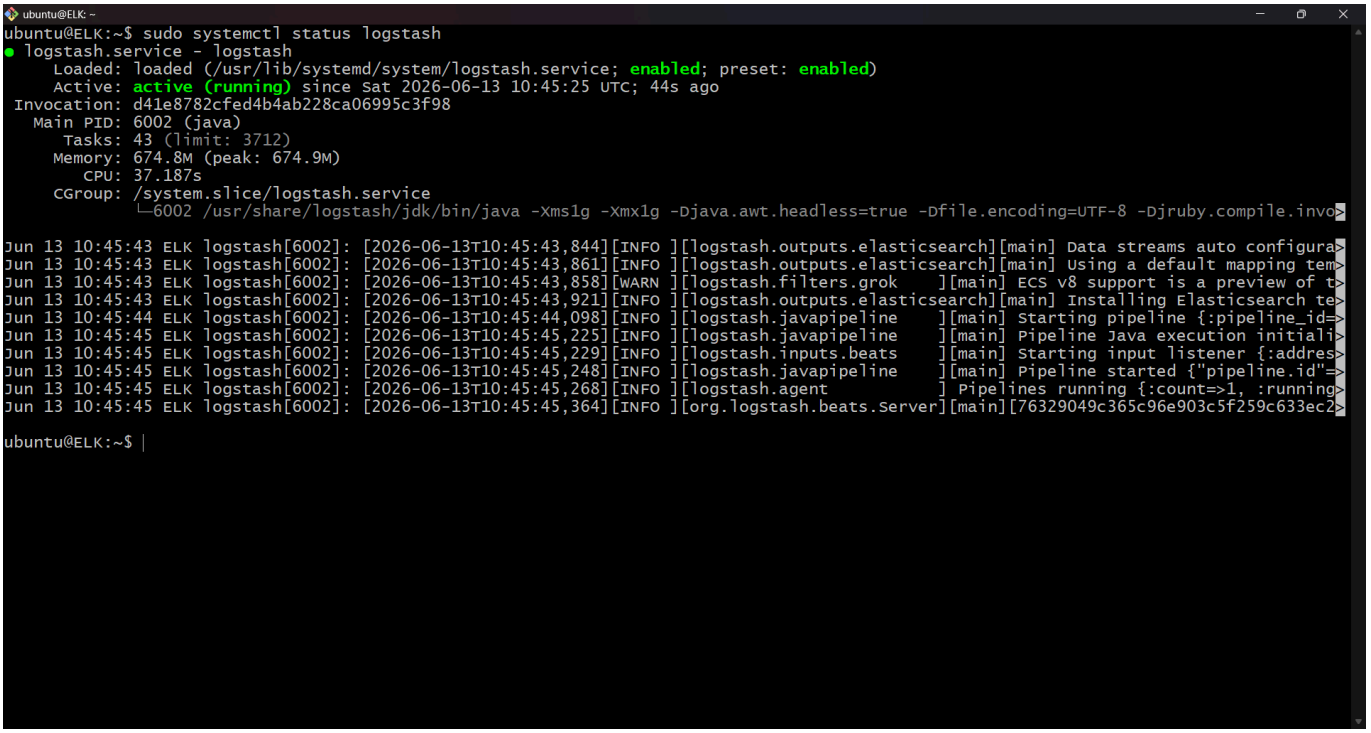
```
sudo systemctl start logstash
sudo systemctl enable logstash
```

> **Logstash status** showing active (running) — pipeline listening on port 5044.


ubuntu@ELK: ~
ubuntu@ELK:~$ sudo apt install logstash -y
Installing:
logstash
Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
Download size: 456 MB
Space needed: 740 MB / 15.2 GB available
Get:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 logstash amd64 1:8.19.16-1 [456 MB]
Fetched 456 MB in 3s (150 MB/s)
Selecting previously unselected package logstash.
(Reading database ... 100150 files and directories currently installed.)
Preparing to unpack .../logstash_1%3a8.19.16-1_amd64.deb ...
Unpacking logstash (1:8.19.16-1) ...
Setting up logstash (1:8.19.16-1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...
Running kernel seems to be up-to-date.
Restarting services...
Service restarts being deferred:
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service
No containers need to be restarted.
User sessions running outdated binaries:
ubuntu @ session #3: sshd-session[1434]
ubuntu @ user manager: (sd-pam)[1258]
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ELK:~$ sudo nano /etc/logstash/conf.d/nginx-pipeline.conf


ubuntu@ELK: ~
GNU nano 8.7.1 /etc/logstash/conf.d/nginx-pipeline.conf *
input {
beats {
port => 5044
}
}
filter {
if [fields][log_type] == "nginx_access" {
grok {
match => {
"message" => '%{IPORHOST:client_ip} - %{DATA:user} [%{HTTPDATE:timestamp}] "%{WORD:method} %{DATA:request} HTTP/%{NUMBER:http_
}
}
mutate {
convert => {
"response_code" => "integer"
"bytes" => "integer"
}
}
if [response_code] >= 500 {
mutate { add_tag => ["server_error", "error"] }
} else if [response_code] >= 400 {
mutate { add_tag => ["client_error", "error"] }
}
date {
match => ["timestamp", "dd/MMM/yyyy:HH:mm:ss Z"]
target => "@timestamp"
}
}
if [fields][log_type] == "nginx_error" {
mutate { add_tag => ["nginx_error"] }
}
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy





Part 4 — Kibana Setup

Install Kibana and configure it to connect to Elasticsearch and listen on all interfaces so the browser can access it.

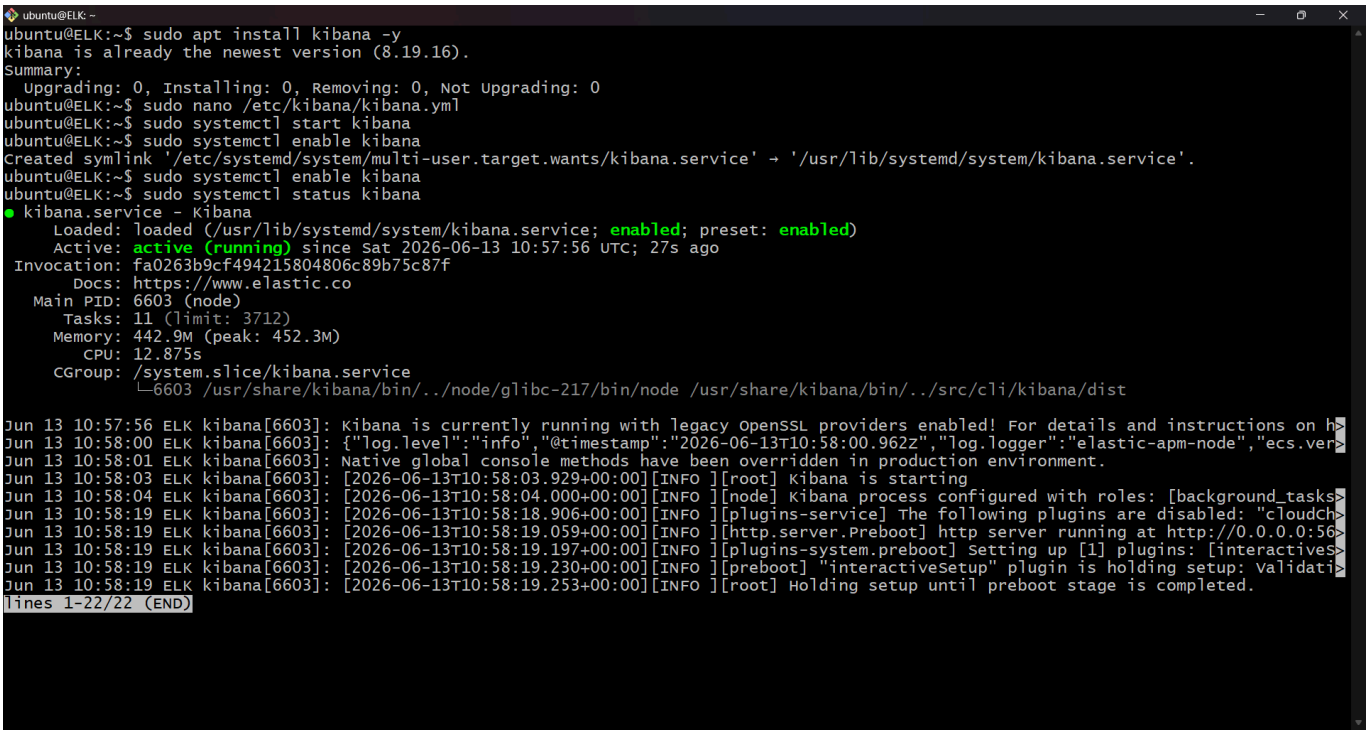
```
sudo apt install kibana -y
```

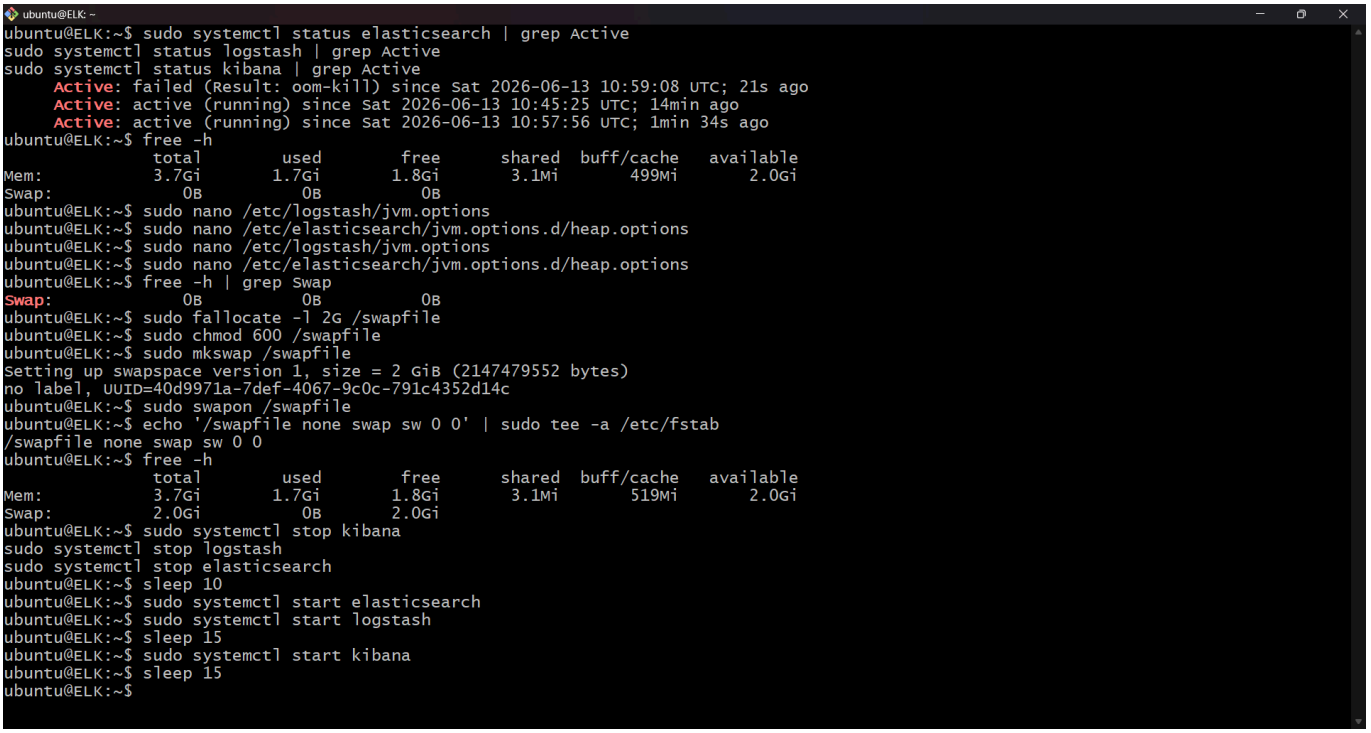
Kibana configuration (`/etc/kibana/kibana.yml`):

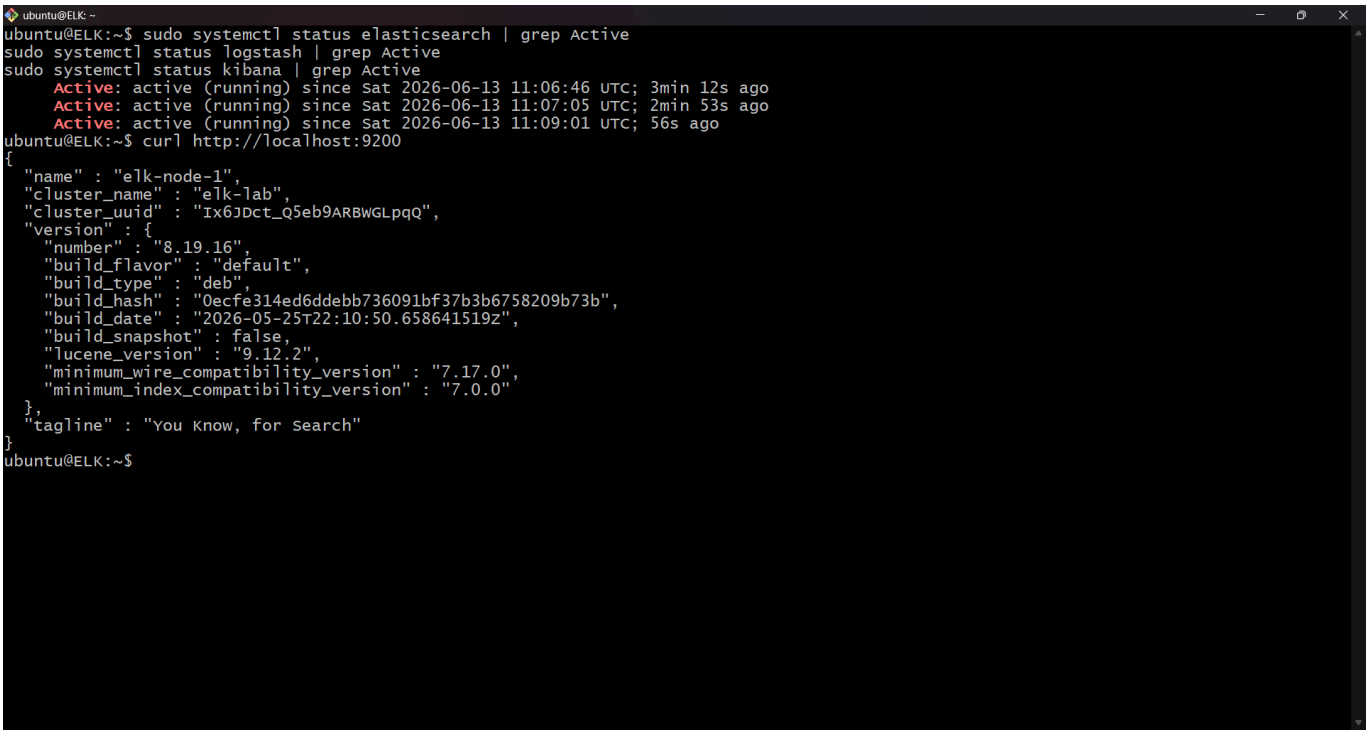
```
server.port: 5601
server.host: "0.0.0.0"
elasticsearch.hosts: ["http://localhost:9200"]
xpack.encryptedSavedObjects.encryptionKey: "your-32-char-key-here"
xpack.reporting.encryptionKey: "your-32-char-key-here"
xpack.security.encryptionKey: "your-32-char-key-here"
```

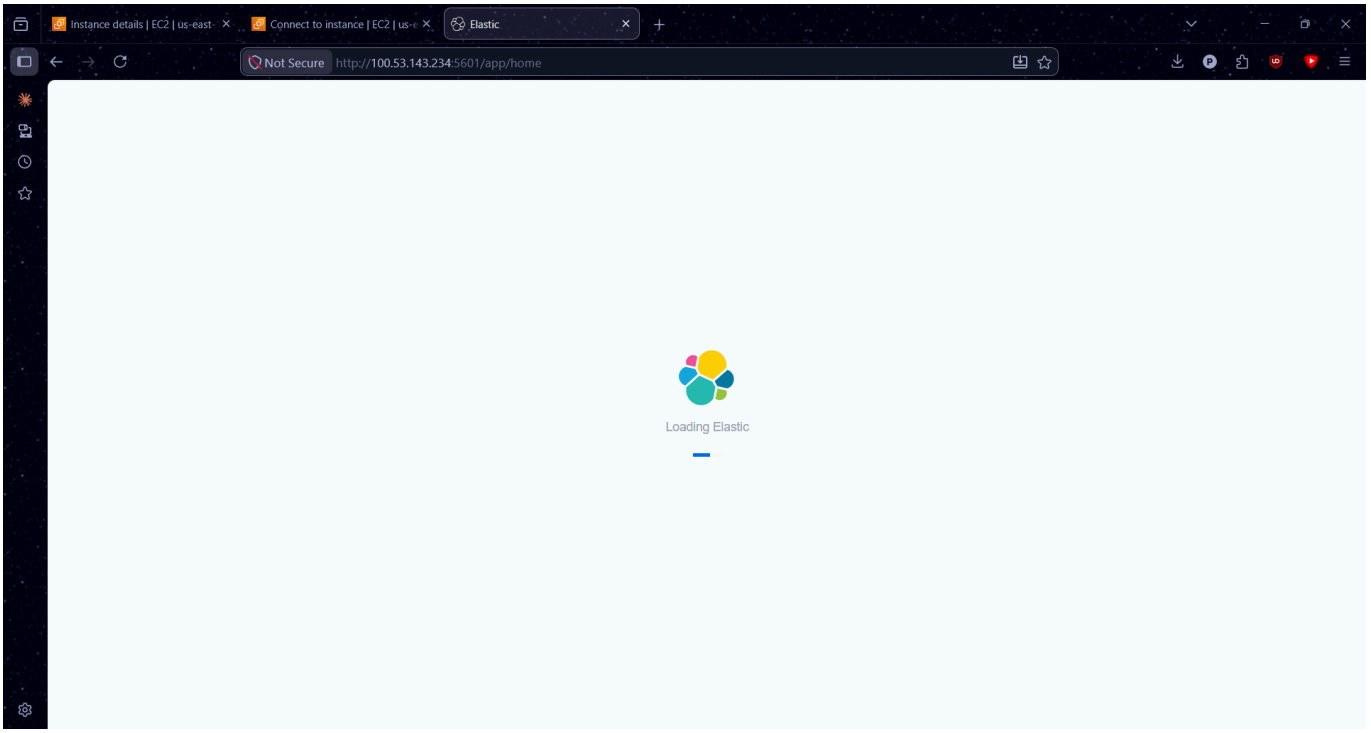
```
sudo systemctl start kibana
sudo systemctl enable kibana
```

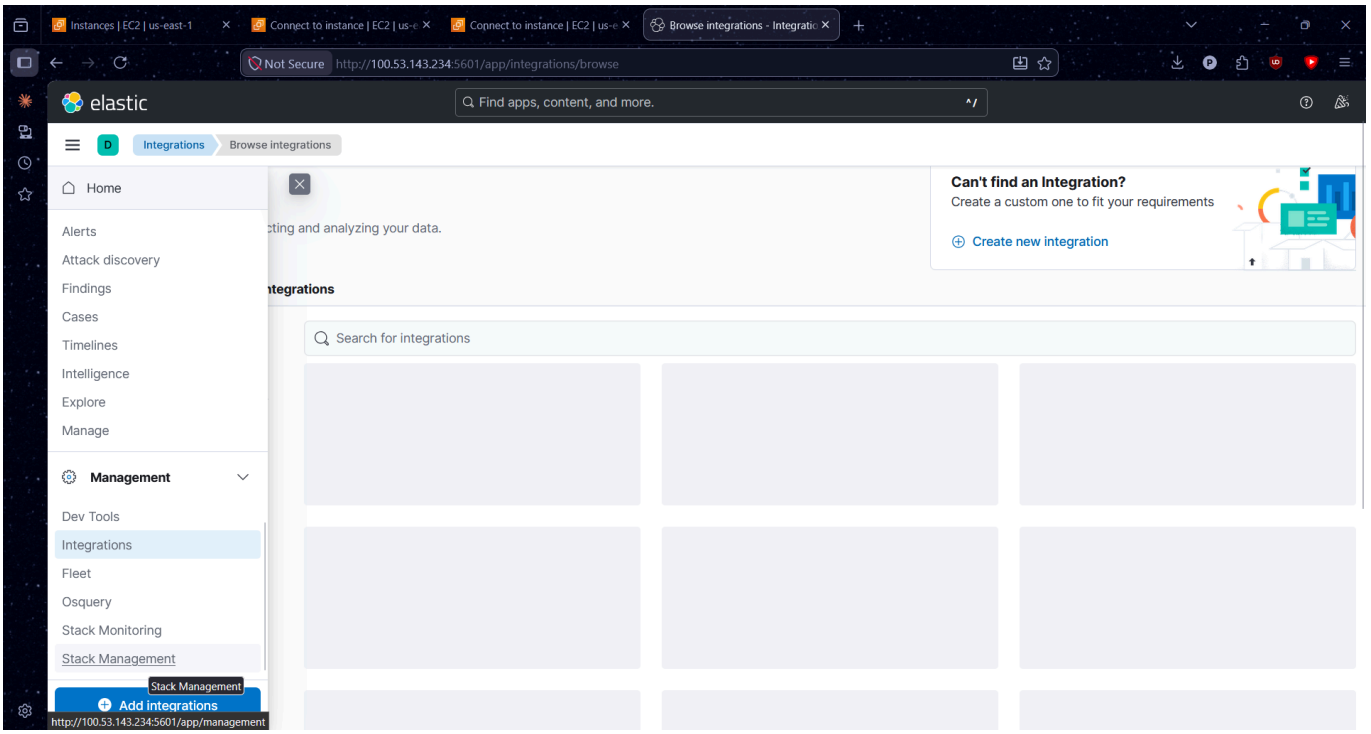
> **Kibana home page** accessible at http://ELK_PUBLIC_IP:5601 showing the Elastic home screen.


ubuntu@ELK: ~
ubuntu@ELK:~$ sudo apt install kibana -y
kibana is already the newest version (8.19.16).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ubuntu@ELK:~$ sudo nano /etc/kibana/kibana.yml
ubuntu@ELK:~$ sudo systemctl start kibana
ubuntu@ELK:~$ sudo systemctl enable kibana
Created symlink '/etc/systemd/system/multi-user.target.wants/kibana.service' → '/usr/lib/systemd/system/kibana.service'.
ubuntu@ELK:~$ sudo systemctl enable kibana
ubuntu@ELK:~$ sudo systemctl status kibana
● kibana.service - Kibana
Loaded: loaded (/usr/lib/systemd/system/kibana.service; enabled; preset: enabled)
Active: active (running) since Sat 2026-06-13 10:57:56 UTC; 27s ago
Invocation: fa0263b9cf494215804806c89b75c87f
Docs: https://www.elastic.co
Main PID: 6603 (node)
Tasks: 11 (limit: 3712)
Memory: 442.9M (peak: 452.3M)
CPU: 12.875s
CGroup: /system.slice/kibana.service
└─6603 /usr/share/kibana/bin/../node/glibc-217/bin/node /usr/share/kibana/bin/../src/cli/kibana/dist
Jun 13 10:57:56 ELK kibana[6603]: Kibana is currently running with legacy OpenSSL providers enabled! For details and instructions on h
Jun 13 10:58:00 ELK kibana[6603]: {"log.level":"info","@timestamp":"2026-06-13T10:58:00.962z","log.logger":"elastic-apm-node","ecs.ver
Jun 13 10:58:01 ELK kibana[6603]: Native global console methods have been overridden in production environment.
Jun 13 10:58:03 ELK kibana[6603]: [2026-06-13T10:58:03.929+00:00][INFO][root] Kibana is starting
Jun 13 10:58:04 ELK kibana[6603]: [2026-06-13T10:58:04.000+00:00][INFO][node] Kibana process configured with roles: [background_tasks
Jun 13 10:58:19 ELK kibana[6603]: [2026-06-13T10:58:18.906+00:00][INFO][plugins-service] The following plugins are disabled: "cloudCh
Jun 13 10:58:19 ELK kibana[6603]: [2026-06-13T10:58:19.059+00:00][INFO][http.server.Preboot] http server running at http://0.0.0.0:56
Jun 13 10:58:19 ELK kibana[6603]: [2026-06-13T10:58:19.197+00:00][INFO][plugins-system.preboot] Setting up [1] plugins: [interactiveS
Jun 13 10:58:19 ELK kibana[6603]: [2026-06-13T10:58:19.230+00:00][INFO][preboot] "interactiveSetup" plugin is holding setup: Validati
Jun 13 10:58:19 ELK kibana[6603]: [2026-06-13T10:58:19.253+00:00][INFO][root] Holding setup until preboot stage is completed.
lines 1-22/22 (END)


ubuntu@ELK: ~
ubuntu@ELK:~$ sudo systemctl status elasticsearch | grep Active
sudo systemctl status logstash | grep Active
sudo systemctl status kibana | grep Active
Active: failed (Result: oom-kill) since Sat 2026-06-13 10:59:08 UTC; 21s ago
Active: active (running) since Sat 2026-06-13 10:45:25 UTC; 14min ago
Active: active (running) since Sat 2026-06-13 10:57:56 UTC; 1min 34s ago
ubuntu@ELK:~$ free -h
total used free shared buff/cache available
Mem: 3.7Gi 1.7Gi 1.8Gi 3.1Mi 499Mi 2.0Gi
Swap: 0B 0B 0B
ubuntu@ELK:~$ sudo nano /etc/logstash/jvm.options
ubuntu@ELK:~$ sudo nano /etc/elasticsearch/jvm.options.d/heap.options
ubuntu@ELK:~$ sudo nano /etc/logstash/jvm.options
ubuntu@ELK:~$ sudo nano /etc/elasticsearch/jvm.options.d/heap.options
ubuntu@ELK:~$ free -h | grep Swap
Swap: 0B 0B 0B
ubuntu@ELK:~$ sudo fallocate -l 2G /swapfile
ubuntu@ELK:~$ sudo chmod 600 /swapfile
ubuntu@ELK:~$ sudo mkswap /swapfile
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=40d9971a-7def-4067-9c0c-791c4352d14c
ubuntu@ELK:~$ sudo swapon /swapfile
ubuntu@ELK:~$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
/swapfile none swap sw 0 0
ubuntu@ELK:~$ free -h
total used free shared buff/cache available
Mem: 3.7Gi 1.7Gi 1.8Gi 3.1Mi 519Mi 2.0Gi
Swap: 2.0Gi 0B 2.0Gi
ubuntu@ELK:~$ sudo systemctl stop kibana
sudo systemctl stop logstash
sudo systemctl stop elasticsearch
ubuntu@ELK:~$ sleep 10
ubuntu@ELK:~$ sudo systemctl start elasticsearch
ubuntu@ELK:~$ sudo systemctl start logstash
ubuntu@ELK:~$ sleep 15
ubuntu@ELK:~$ sudo systemctl start kibana
ubuntu@ELK:~$ sleep 15
ubuntu@ELK:~$


ubuntu@ELK: ~
ubuntu@ELK:~$ sudo systemctl status elasticsearch | grep Active
sudo systemctl status logstash | grep Active
sudo systemctl status kibana | grep Active
Active: active (running) since Sat 2026-06-13 11:06:46 UTC; 3min 12s ago
Active: active (running) since Sat 2026-06-13 11:07:05 UTC; 2min 53s ago
Active: active (running) since Sat 2026-06-13 11:09:01 UTC; 56s ago
ubuntu@ELK:~$ curl http://localhost:9200
{
"name" : "elk-node-1",
"cluster_name" : "elk-lab",
"cluster_uuid" : "Ix6JDct_Q5eb9ARBWGLpqQ",
"version" : {
"number" : "8.19.16",
"build_flavor" : "default",
"build_type" : "deb",
"build_hash" : "0ecfe314ed6ddebb736091bf37b3b6758209b73b",
"build_date" : "2026-05-25T22:10:50.658641519Z",
"build_snapshot" : false,
"lucene_version" : "9.12.2",
"minimum_wire_compatibility_version" : "7.17.0",
"minimum_index_compatibility_version" : "7.0.0"
},
"tagline" : "You Know, for Search"
}
ubuntu@ELK:~$


Instance details | EC2 | us-east-
Connect to instance | EC2 | us-e
Elastic
Not Secure
http://100.53.143.234:5601/app/home
Loading Elastic



Part 5 — Nginx and Filebeat on App Servers

Install Nginx and Filebeat on both app servers. Filebeat watches the Nginx log files and ships new lines to Logstash using the ELK server's private IP to keep traffic inside the VPC.

```
# Install Nginx
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx

# Install Filebeat
sudo apt install filebeat -y
```

Filebeat configuration (`/etc/filebeat/filebeat.yml`):

```
filebeat.inputs:
  - type: log
    enabled: true
    paths:
      - /var/log/nginx/access.log
    fields:
      log_type: nginx_access
      server: app-server-1
    fields_under_root: true

  - type: log
    enabled: true
    paths:
      - /var/log/nginx/error.log
```

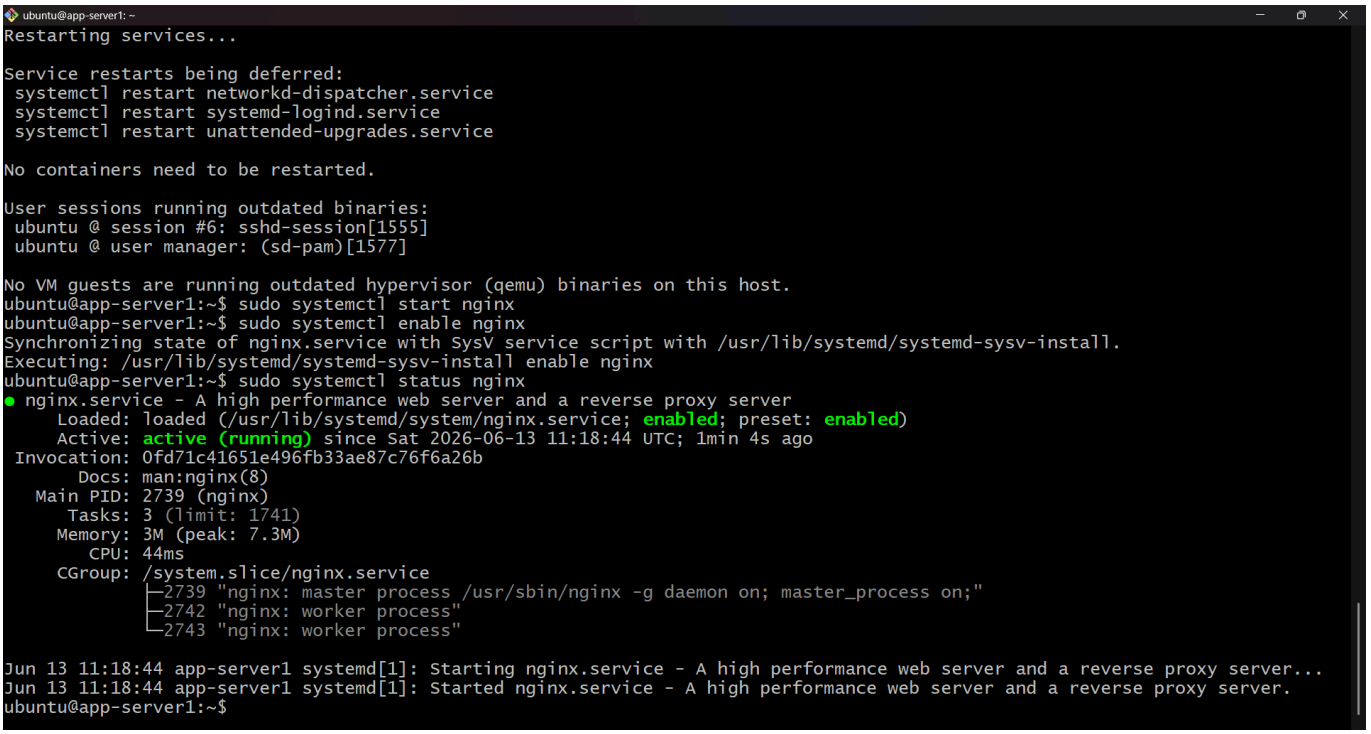
```
    fields:
      log_type: nginx_error
      server: app-server-1
    fields_under_root: true

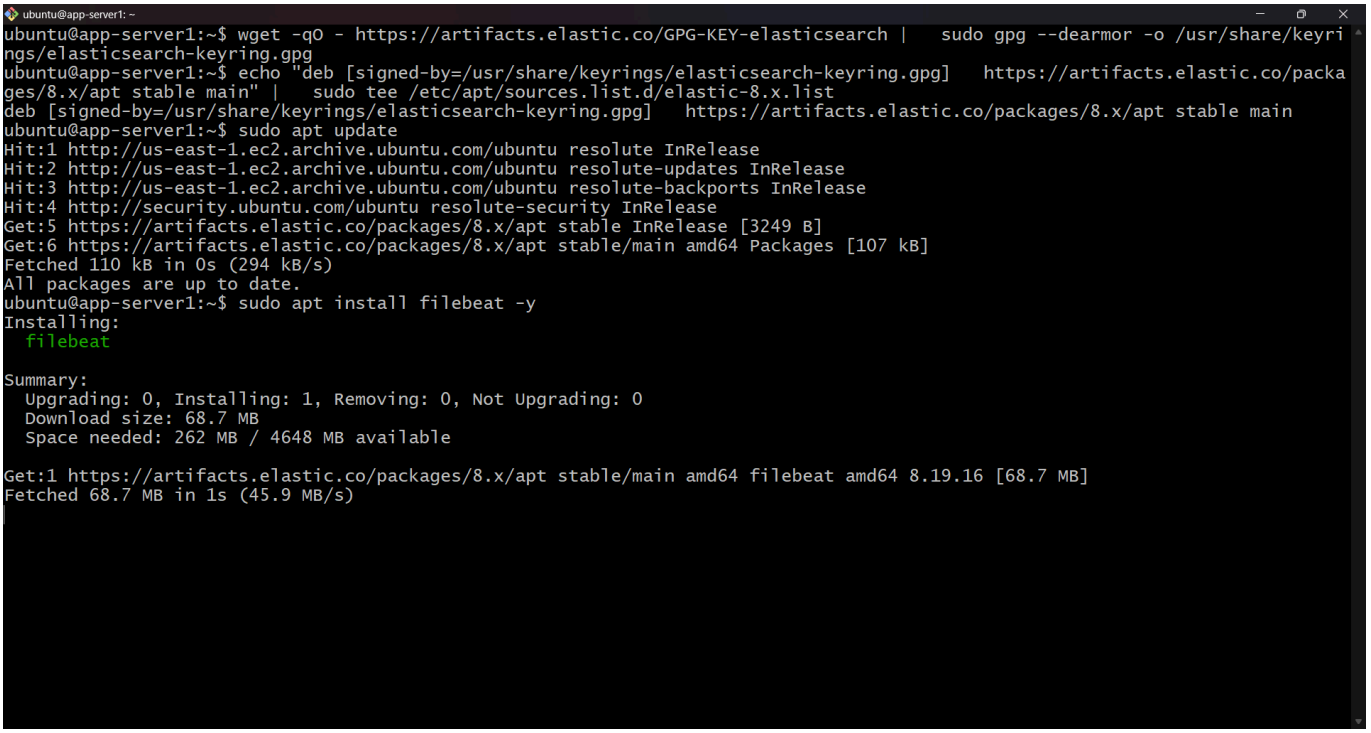
output.logstash:
  hosts: ["ELK_PRIVATE_IP:5044"]
```

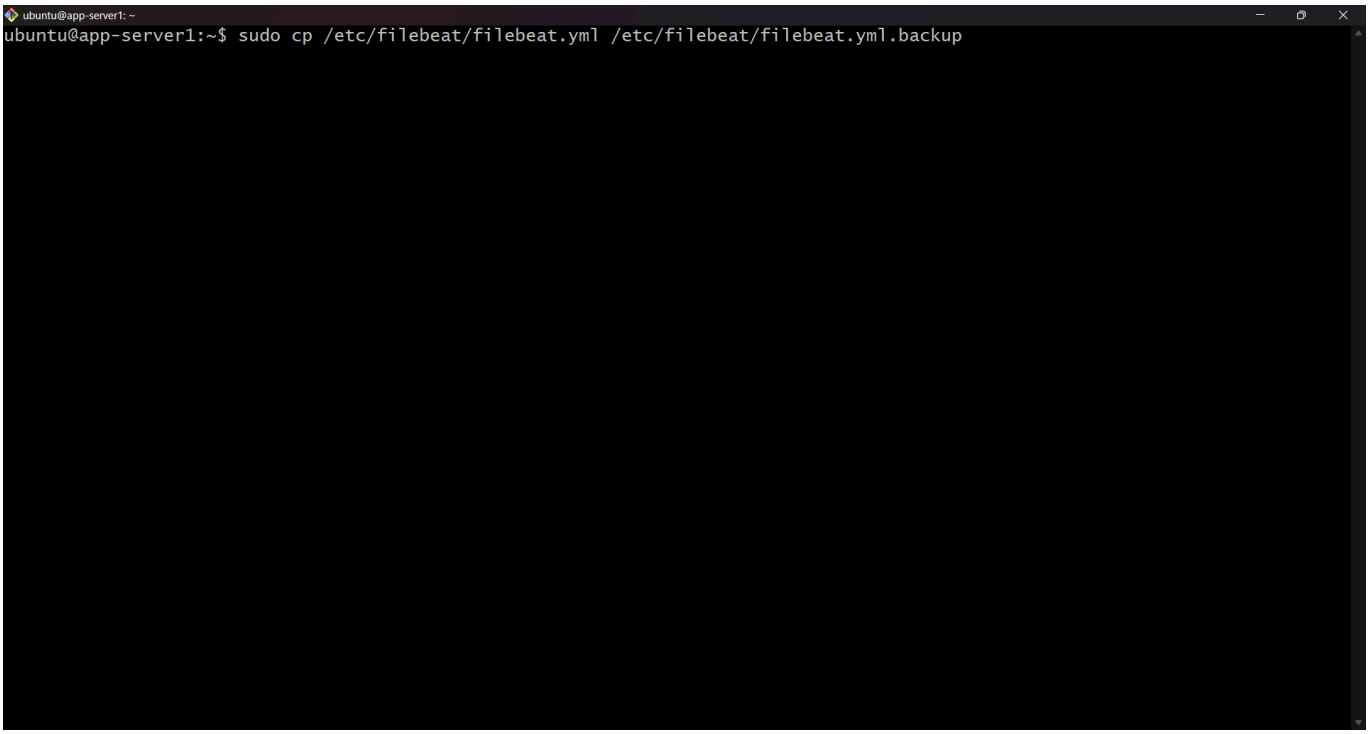
```
sudo systemctl start filebeat
sudo systemctl enable filebeat

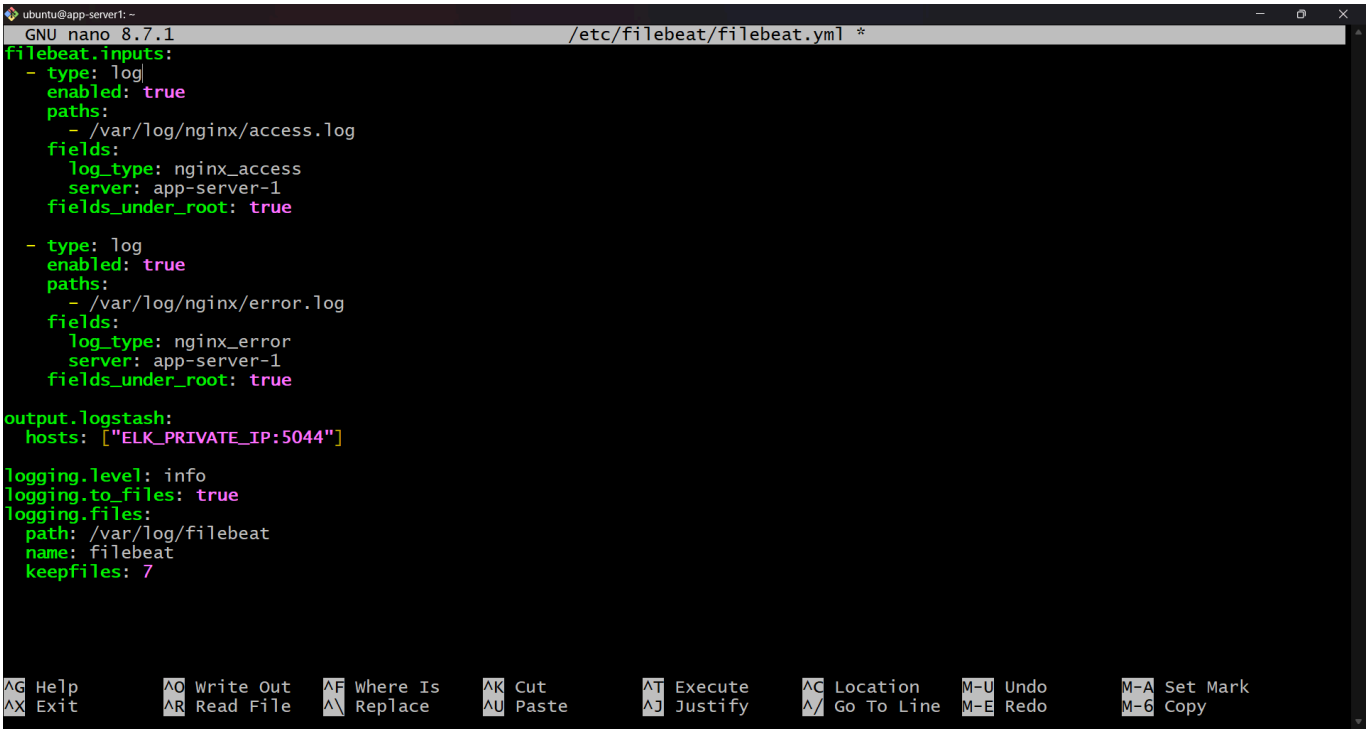
# Test connection to Logstash
sudo filebeat test output
```

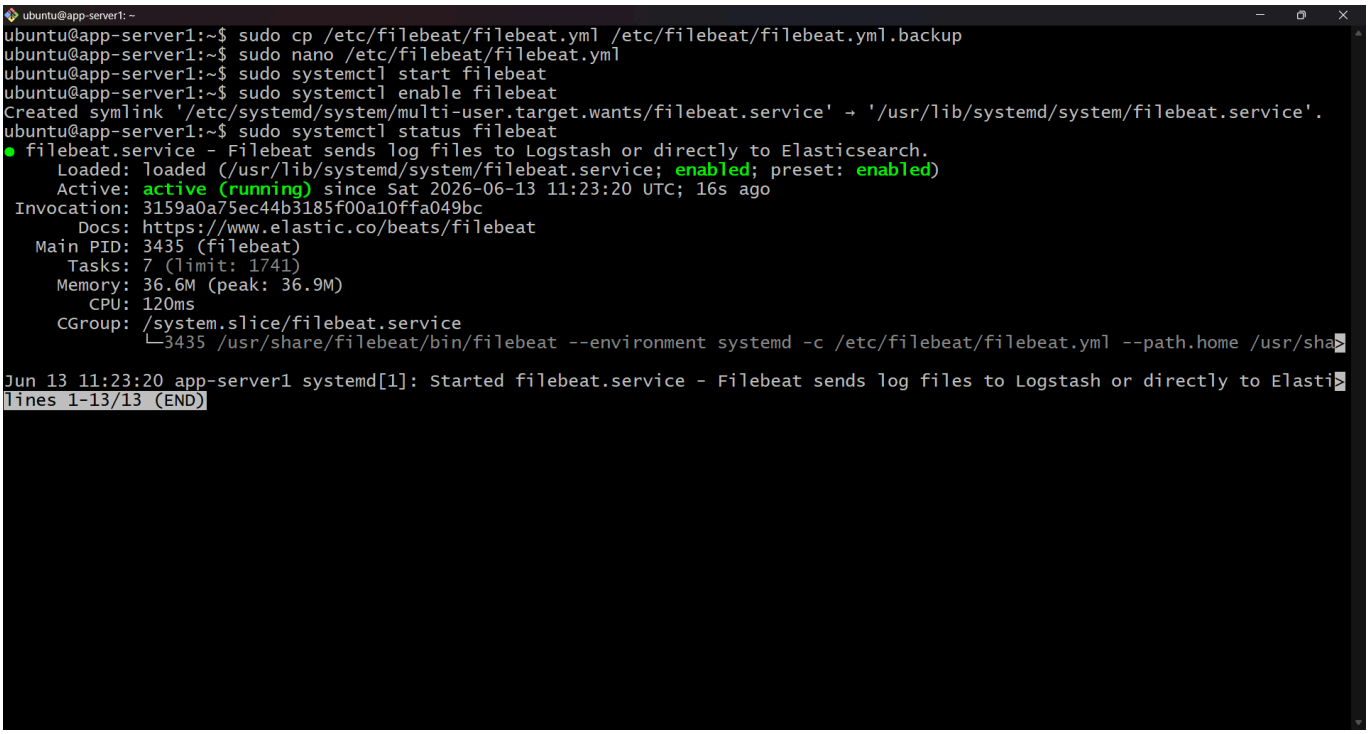
> **Filebeat test output on app-server-1** — showing parse host OK, dns lookup OK, dial up OK, talk to server OK.

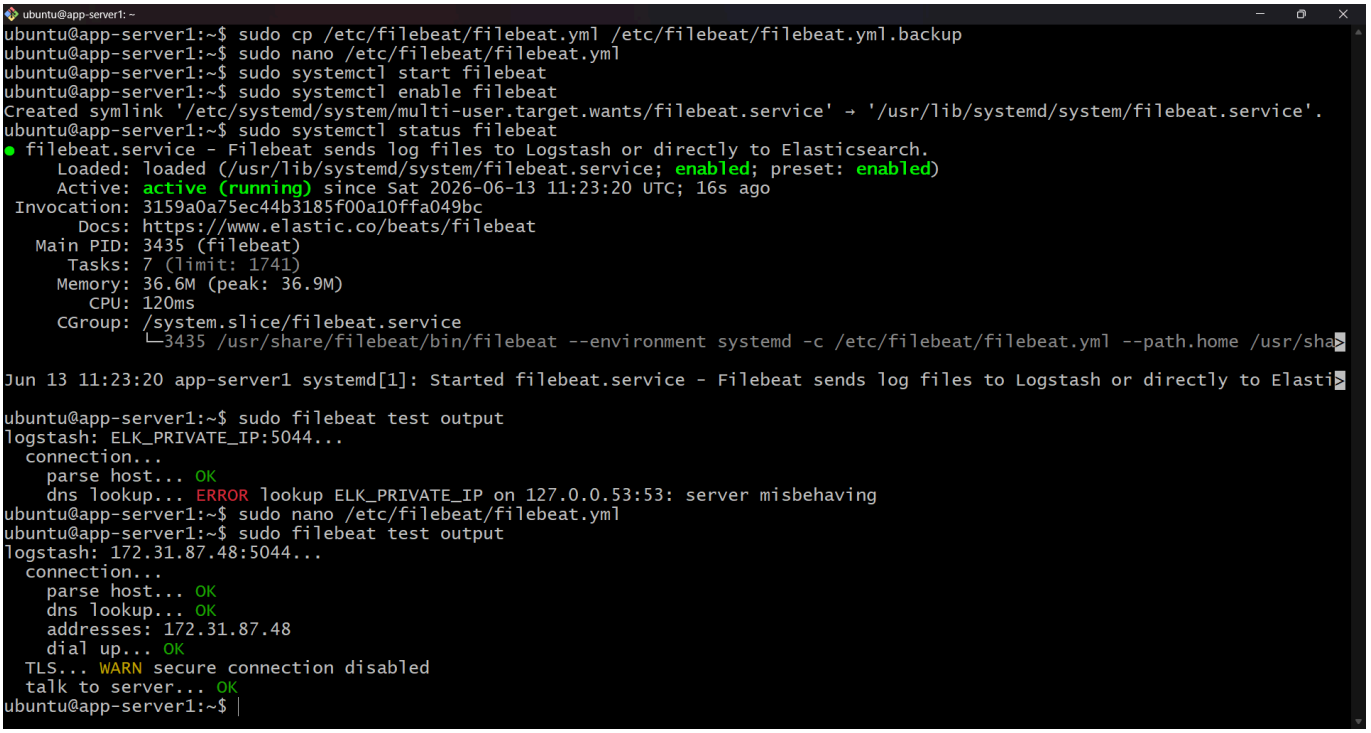

ubuntu@app-server1: ~
Restarting services...
Service restarts being deferred:
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service
No containers need to be restarted.
User sessions running outdated binaries:
ubuntu @ session #6: sshd-session[1555]
ubuntu @ user manager: (sd-pam)[1577]
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@app-server1:~$ sudo systemctl start nginx
ubuntu@app-server1:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@app-server1:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
Active: active (running) since Sat 2026-06-13 11:18:44 UTC; 1min 4s ago
Invocation: 0fd71c41651e496fb33ae87c76f6a26b
Docs: man:nginx(8)
Main PID: 2739 (nginx)
Tasks: 3 (limit: 1741)
Memory: 3M (peak: 7.3M)
CPU: 44ms
CGroup: /system.slice/nginx.service
├─2739 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
├─2742 "nginx: worker process"
└─2743 "nginx: worker process"
Jun 13 11:18:44 app-server1 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jun 13 11:18:44 app-server1 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@app-server1:~$

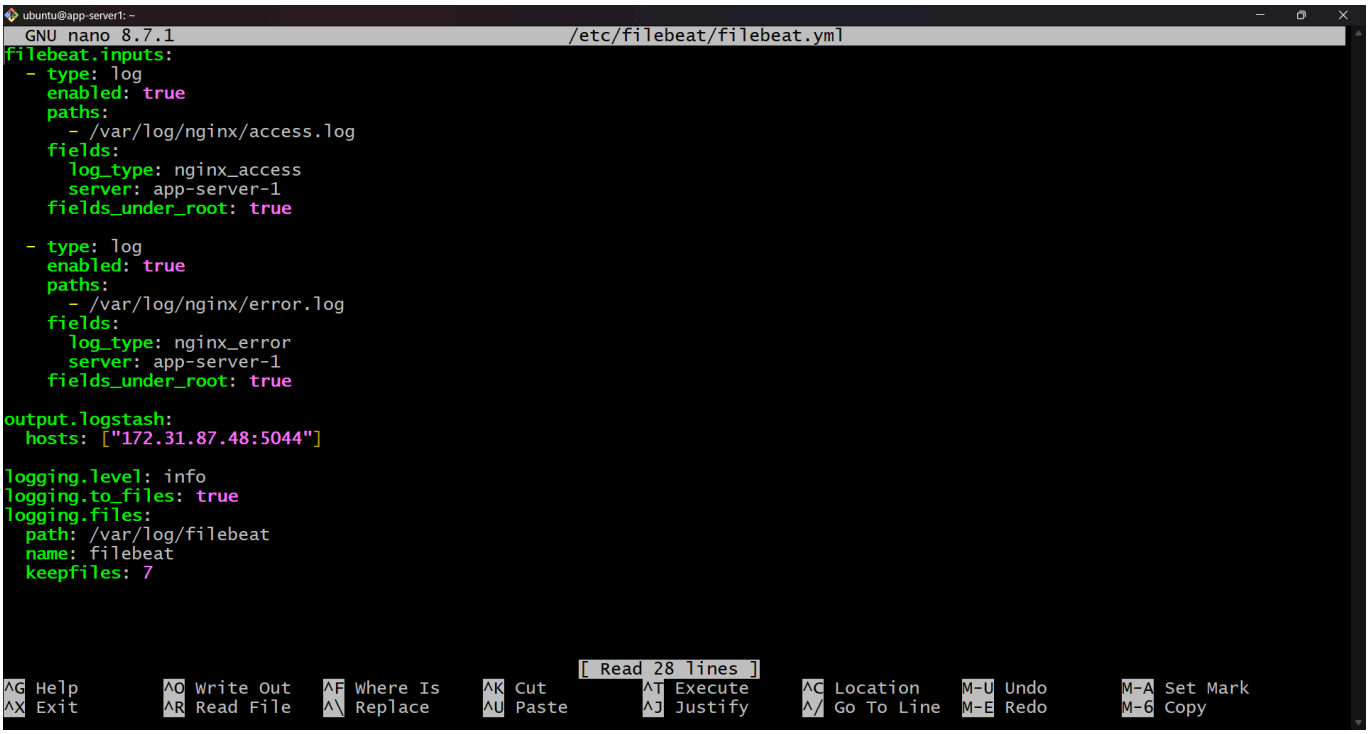

ubuntu@app-server1: ~
ubuntu@app-server1:~$ wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
ubuntu@app-server1:~$ echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elastic-8.x.list
deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main
ubuntu@app-server1:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
Get:5 https://artifacts.elastic.co/packages/8.x/apt stable InRelease [3249 B]
Get:6 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 Packages [107 kB]
Fetched 110 kB in 0s (294 kB/s)
All packages are up to date.
ubuntu@app-server1:~$ sudo apt install filebeat -y
Installing:
filebeat
Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
Download size: 68.7 MB
Space needed: 262 MB / 4648 MB available
Get:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 filebeat amd64 8.19.16 [68.7 MB]
Fetched 68.7 MB in 1s (45.9 MB/s)

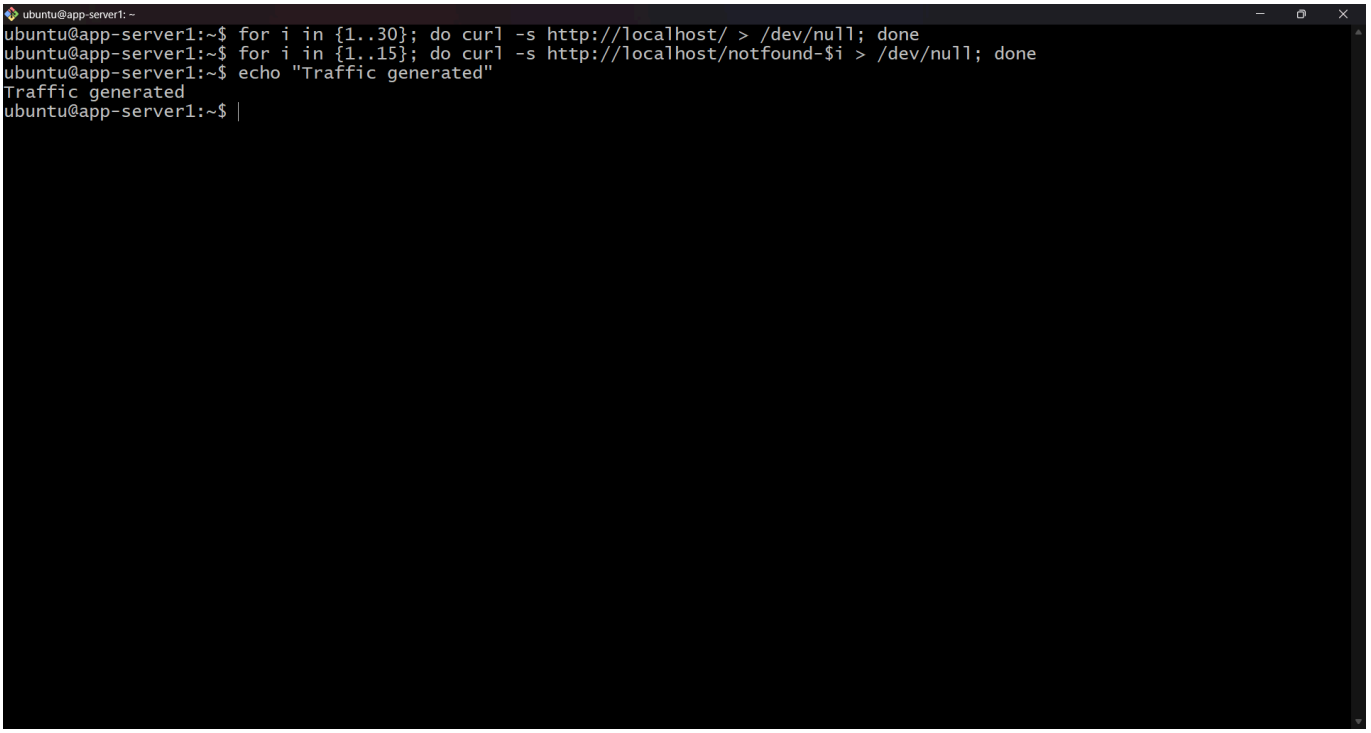

ubuntu@app-server1: ~
ubuntu@app-server1:~$ sudo cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.backup


ubuntu@app-server1: ~
GNU nano 8.7.1 /etc/filebeat/filebeat.yml *
filebeat.inputs:
- type: log
enabled: true
paths:
- /var/log/nginx/access.log
fields:
log_type: nginx_access
server: app-server-1
fields_under_root: true
- type: log
enabled: true
paths:
- /var/log/nginx/error.log
fields:
log_type: nginx_error
server: app-server-1
fields_under_root: true
output.logstash:
hosts: ["ELK_PRIVATE_IP:5044"]
logging.level: info
logging.to_files: true
logging.files:
path: /var/log/filebeat
name: filebeat
keepfiles: 7
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

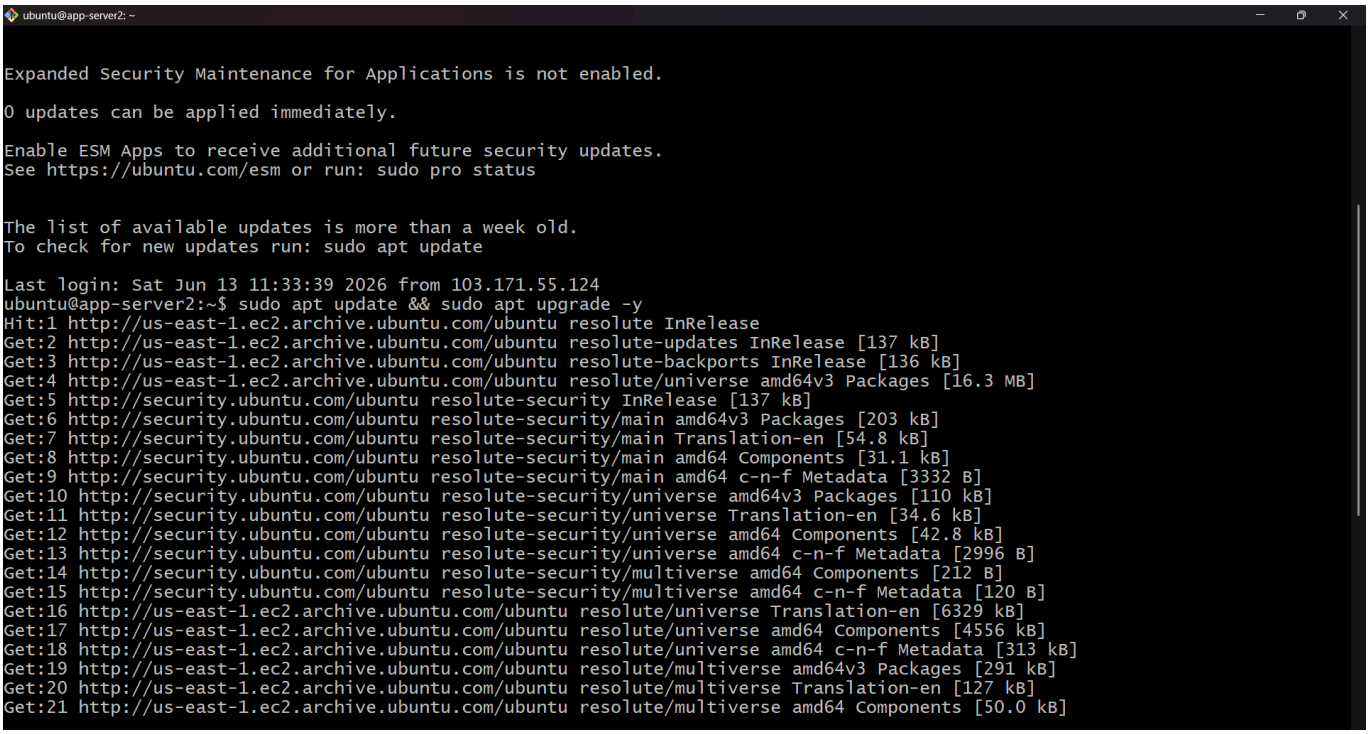

ubuntu@app-server1: ~
ubuntu@app-server1:~$ sudo cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.backup
ubuntu@app-server1:~$ sudo nano /etc/filebeat/filebeat.yml
ubuntu@app-server1:~$ sudo systemctl start filebeat
ubuntu@app-server1:~$ sudo systemctl enable filebeat
Created symlink '/etc/systemd/system/multi-user.target.wants/filebeat.service' → '/usr/lib/systemd/system/filebeat.service'.
ubuntu@app-server1:~$ sudo systemctl status filebeat
● filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
Loaded: loaded (/usr/lib/systemd/system/filebeat.service; enabled; preset: enabled)
Active: active (running) since Sat 2026-06-13 11:23:20 UTC; 16s ago
Invocation: 3159a0a75ec44b3185f00a10ffa049bc
Docs: https://www.elastic.co/beats/filebeat
Main PID: 3435 (filebeat)
Tasks: 7 (limit: 1741)
Memory: 36.6M (peak: 36.9M)
CPU: 120ms
CGroup: /system.slice/filebeat.service
└─3435 /usr/share/filebeat/bin/filebeat --environment systemd -c /etc/filebeat/filebeat.yml --path.home /usr/sha
Jun 13 11:23:20 app-server1 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasti
lines 1-13/13 (END)

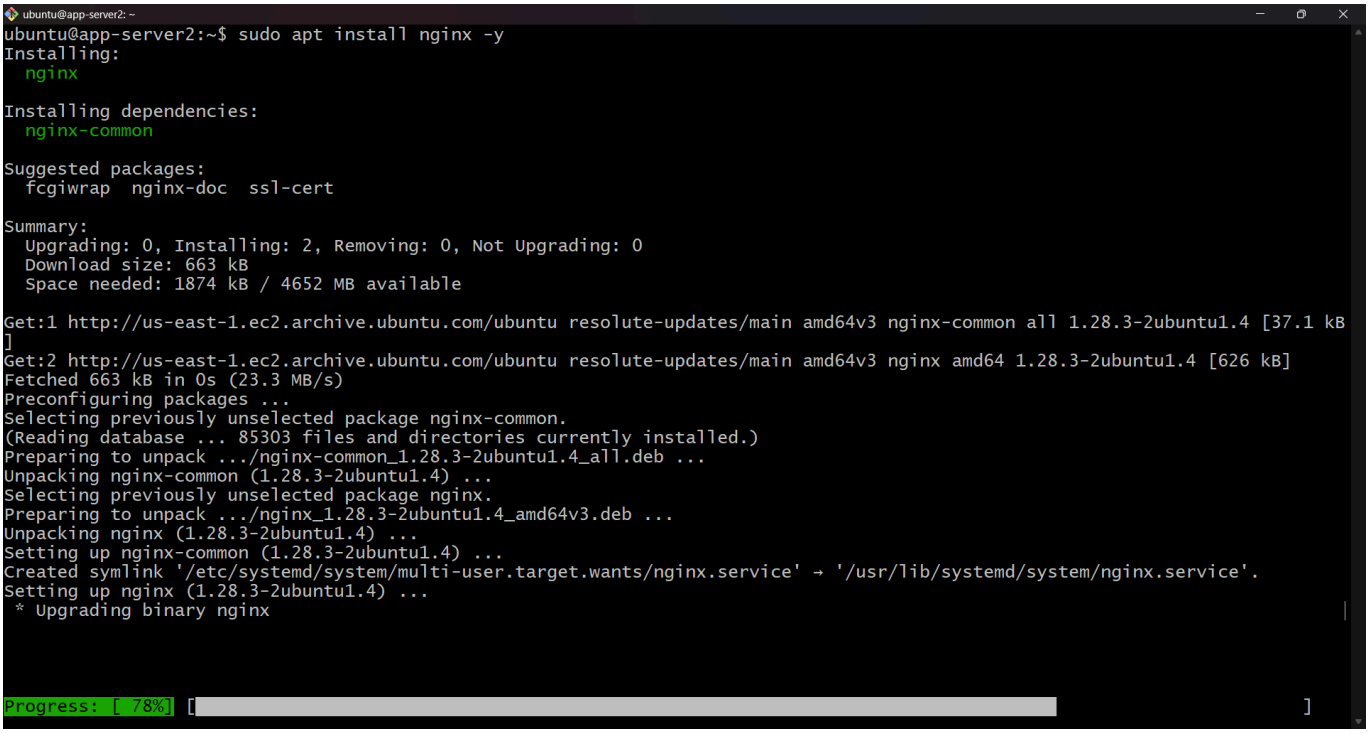

ubuntu@app-server1: ~
ubuntu@app-server1:~$ sudo cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.backup
ubuntu@app-server1:~$ sudo nano /etc/filebeat/filebeat.yml
ubuntu@app-server1:~$ sudo systemctl start filebeat
ubuntu@app-server1:~$ sudo systemctl enable filebeat
Created symlink '/etc/systemd/system/multi-user.target.wants/filebeat.service' → '/usr/lib/systemd/system/filebeat.service'.
ubuntu@app-server1:~$ sudo systemctl status filebeat
● filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
Loaded: loaded (/usr/lib/systemd/system/filebeat.service; enabled; preset: enabled)
Active: active (running) since Sat 2026-06-13 11:23:20 UTC; 16s ago
Invocation: 3159a0a75ec44b3185f00a10ffa049bc
Docs: https://www.elastic.co/beats/filebeat
Main PID: 3435 (filebeat)
Tasks: 7 (limit: 1741)
Memory: 36.6M (peak: 36.9M)
CPU: 120ms
CGroup: /system.slice/filebeat.service
└─3435 /usr/share/filebeat/bin/filebeat --environment systemd -c /etc/filebeat/filebeat.yml --path.home /usr/sha
Jun 13 11:23:20 app-server1 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasti
ubuntu@app-server1:~$ sudo filebeat test output
logstash: ELK_PRIVATE_IP:5044...
connection...
parse host... OK
dns lookup... ERROR lookup ELK_PRIVATE_IP on 127.0.0.53:53: server misbehaving
ubuntu@app-server1:~$ sudo nano /etc/filebeat/filebeat.yml
ubuntu@app-server1:~$ sudo filebeat test output
logstash: 172.31.87.48:5044...
connection...
parse host... OK
dns lookup... OK
addresses: 172.31.87.48
dial up... OK
TLS... WARN secure connection disabled
talk to server... OK
ubuntu@app-server1:~$

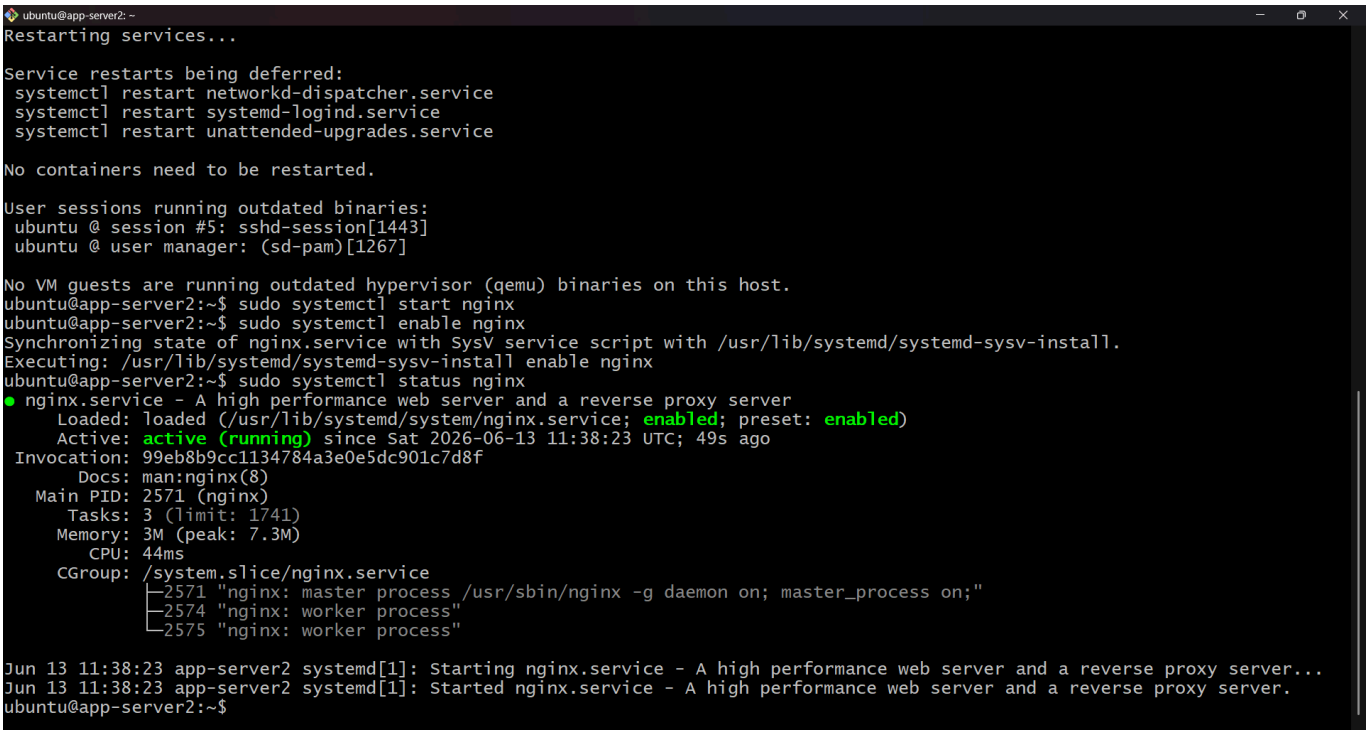


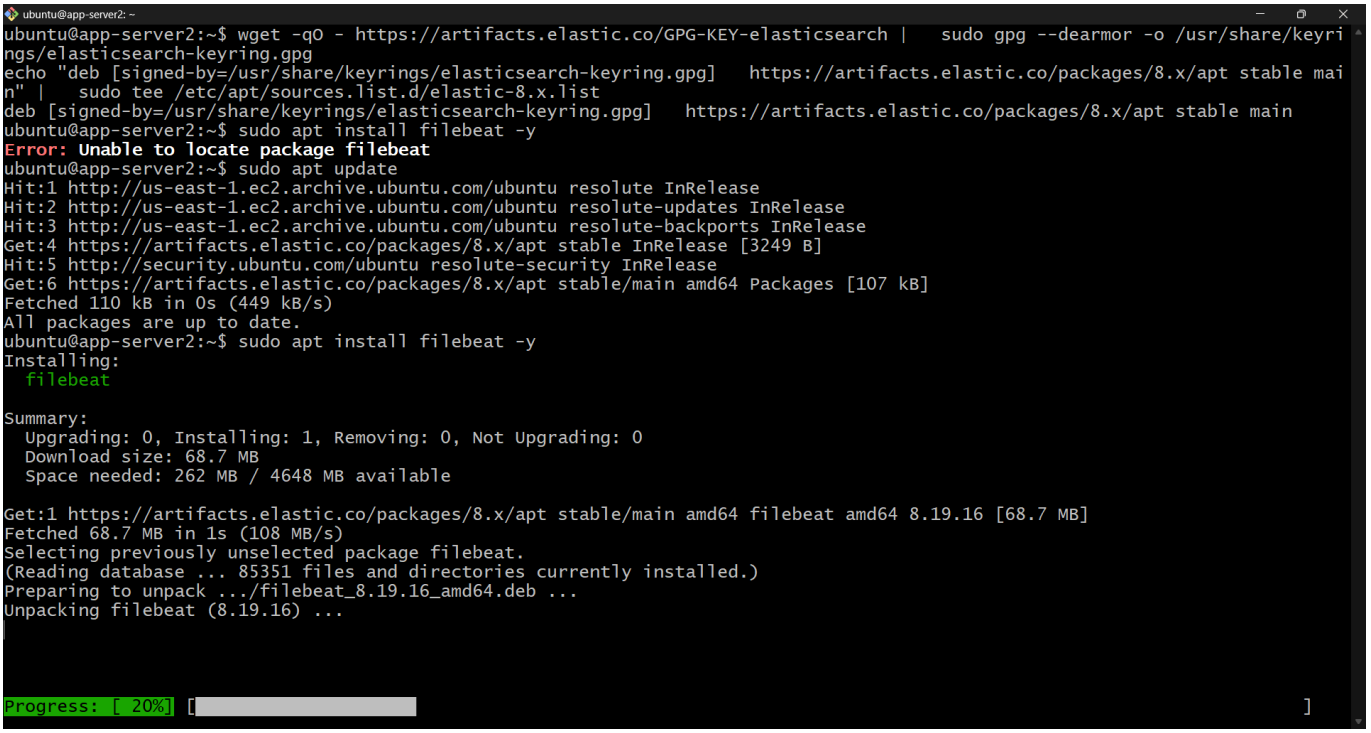


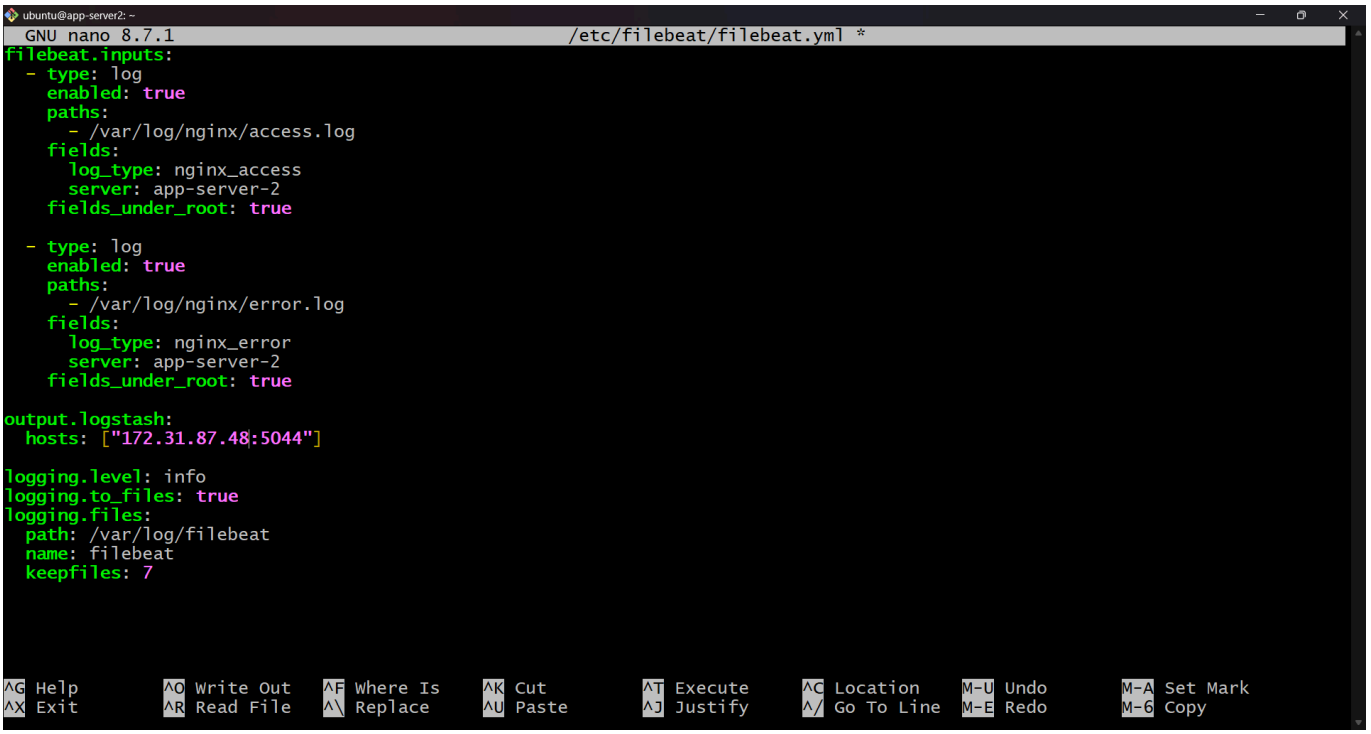
> **Filebeat test output on app-server-2** — same successful connection to Logstash.

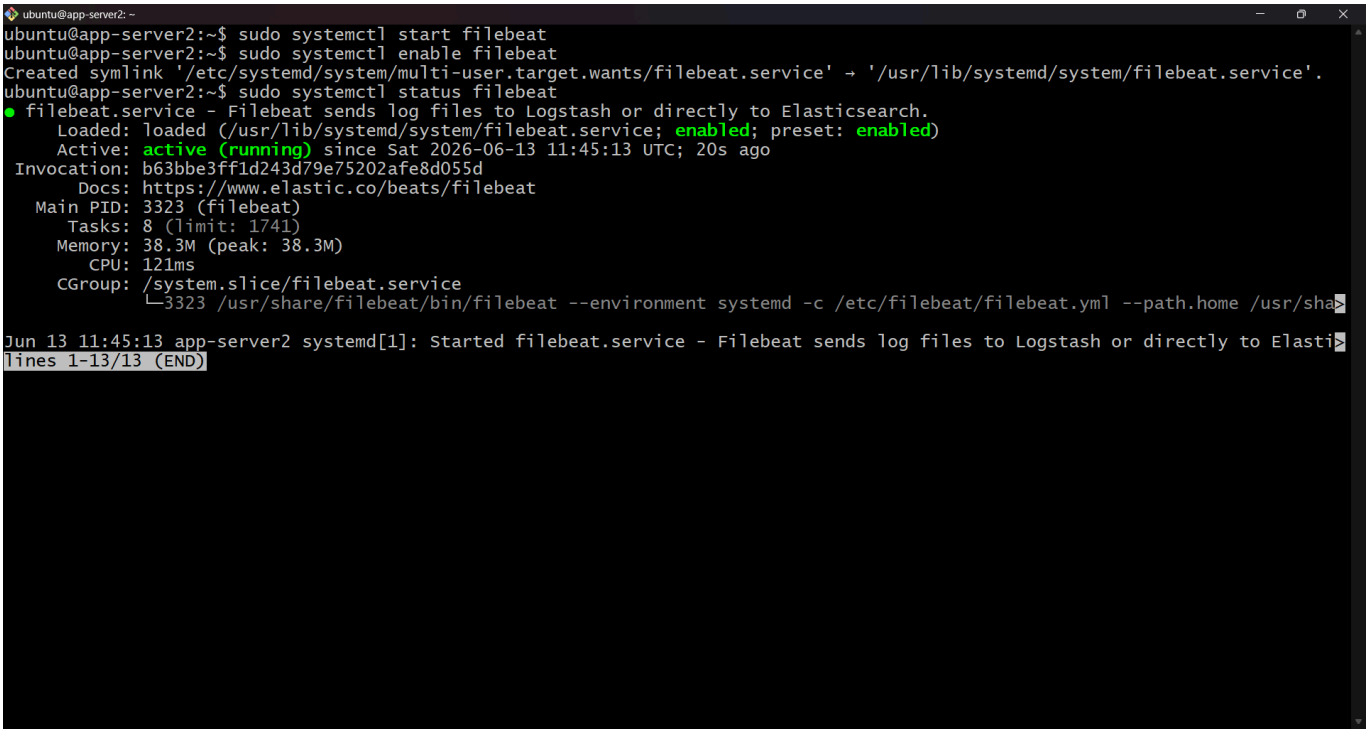

ubuntu@app-server2: ~
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Sat Jun 13 11:33:39 2026 from 103.171.55.124
ubuntu@app-server2:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 Packages [16.3 MB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64v3 Packages [203 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [54.8 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [31.1 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/main amd64 c-n-f Metadata [3332 B]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe amd64v3 Packages [110 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [34.6 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [42.8 kB]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 c-n-f Metadata [2996 B]
Get:14 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 Components [212 B]
Get:15 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 c-n-f Metadata [120 B]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe Translation-en [6329 kB]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64 Components [4556 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64 c-n-f Metadata [313 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/multiverse amd64v3 Packages [291 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/multiverse Translation-en [127 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/multiverse amd64 Components [50.0 kB]


ubuntu@app-server2: ~
ubuntu@app-server2:~$ sudo apt install nginx -y
Installing:
nginx
Installing dependencies:
nginx-common
Suggested packages:
fcgiwrap nginx-doc ssl-cert
Summary:
Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
Download size: 663 kB
Space needed: 1874 kB / 4652 MB available
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 nginx-common all 1.28.3-2ubuntu1.4 [37.1 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 nginx amd64 1.28.3-2ubuntu1.4 [626 kB]
Fetched 663 kB in 0s (23.3 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 85303 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.4_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.4) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.4_amd64v3.deb ...
Unpacking nginx (1.28.3-2ubuntu1.4) ...
Setting up nginx-common (1.28.3-2ubuntu1.4) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1.4) ...
* Upgrading binary nginx
Progress: [78%] [


ubuntu@app-server2: ~
Restarting services...
Service restarts being deferred:
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service
No containers need to be restarted.
User sessions running outdated binaries:
ubuntu @ session #5: sshd-session[1443]
ubuntu @ user manager: (sd-pam)[1267]
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@app-server2:~$ sudo systemctl start nginx
ubuntu@app-server2:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@app-server2:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
Active: active (running) since Sat 2026-06-13 11:38:23 UTC; 49s ago
Invocation: 99eb8b9cc1134784a3e0e5dc901c7d8f
Docs: man:nginx(8)
Main PID: 2571 (nginx)
Tasks: 3 (limit: 1741)
Memory: 3M (peak: 7.3M)
CPU: 44ms
CGroup: /system.slice/nginx.service
├─2571 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
├─2574 "nginx: worker process"
└─2575 "nginx: worker process"
Jun 13 11:38:23 app-server2 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jun 13 11:38:23 app-server2 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@app-server2:~$


ubuntu@app-server2: ~
ubuntu@app-server2:~$ wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elastic-8.x.list
deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main
ubuntu@app-server2:~$ sudo apt install filebeat -y
Error: Unable to locate package filebeat
ubuntu@app-server2:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:4 https://artifacts.elastic.co/packages/8.x/apt stable InRelease [3249 B]
Hit:5 http://security.ubuntu.com/ubuntu resolute-security InRelease
Get:6 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 Packages [107 kB]
Fetched 110 kB in 0s (449 kB/s)
All packages are up to date.
ubuntu@app-server2:~$ sudo apt install filebeat -y
Installing:
filebeat
Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
Download size: 68.7 MB
Space needed: 262 MB / 4648 MB available
Get:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 filebeat amd64 8.19.16 [68.7 MB]
Fetched 68.7 MB in 1s (108 MB/s)
Selecting previously unselected package filebeat.
(Reading database ... 85351 files and directories currently installed.)
Preparing to unpack .../filebeat_8.19.16_amd64.deb ...
Unpacking filebeat (8.19.16) ...
Progress: [20%] [



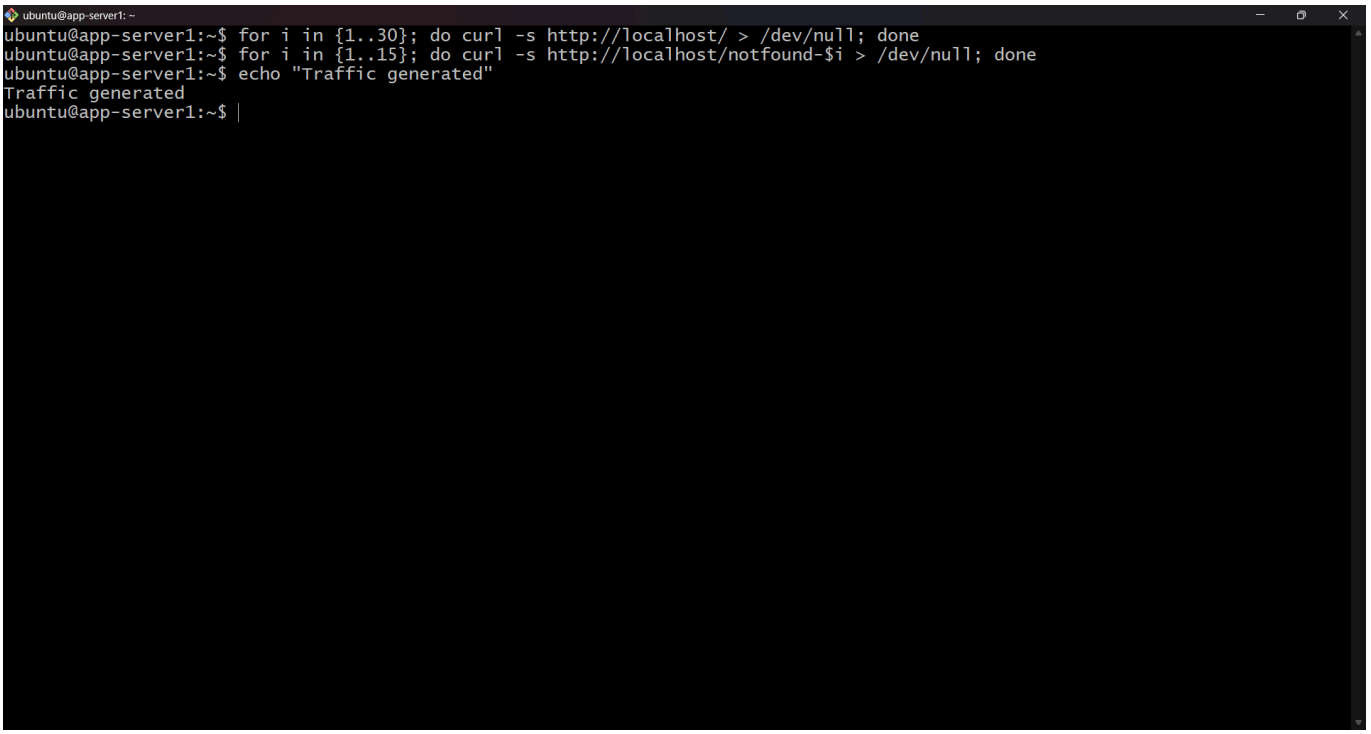


Part 6 — Verify Log Pipeline

Generate test traffic on both app servers and confirm logs are flowing through to Elasticsearch.

```
# Generate normal requests (run on both app servers)
for i in {1..50}; do curl -s http://localhost/ > /dev/null; done

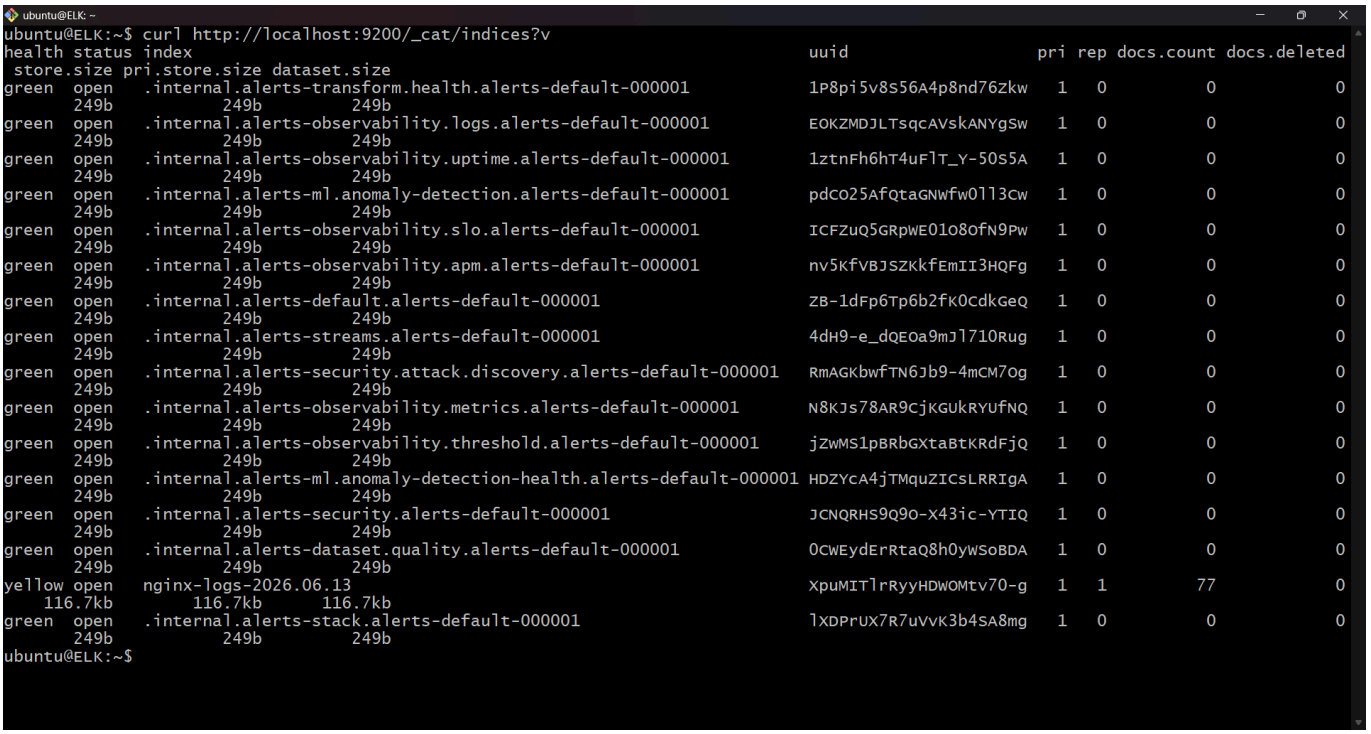
# Generate 404 errors
for i in {1..20}; do curl -s http://localhost/notfound-$i > /dev/null; done
```

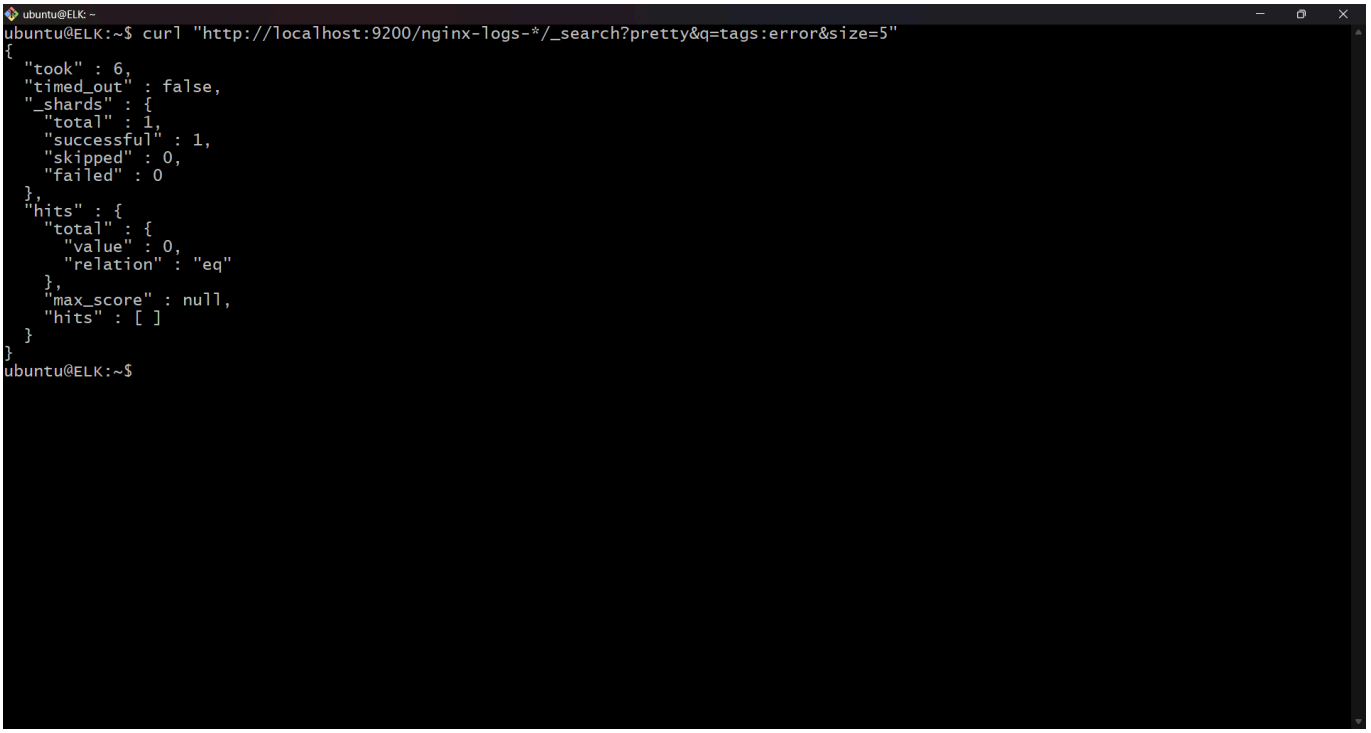


Verify on ELK server:

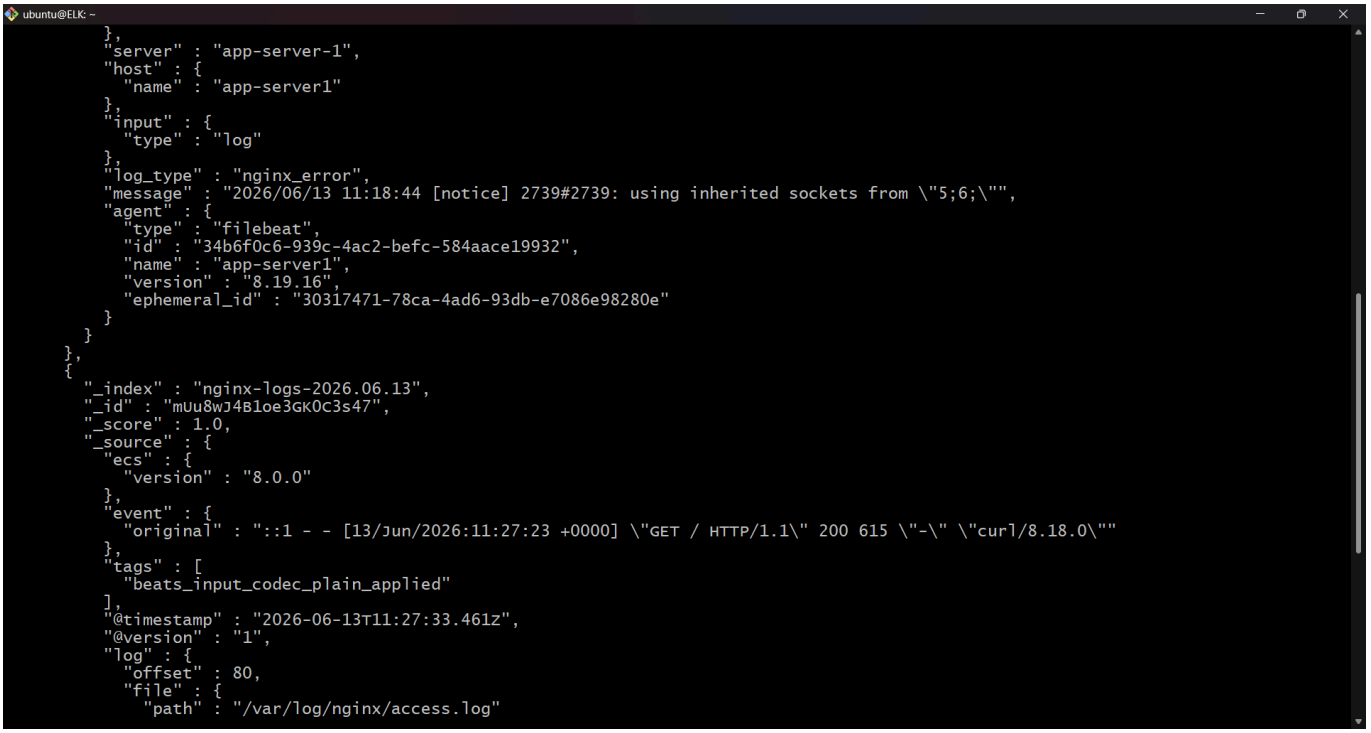
```
# Check Elasticsearch index was created
curl http://localhost:9200/_cat/indices?v

# Confirm parsed fields exist
curl "http://localhost:9200/nginx-logs-*/_search?pretty&size=1" | \
  grep -E "client_ip|response_code|method"
```


ubuntu@ELK: ~
ubuntu@ELK:~$ curl http://localhost:9200/_cat/indices?v
health status index uuid pri rep docs.count docs.deleted
store.size pri.store.size dataset.size
green open .internal.alerts-transform.health.alerts-default-000001 1P8pi5v8S56A4p8nd76Zkw 1 0 0 0
249b 249b 249b
green open .internal.alerts-observability.logs.alerts-default-000001 EOKZMDJLTsqcAVskANYgSw 1 0 0 0
249b 249b 249b
green open .internal.alerts-observability.uptime.alerts-default-000001 1ztnFh6hT4uFlT_Y-5OS5A 1 0 0 0
249b 249b 249b
green open .internal.alerts-ml.anomaly-detection.alerts-default-000001 pdCO25AfQtaGNWfw0ll3Cw 1 0 0 0
249b 249b 249b
green open .internal.alerts-observability.slo.alerts-default-000001 ICFZuQ5GRpWE01o8OfN9Pw 1 0 0 0
249b 249b 249b
green open .internal.alerts-observability.apm.alerts-default-000001 nv5KfVBJSZkkfEmII3HQFg 1 0 0 0
249b 249b 249b
green open .internal.alerts-default.alerts-default-000001 ZB-1dFp6Tp6b2fkOCdkGeQ 1 0 0 0
249b 249b 249b
green open .internal.alerts-streams.alerts-default-000001 4dH9-e_dQEOa9mJl71ORug 1 0 0 0
249b 249b 249b
green open .internal.alerts-security.attack.discovery.alerts-default-000001 RmAGKbwfTN6Jb9-4mCM7Og 1 0 0 0
249b 249b 249b
green open .internal.alerts-observability.metrics.alerts-default-000001 N8KJs78AR9CjKGUkRYUfNQ 1 0 0 0
249b 249b 249b
green open .internal.alerts-observability.threshold.alerts-default-000001 jZwMS1pBRbGXtaBtKRdFjQ 1 0 0 0
249b 249b 249b
green open .internal.alerts-ml.anomaly-detection-health.alerts-default-000001 HDZYcA4jTMquZICsLRRIgA 1 0 0 0
249b 249b 249b
green open .internal.alerts-security.alerts-default-000001 JCNQRHS9Q9O-X43ic-YTIQ 1 0 0 0
249b 249b 249b
green open .internal.alerts-dataset.quality.alerts-default-000001 OCWEydErRtaQ8hOyWSoBDA 1 0 0 0
249b 249b 249b
yellow open nginx-logs-2026.06.13 XpuMITlrRyyHDWOMtv7O-g 1 1 77 0
116.7kb 116.7kb 116.7kb
green open .internal.alerts-stack.alerts-default-000001 lXDPrUX7R7uVvK3b4sA8mg 1 0 0 0
249b 249b 249b
ubuntu@ELK:~$


ubuntu@ELK: ~
ubuntu@ELK:~$ curl "http://localhost:9200/nginx-logs-*/_search?pretty&q=tags:error&size=5"
{
"took" : 6,
"timed_out" : false,
"_shards" : {
"total" : 1,
"successful" : 1,
"skipped" : 0,
"failed" : 0
},
"hits" : {
"total" : {
"value" : 0,
"relation" : "eq"
},
"max_score" : null,
"hits" : [ ]
}
}
ubuntu@ELK:~$





Part 7 — Kibana Dashboard

Create the Nginx Monitoring Dashboard with 4 visualizations in Kibana Lens.

Visualization 1 — Error Rate Over Time

- Chart type: Line
- Horizontal axis: @timestamp
- Vertical axis: Count of records
- Filter: `tags : "error"`
- Shows: 4xx and 5xx error count over time

Visualization 2 — Requests Per Minute

- Chart type: Area
- Horizontal axis: @timestamp
- Vertical axis: Count of records
- No filter — shows total traffic

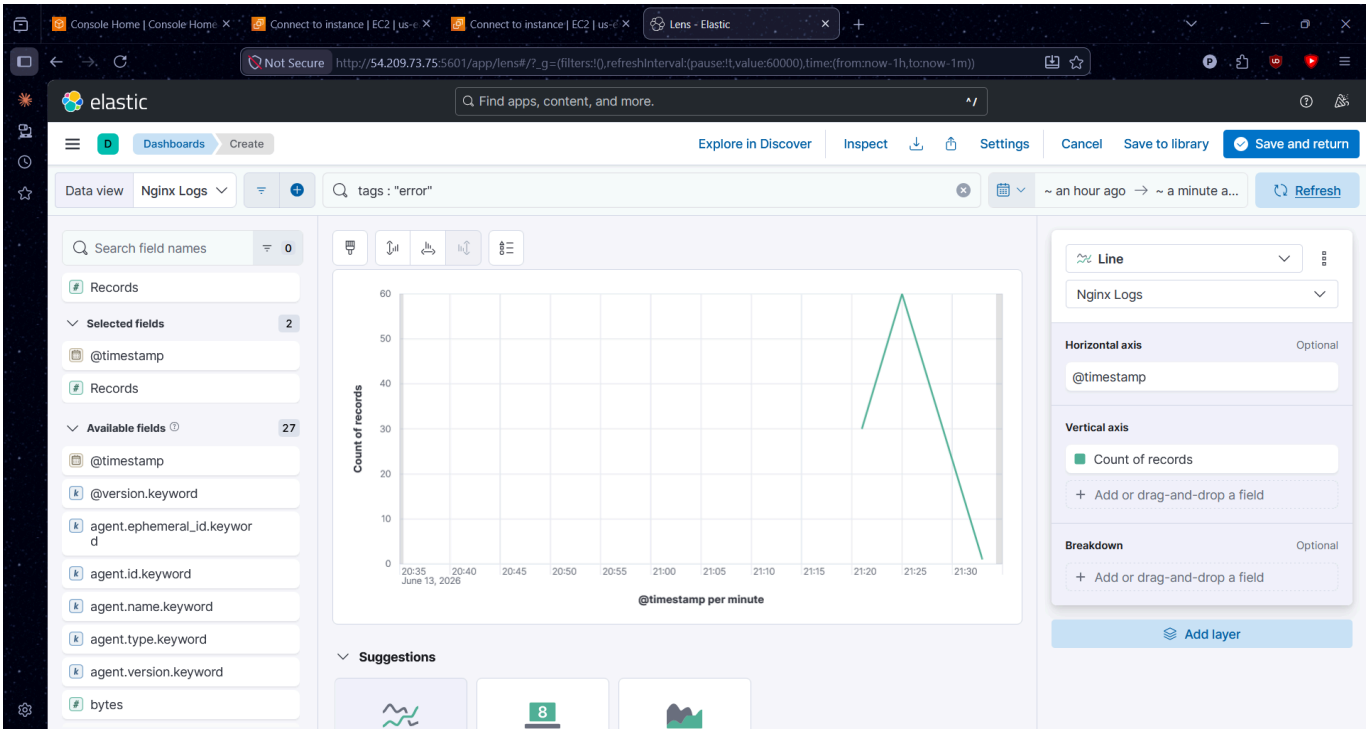
Visualization 3 — Top Client IPs

- Chart type: Pie
- Slice by: client_ip.keyword
- Metric: Count of records
- Shows: top 10 IP addresses making requests

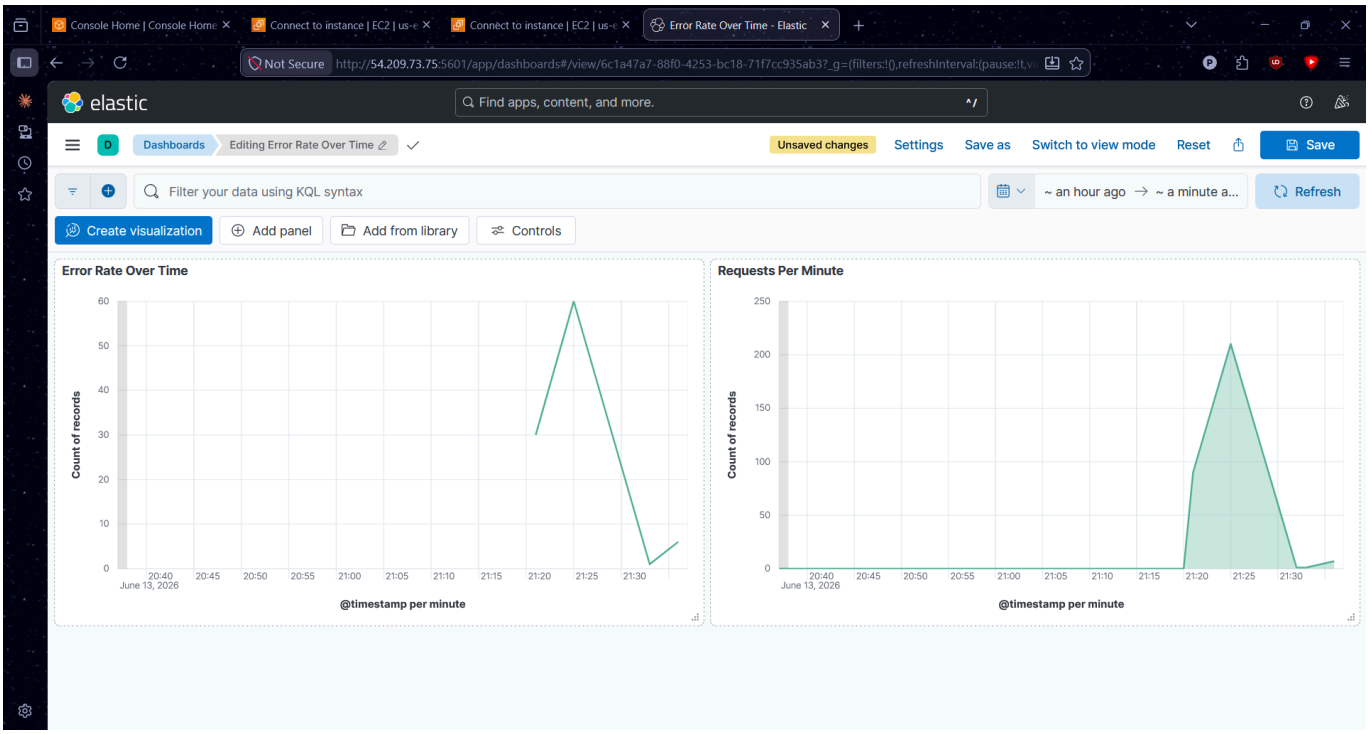
Visualization 4 — HTTP Response Code Breakdown

- Chart type: Pie
- Slice by: response_code
- Metric: Count of records
- Shows: distribution of 200, 404, 405 etc.

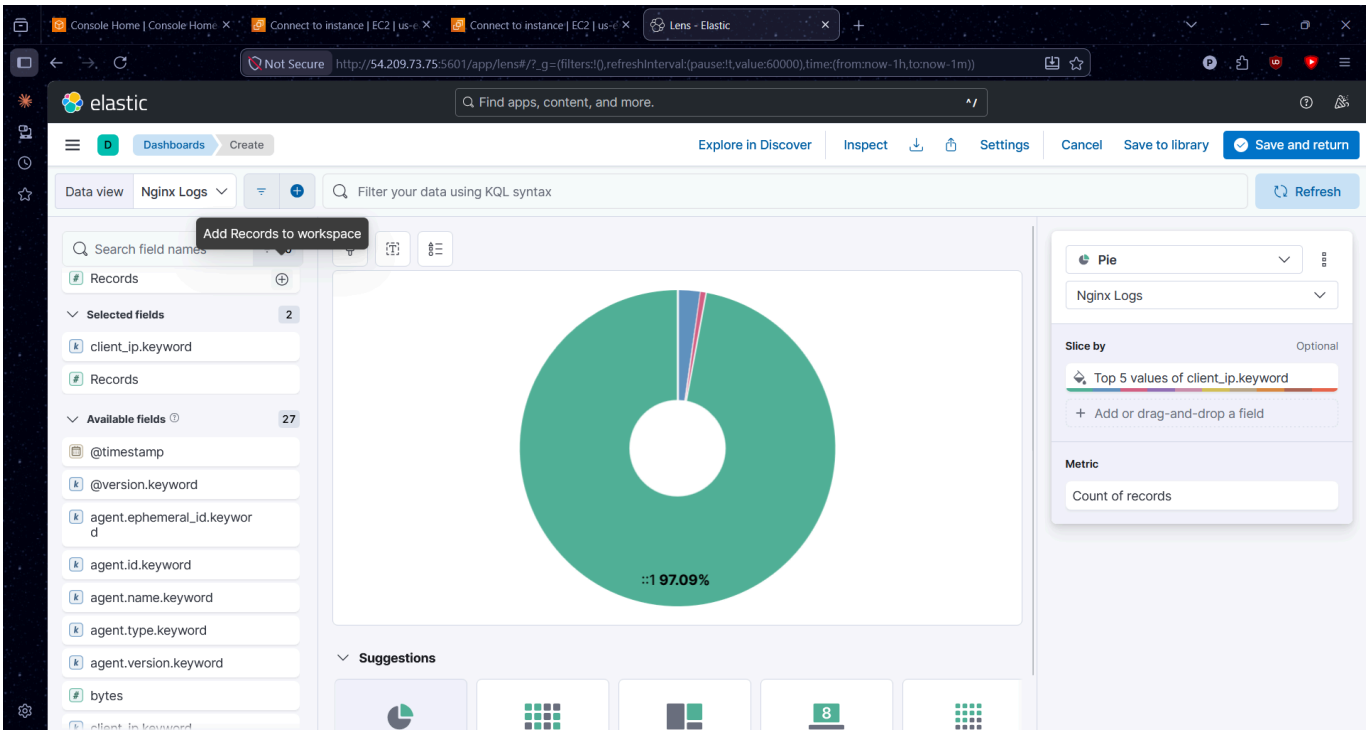
> **Error Rate Over Time** — line chart showing error spike when 404 traffic was generated.



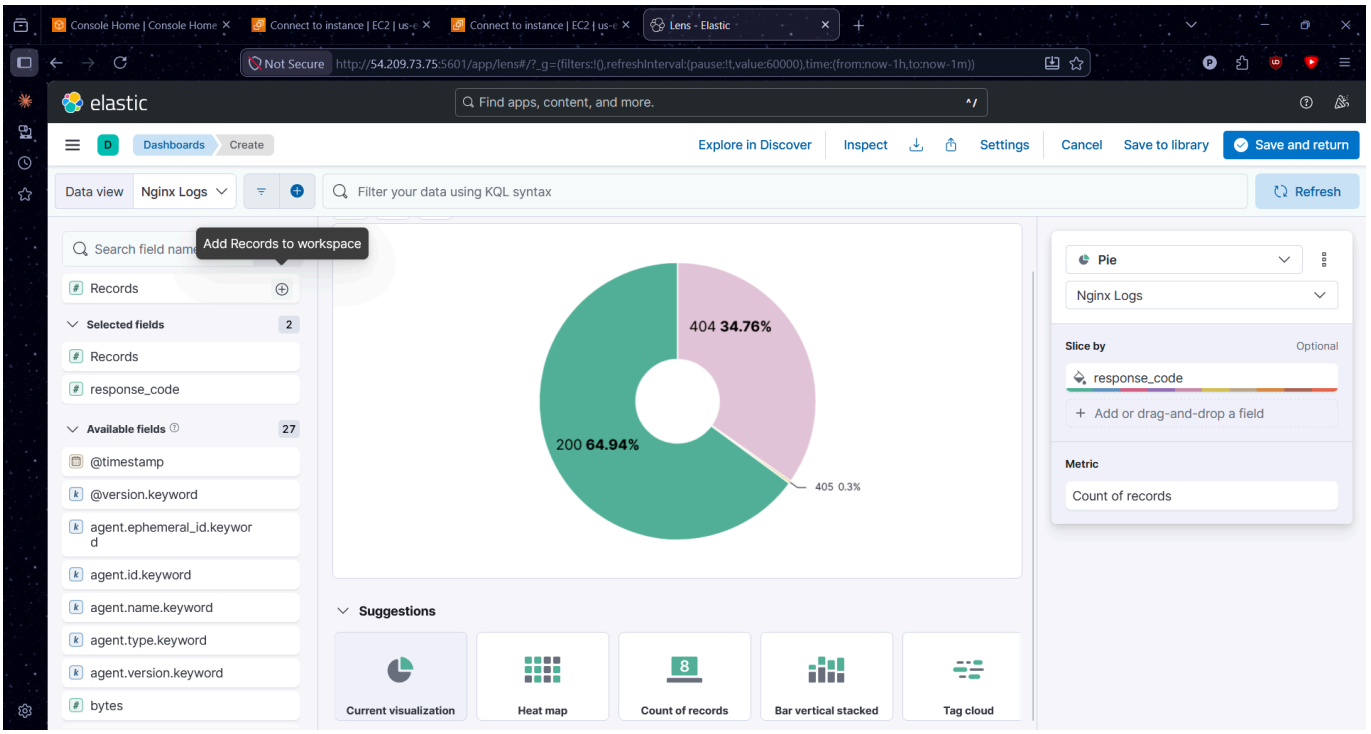
> **Requests Per Minute** — area chart showing total traffic volume over time.



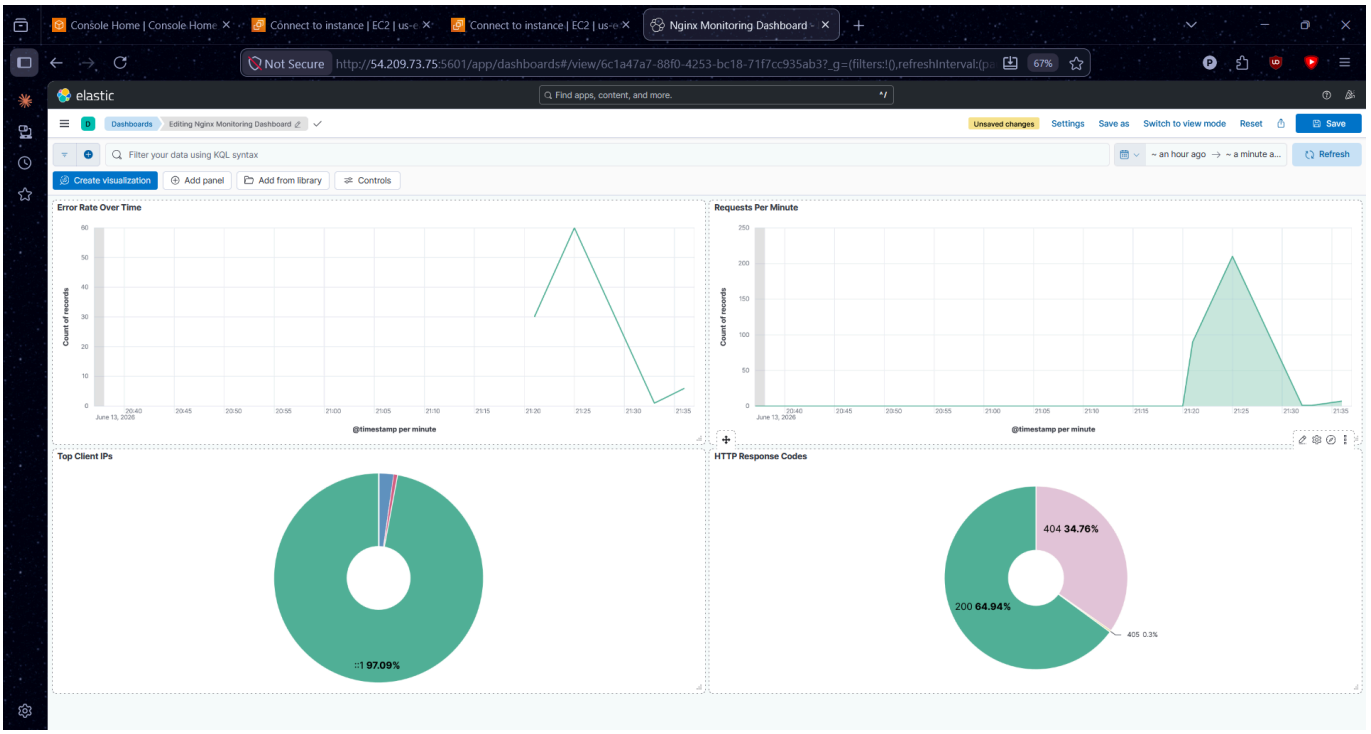
Top Client IPs — pie chart showing which IPs generated the most traffic.



HTTP Response Code Breakdown — pie chart showing 200: 64.94%, 404: 34.76%, 405: 0.3%.



Full Nginx Monitoring Dashboard — all 4 visualizations on one screen.



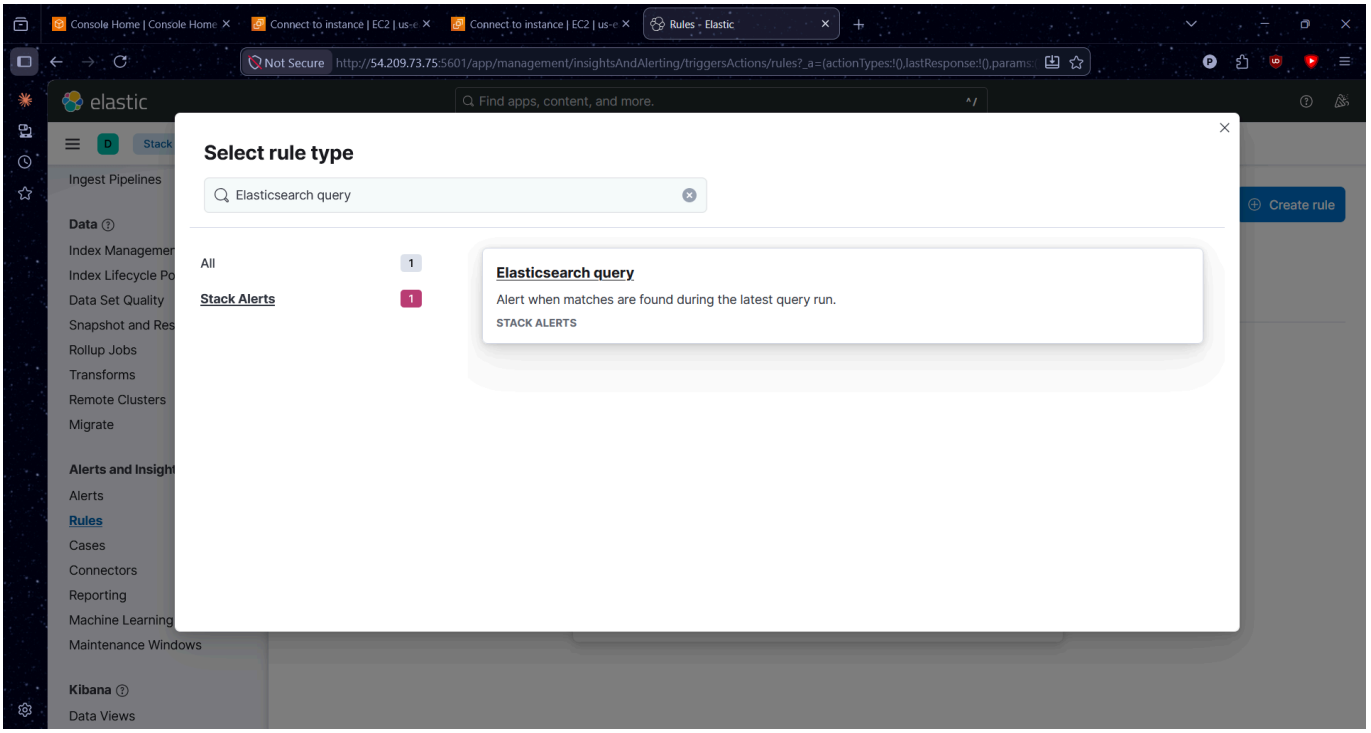
Part 8 — Alert Configuration

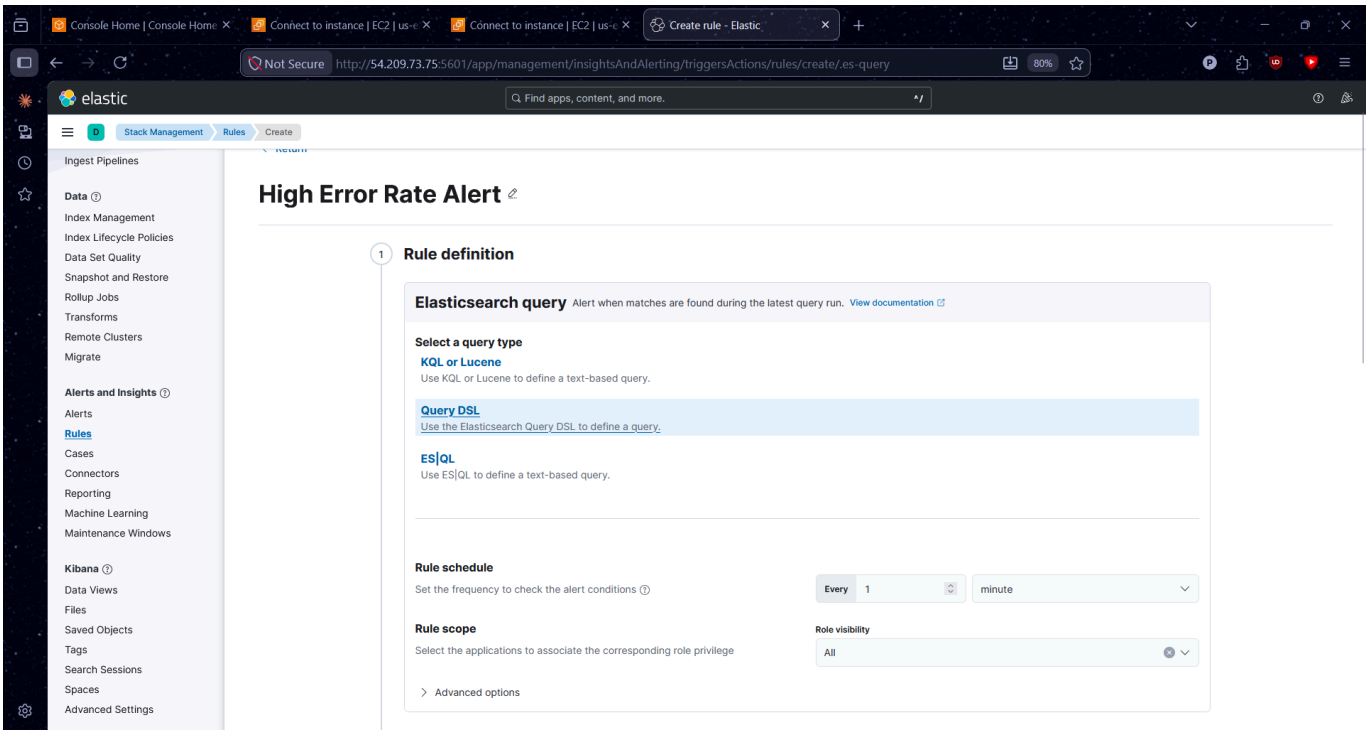
Create an alert rule in Kibana that fires automatically when the error count exceeds 10 in 5 minutes — enabling proactive error detection without manual log checking.

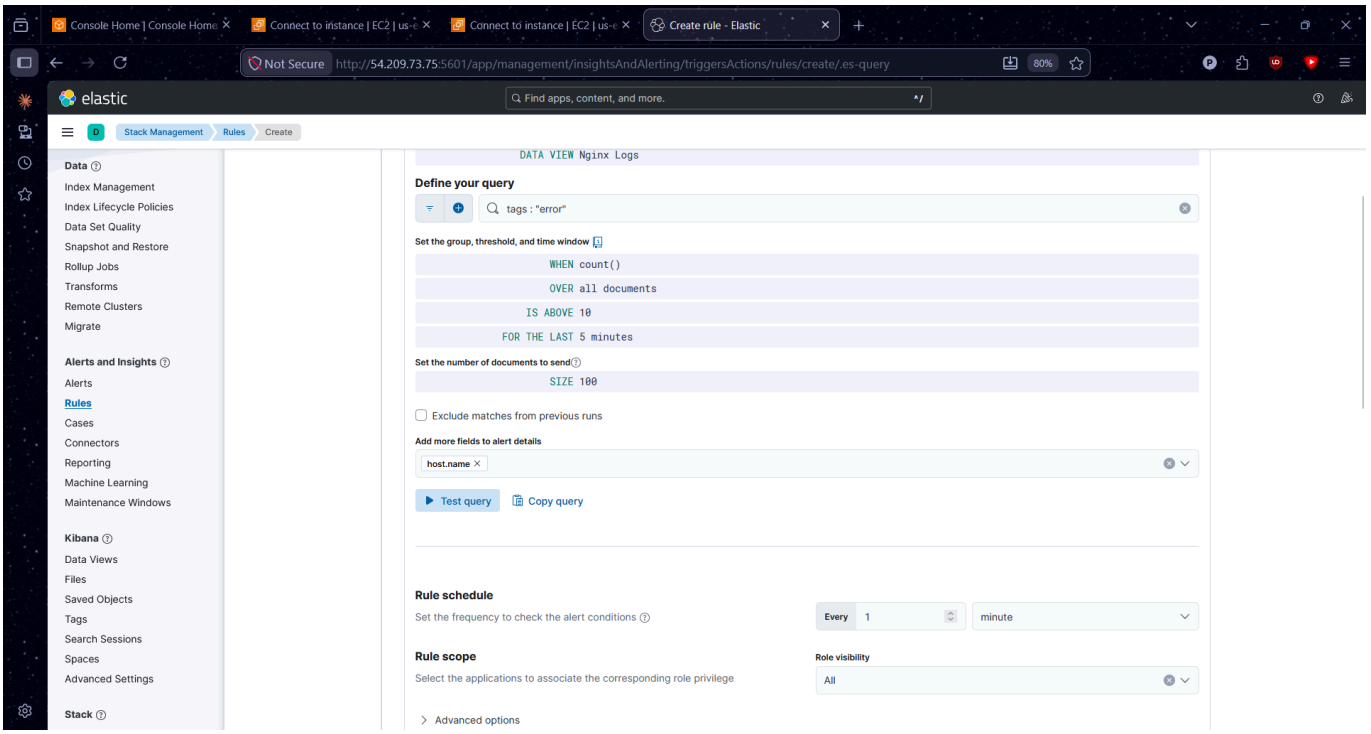
```
Stack Management → Rules → Create rule
Rule type: Elasticsearch query
Query type: KQL or Lucene
Data view: Nginx Logs
```

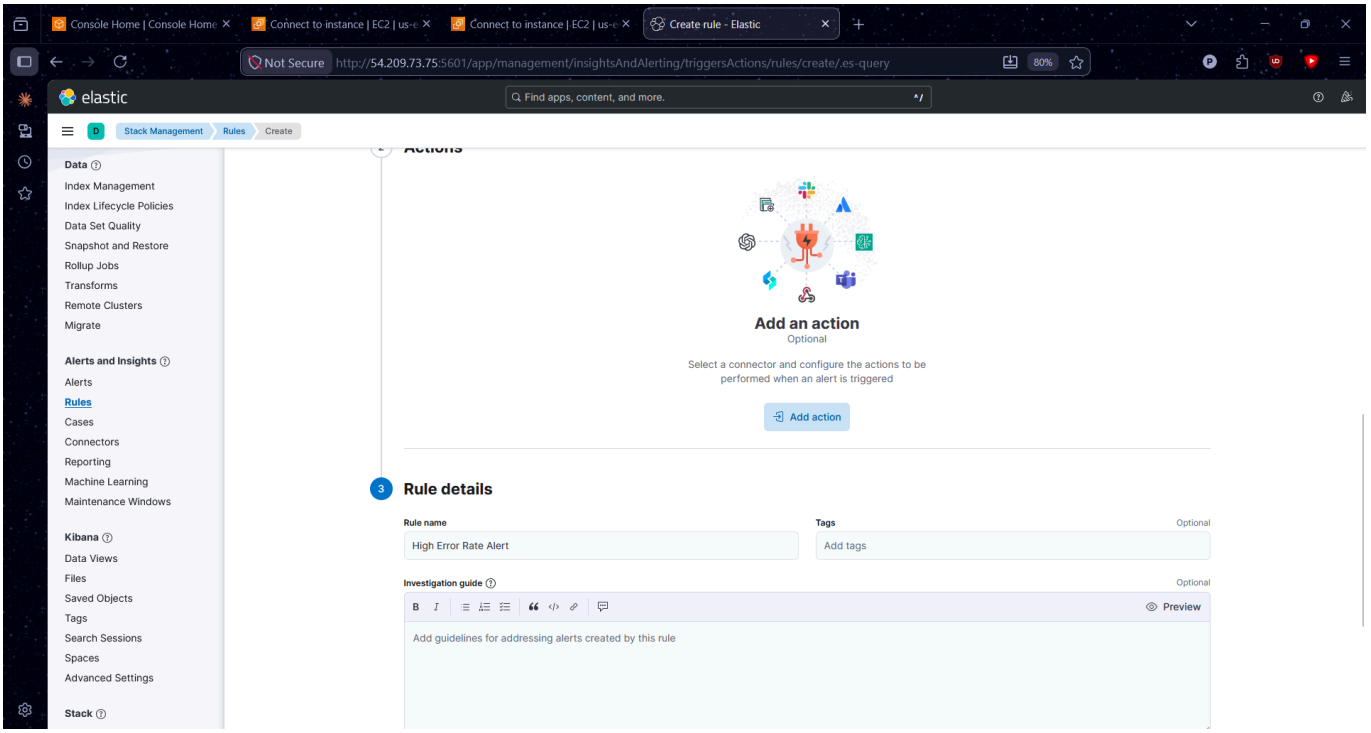
```
Query: tags : "error"
Condition: WHEN count() IS ABOVE 10 FOR THE LAST 5 minutes
Schedule: Every 1 minute
Rule name: High Error Rate Alert
```

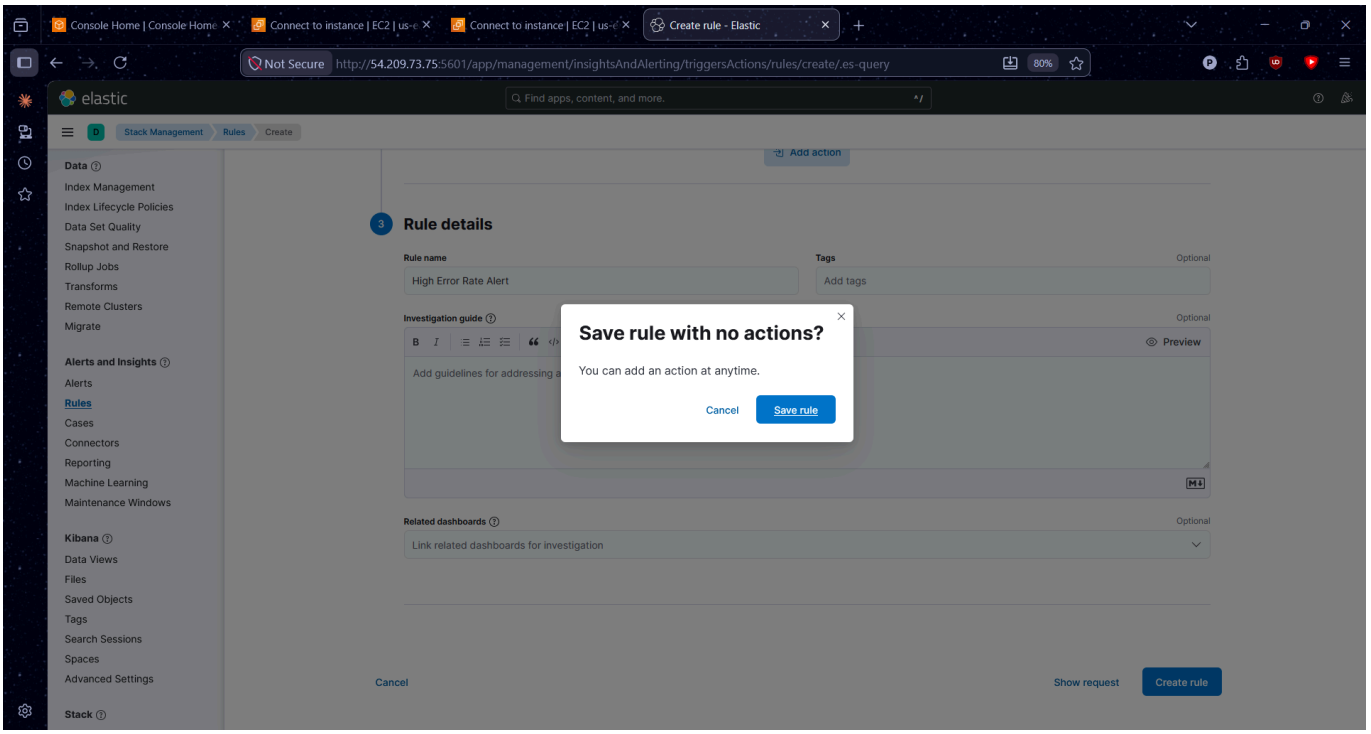
> **Alert rule creation page** — showing High Error Rate Alert with query and threshold configured.



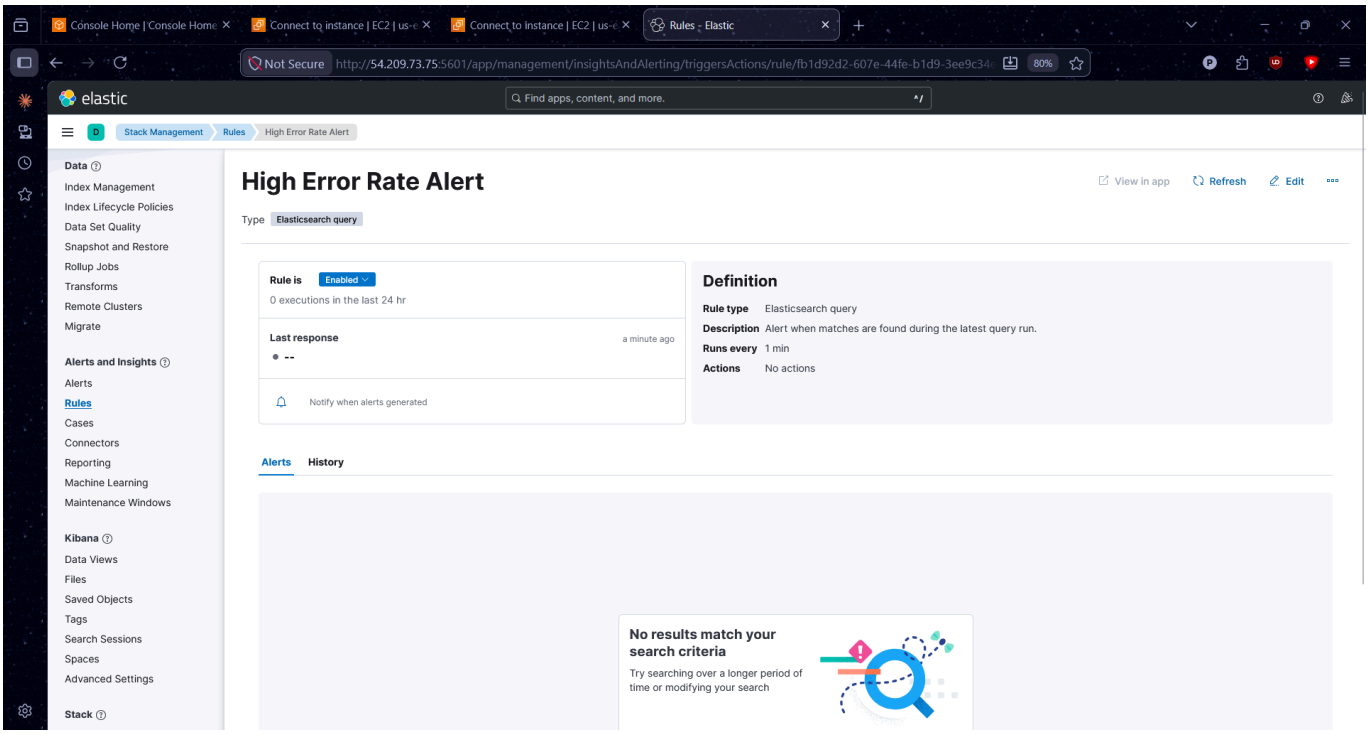


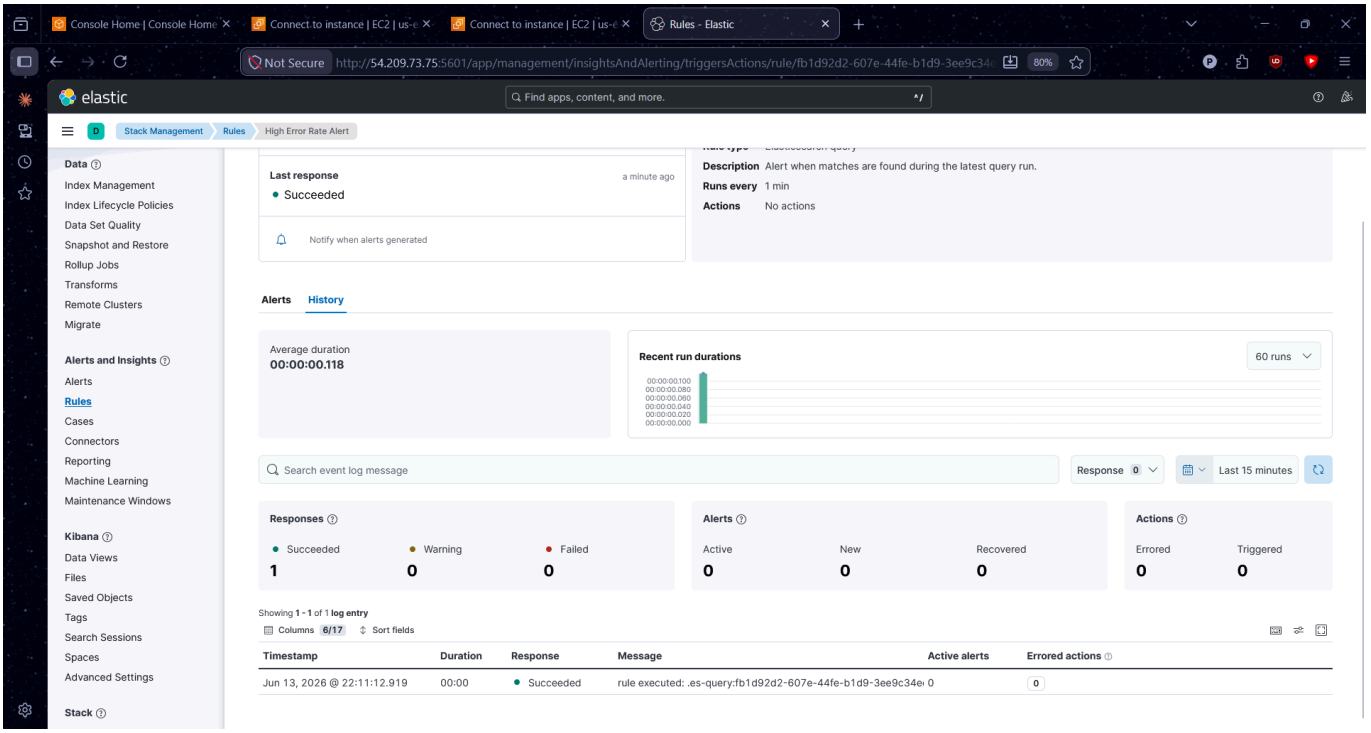

Console Home | Console Home
Connect to instance | EC2 | us-e
Connect to instance | EC2 | us-e
Create rule - Elastic
Not Secure
http://54.209.73.75:5601/app/management/insightsAndAlerting/triggersActions/rules/create/.es-query
80%
elastic
Find apps, content, and more.
Stack Management
Rules
Create
Data
Index Management
Index Lifecycle Policies
Data Set Quality
Snapshot and Restore
Rollup Jobs
Transforms
Remote Clusters
Migrate
Alerts and Insights
Alerts
Rules
Cases
Connectors
Reporting
Machine Learning
Maintenance Windows
Kibana
Data Views
Files
Saved Objects
Tags
Search Sessions
Spaces
Advanced Settings
Stack
DATA VIEW Nginx Logs
Define your query
tags : "error"
Set the group, threshold, and time window
WHEN count()
OVER all documents
IS ABOVE 10
FOR THE LAST 5 minutes
Set the number of documents to send
SIZE 100
Exclude matches from previous runs
Add more fields to alert details
host.name
Test query
Copy query
Rule schedule
Set the frequency to check the alert conditions
Every 1
minute
Rule scope
Select the applications to associate the corresponding role privilege
Role visibility
All
Advanced options


Console Home | Console Home
Connect to instance | EC2 | us-e
Connect to instance | EC2 | us-e
Create rule - Elastic
Not Secure
http://54.209.73.75:5601/app/management/insightsAndAlerting/triggersActions/rules/create/.es-query
80%
elastic
Find apps, content, and more.
Stack Management
Rules
Create
Data
Index Management
Index Lifecycle Policies
Data Set Quality
Snapshot and Restore
Rollup Jobs
Transforms
Remote Clusters
Migrate
Alerts and Insights
Alerts
Rules
Cases
Connectors
Reporting
Machine Learning
Maintenance Windows
Kibana
Data Views
Files
Saved Objects
Tags
Search Sessions
Spaces
Advanced Settings
Stack
Add an action
Optional
Select a connector and configure the actions to be performed when an alert is triggered
Add action
3
Rule details
Rule name
High Error Rate Alert
Tags
Optional
Add tags
Investigation guide
Optional
Preview
Add guidelines for addressing alerts created by this rule



Rules list — showing High Error Rate Alert saved with status OK and last run time.





💡 What I Learned

Probably the biggest thing I got out of this project was actually understanding what each part of ELK is for, instead of just knowing the acronym. Filebeat is basically a lightweight log shipper — it collects logs and forwards them. Logstash is where the processing happens, including parsing and transforming the logs. Elasticsearch stores and indexes the data so it can be searched efficiently.

Grok patterns took me a while to understand. You're essentially using pattern matching to break an unstructured Nginx log line into individual fields that can be queried and filtered later. Once it clicked, it made a lot more sense, but the syntax definitely wasn't intuitive at first.

Kibana Lens was another useful discovery. I expected to need a query language just to build basic charts, but a lot of it can be done through drag-and-drop. Selecting fields and putting them on axes or into slices is enough to build things like line charts and pie charts from the log data. I also learned an important networking lesson while testing the setup. I initially had Filebeat sending logs to the Logstash public IP. It worked, but since both servers were inside the same VPC, using the private IP made more sense for internal communication and avoided exposing the Logstash endpoint publicly.

Elasticsearch and memory were another lesson. I initially put Elasticsearch on a `t2.small`, thinking it would be enough for a small test setup, but it kept getting killed because of memory pressure. I had to increase the instance resources and configure the JVM heap properly — in my setup, `512MB` on a `c7i-flex.large` — before it became stable.

Kibana alerting was fairly straightforward once I got to it. You define a rule, have it evaluate the Elasticsearch data on a schedule, and trigger an alert when a condition or threshold is met. It's useful because the dashboard doesn't have to be watched constantly just to catch an increase in errors.

Project Summary

This project started with a simple problem: when something goes wrong in production, how do you quickly figure out what happened?

If the logs are spread across multiple servers, troubleshooting means connecting to each machine, searching through different log files, and trying to piece the events together. The more servers you have, the harder that becomes.

The goal of this project was to bring those logs into one place using the ELK Stack. Filebeat collects the Nginx logs, Logstash processes and parses them, Elasticsearch stores and indexes them, and Kibana provides a central place to search and visualize the data.

One of the more interesting parts was getting the Logstash Grok pattern right. Nginx logs start as plain text, with information such as the client IP, timestamp, HTTP method, URL, response code, and response size all contained in a single line. The Grok pattern breaks that line into separate fields so Elasticsearch can work with the data properly. Once the logs are structured, I can filter for things like `response_code: 404`, look at requests by `client_ip`, or identify logs tagged as errors.

I also added alerting so the system doesn't depend on someone constantly watching the Kibana dashboard. The alert checks the log data periodically and can trigger when the error rate crosses a defined threshold.

The main thing I wanted to understand was the complete flow — from a request generating a log on an EC2 server, through Filebeat and Logstash, into Elasticsearch, and finally into Kibana where the data can be searched, visualized, and monitored.

Conclusion

Building this project made it pretty clear why centralized logging matters once you've got more than a couple of servers running. Instead of hopping between machines and digging through separate log files, everything ends up in one place where you can actually search and analyze it.

What helped more than anything was working through the whole pipeline instead of learning each ELK component in isolation. Watching a log go from Filebeat, get parsed by Logstash, land in Elasticsearch, and finally show up in Kibana made the relationship between all four pieces much clearer than reading about them separately.

The Grok patterns were probably the part I spent the most time on. That's not something you fully understand from theory alone. The same goes for figuring out why Elasticsearch kept crashing until I sorted out the memory settings, and learning why using private IPs between instances made more sense than defaulting to public IPs.

If there's one thing I'd take away from this project, it's that logs become much more useful when they're structured and centralized. Having unstructured log files sitting on different servers isn't very helpful when something breaks and you need to figure out what happened quickly.

👷 Author

Premkumar Patil Cloud & DevOps Enthusiast | Learning AWS and SRE

LinkedIn Connect Medium Follow GitHub Follow

📌 Tags

AWS ELK Stack Elasticsearch Logstash Kibana Filebeat Nginx Log Aggregation Centralized Logging Monitoring Alerting Grok SRE DevOps Ubuntu EC2 Cloud Engineering